

The Haira Programming Language

Language Specification

Version 1.0 Draft

Haira Team

February 12, 2026

© 2024-2026 Haira Team

This specification is licensed under CC BY 4.0.

Document Version: 1.0 Draft

Language Version: 1.0

Last Updated: February 12, 2026

Contents

Preface	13
I Core Language	15
1 Overview	17
1.1 Hello, Agent	17
1.2 What Haira Replaces	18
1.3 Language Characteristics	18
1.3.1 Four Agentic Primitives	18
1.3.2 Explicit Imports	18
1.3.3 Go-Style Error Handling	18
1.3.4 Agents and Tools	19
1.3.5 Workflows with Triggers	19
1.3.6 Streaming and Conversations	20
1.4 Type System	20
1.5 Control Flow	20
1.6 Functions	21
1.7 Concurrency	21
1.8 Pipe Operator	22
1.9 A Complete Example	22
1.10 Implementation Status	23
2 Lexical Structure	25
2.1 Source Encoding	25
2.2 Line Structure	25
2.2.1 Line Continuation	25
2.3 Comments	26
2.3.1 Single-Line Comments	26
2.3.2 Multi-Line Comments	26
2.3.3 Documentation Comments	26
2.4 Tokens	26
2.5 Keywords	26
2.6 Identifiers	27
2.6.1 Naming Conventions	27
2.7 Literals	27
2.7.1 Integer Literals	27
2.7.2 Floating-Point Literals	28
2.7.3 Boolean Literals	28
2.7.4 String Literals	29
2.7.5 String Interpolation	29

2.7.6	Raw Strings	29
2.7.7	Triple-Quoted Strings	30
2.7.8	Nil Literal	30
2.8	Operators	30
2.8.1	Arithmetic Operators	30
2.8.2	Comparison Operators	31
2.8.3	Logical Operators	31
2.8.4	Assignment Operators	31
2.8.5	Bitwise Operators	31
2.8.6	Bitwise Assignment Operators	31
2.8.7	Other Operators	31
2.9	Delimiters	31
2.10	Whitespace	31
2.11	Lexical Grammar Summary	31
3	Types	35
3.1	Type System Overview	35
3.2	Primitive Types	35
3.2.1	Integer Types	35
3.2.2	Floating-Point Types	36
3.2.3	Boolean Type	36
3.2.4	String Type	36
3.3	Composite Types	36
3.3.1	Arrays	36
3.3.2	Maps	37
3.3.3	Tuples	37
3.4	Option Types	38
3.4.1	Unwrapping Options	38
3.5	Struct Types	38
3.5.1	Struct Methods	39
3.5.2	Nested Structs	39
3.6	Enum Types	40
3.6.1	Enums with Associated Data	40
3.7	Type Aliases	41
3.8	Function Types	41
3.9	Generic Types	41
3.9.1	Type Constraints	42
3.10	Stream Types	42
3.10.1	Stream Type Syntax	42
3.10.2	Consuming Streams	43
3.10.3	Streams as Return Types	43
3.10.4	Stream Properties	43
3.11	The Error Type	43
3.12	Type Inference	44
3.13	Structural Typing	44
3.14	Type Grammar	45
4	Expressions	47
4.1	Primary Expressions	47
4.1.1	Literals	47
4.1.2	Identifiers	47
4.1.3	Parenthesized Expressions	47

4.2	Arithmetic Expressions	47
4.2.1	Integer Division	48
4.2.2	Modulo	48
4.3	Comparison Expressions	48
4.3.1	Equality	48
4.4	Logical Expressions	48
4.4.1	Short-Circuit Evaluation	48
4.5	Bitwise Expressions	49
4.5.1	Bitwise AND	49
4.5.2	Bitwise OR	49
4.5.3	Bitwise XOR	49
4.5.4	Bitwise NOT	49
4.5.5	Left Shift	50
4.5.6	Right Shift	50
4.5.7	Logical Right Shift	50
4.5.8	Common Bit Manipulation Patterns	50
4.5.9	Compound Bitwise Assignment	51
4.6	String Expressions	51
4.6.1	Concatenation	51
4.6.2	String Interpolation	51
4.7	Array and Map Expressions	52
4.7.1	Array Literals	52
4.7.2	Map Literals	52
4.7.3	Index Access	52
4.8	Field Access	52
4.9	Function Calls	52
4.9.1	Chained Calls	52
4.10	Pipe Expressions	53
4.10.1	Pipe with Arguments	53
4.10.2	Complex Pipelines	53
4.11	Range Expressions	53
4.12	Conditional Expressions	54
4.12.1	If Expression	54
4.12.2	Match Expression	54
4.13	Block Expressions	54
4.14	Struct Instantiation	54
4.15	Closure Expressions	55
4.16	Option Expressions	55
4.16.1	Nil Coalescing	55
4.16.2	Optional Chaining	55
4.17	Type Assertions	55
4.18	Operator Precedence	55
4.19	Expression Grammar	56
5	Statements	59
5.1	Variable Declarations	59
5.1.1	With Inference	59
5.1.2	With Explicit Type	59
5.1.3	Multiple Declarations	59
5.1.4	Uninitialized Variables	59
5.2	Assignment Statements	59
5.2.1	Multiple Assignment	60

5.2.2	Field Assignment	60
5.2.3	Index Assignment	60
5.3	Expression Statements	60
5.4	If Statements	60
5.4.1	Condition Binding	61
5.5	For Statements	61
5.5.1	Range-Based For	61
5.5.2	Collection Iteration	61
5.5.3	String Iteration	61
5.6	While Statements	62
5.7	Match Statements	62
5.7.1	Basic Matching	62
5.7.2	Multiple Patterns	62
5.7.3	Range Patterns	62
5.7.4	Guard Clauses	63
5.7.5	Destructuring Patterns	63
5.7.6	Match Exhaustiveness	63
5.8	Block Statements	64
5.9	Return Statements	64
5.9.1	Implicit Return	64
5.10	Break Statements	64
5.10.1	Labeled Break	65
5.11	Continue Statements	65
5.11.1	Labeled Continue	65
5.12	Import Statements	65
5.13	Defer Statements	66
5.13.1	Multiple Defers	66
5.14	Statement Grammar	66
6	Functions	69
6.1	Function Declarations	69
6.1.1	Basic Examples	69
6.1.2	No Return Type	69
6.2	Parameters	70
6.2.1	Required Parameters	70
6.2.2	Default Parameters	70
6.2.3	Named Arguments	70
6.2.4	Variadic Parameters	70
6.3	Return Values	71
6.3.1	Single Return	71
6.3.2	Multiple Returns	71
6.3.3	Implicit Return	71
6.3.4	Early Return	71
6.4	Function Types	71
6.5	Closures	72
6.5.1	Closure Syntax	72
6.5.2	Capture Semantics	72
6.6	Methods	73
6.6.1	Method Receiver	74
6.7	Generic Functions	74
6.7.1	Type Constraints	74
6.8	Higher-Order Functions	75

6.9	Recursion	75
6.9.1	Tail Call Optimization	75
6.10	Function Overloading	76
6.11	The main Function	76
6.12	Function Grammar	76
7	Error Handling	77
7.1	Philosophy	77
7.2	The Error Type	77
7.2.1	Creating Errors	77
7.3	Error Return Pattern	78
7.4	Handling Errors	78
7.4.1	Basic Error Checking	78
7.4.2	Propagating Errors	78
7.4.3	Wrapping Errors	78
7.5	Error Checking Patterns	79
7.5.1	Guard Pattern	79
7.5.2	Defer for Cleanup	79
7.6	Error Utilities	80
7.6.1	Checking Error Types	80
7.6.2	Error Chain Traversal	80
7.7	Option Types and Errors	81
7.7.1	Converting Between Options and Errors	81
7.8	Panic and Recovery	81
7.8.1	When to Use Panic	82
7.8.2	When to Use Errors	82
7.9	Best Practices	82
7.9.1	Return Errors, Don't Log and Continue	82
7.9.2	Add Context When Wrapping	83
7.9.3	Handle Errors at the Right Level	83
7.10	Error Handling Grammar	83
II	Concurrency and Modules	85
8	Concurrency	87
8.1	Concurrency Model	87
8.2	Spawn	87
8.2.1	Spawning Functions	87
8.2.2	Spawn Returns Handle	88
8.3	Channels	88
8.3.1	Sending and Receiving	88
8.3.2	Basic Channel Example	88
8.3.3	Buffered Channels	89
8.3.4	Closing Channels	89
8.3.5	Checking Channel State	89
8.4	Select	90
8.4.1	Select with Default	90
8.4.2	Select with Timeout	90
8.4.3	Select in Loop	91
8.5	Common Patterns	91
8.5.1	Worker Pool	91

8.5.2	Fan-Out, Fan-In	92
8.5.3	Pipeline	92
8.5.4	Semaphore	93
8.6	Channel Types	94
8.6.1	Directional Channels	94
8.7	Synchronization	95
8.7.1	WaitGroup	95
8.7.2	Mutex	95
8.8	Concurrency Grammar	96
9	Modules and Imports	97
9.1	Module System Overview	97
9.2	Import Syntax	97
9.2.1	Basic Import	97
9.2.2	Aliased Import	97
9.2.3	Selective Import	98
9.2.4	Glob Import	98
9.3	Module Resolution	98
9.3.1	Standard Library	98
9.3.2	Project-Local Imports	99
9.3.3	External Packages	99
9.4	Module Definitions	99
9.4.1	File as Module	99
9.4.2	Exporting	99
9.4.3	Package Initialization	100
9.5	Package Structure	100
9.5.1	Single-File Package	100
9.5.2	Multi-File Package	100
9.5.3	The mod.haira File	101
9.6	Import Order	101
9.7	Circular Dependencies	101
9.8	Conditional Imports	102
9.9	The haira.toml File	102
9.10	Module Grammar	102
10	Standard Library	105
10.1	Core Modules	105
10.2	io Module	105
10.2.1	io Functions	105
10.3	string Module	106
10.3.1	string Functions	106
10.4	array Module	107
10.5	map Module	108
10.6	math Module	108
10.7	conv Module	109
10.8	Extended Modules	110
10.8.1	fs Module	110
10.8.2	os Module	110
10.8.3	time Module	111
10.8.4	json Module	111
10.8.5	http Module	112
10.8.6	regex Module	112

10.8.7 crypto Module	113
10.8.8 net Module	113
10.9 Testing Module	113
III Agentic Orchestration	115
11 Providers	117
11.1 Provider Declaration	117
11.1.1 Cloud Providers	117
11.1.2 Local Providers	117
11.2 Provider Fields	118
11.3 Environment Variables	118
11.4 Referencing Providers	118
11.5 Multiple Providers	118
11.6 Provider Grammar	119
12 Tools	121
12.1 Tool Declaration	121
12.2 Basic Examples	121
12.2.1 HTTP Tool	121
12.2.2 Database Tool	122
12.2.3 External Service Tool	122
12.2.4 Agent-Delegating Tool	122
12.3 Triple-Quote Description	122
12.4 Auto-Generated JSON Schema	123
12.5 Tool Packs (Standard Library)	123
12.6 Error Handling in Tools	124
12.7 Tool Grammar	124
13 Agents	125
13.1 Agent Declaration	125
13.2 Agent Fields	125
13.2.1 Multiline System Prompts	125
13.3 Agent Methods	126
13.3.1 .ask() — Text In, Text Out	126
13.3.2 .run() — Structured In, Structured Out	126
13.3.3 .stream() — Text In, Streaming Out	126
13.4 Memory	127
13.4.1 No Memory (Default)	127
13.4.2 Conversation Memory	127
13.4.3 Summary Memory	127
13.4.4 Memory Types	128
13.5 Sessions	128
13.6 Tool-Calling Loop	128
13.7 Error Handling	129
13.8 Examples	129
13.8.1 Stateless Classifier	129
13.8.2 Research Agent with Tools	130
13.8.3 Conversational Assistant	130
13.9 Agent Grammar	130

14 Workflows	133
14.1 Workflow Declaration	133
14.2 Triggers	133
14.2.1 @webhook — HTTP Endpoint	133
14.2.2 @websocket — Bidirectional Streaming	134
14.2.3 @cron — Scheduled Execution	134
14.2.4 @event — Event-Driven	134
14.2.5 @manual — CLI Only	134
14.2.6 No Trigger (Sub-Workflow)	135
14.3 Sequential Steps	135
14.4 Parallel Execution	135
14.5 Branching	136
14.6 Loops	136
14.7 Sub-Workflows	137
14.8 Error Handling	137
14.9 Running Workflows	138
14.9.1 From main()	138
14.9.2 Server Mode	138
14.9.3 CLI	138
14.10 Workflow Grammar	138
15 Chatbot Patterns	139
15.1 Simple REST Chatbot	139
15.2 Streaming WebSocket Chatbot	140
15.3 Dual-Mode Chatbot	140
15.4 RAG Chatbot	141
15.5 Multi-Agent Routing	142
15.6 Chatbot with Structured Actions	143
15.7 Best Practices	143
IV Implementation	145
16 Complete Grammar	147
16.1 Notation	147
16.2 Lexical Grammar	147
16.2.1 Characters	147
16.2.2 Whitespace and Comments	147
16.2.3 Identifiers	148
16.2.4 Keywords	148
16.2.5 Literals	148
16.2.6 Operators	149
16.2.7 Delimiters	149
16.3 Syntactic Grammar	149
16.3.1 Program Structure	149
16.3.2 Imports	150
16.3.3 Core Declarations	150
16.3.4 Attributes	151
16.3.5 Decorators	151
16.3.6 Provider Declarations	151
16.3.7 Tool Declarations	151
16.3.8 Agent Declarations	152

16.3.9 Workflow Declarations	152
16.3.10 Types	152
16.3.11 Statements	153
16.3.12 Expressions	155
16.3.13 Patterns	156
16.4 Grammar Summary	157
16.4.1 Entry Points	157
16.4.2 Precedence (High to Low)	157
16.4.3 Associativity	158
17 Compiler Architecture	159
17.1 Overview	159
17.2 Lexer	160
17.2.1 Token Types	160
17.2.2 Keywords	160
17.2.3 Lexer Implementation	161
17.3 Parser	162
17.3.1 AST Nodes	162
17.3.2 Agentic AST Nodes	162
17.3.3 Parsing Agentic Constructs	163
17.4 Resolver	164
17.4.1 Symbol Table	164
17.4.2 Provider Resolution	165
17.4.3 Tool Registration	165
17.4.4 Agent Validation	166
17.4.5 Workflow Trigger Validation	167
17.5 Type Checker	168
17.5.1 Type Representation	168
17.5.2 Agent Method Type Checking	169
17.5.3 Left-Side Type Annotation Inference	170
17.5.4 Workflow Input/Output Types	170
17.5.5 Tool Auto-Schema Generation	170
17.6 Lowering	171
17.6.1 HIR Structure	171
17.6.2 Provider Lowering	172
17.6.3 Tool Lowering	172
17.6.4 Agent Lowering	173
17.6.5 Agent Method Call Lowering	173
17.6.6 Workflow Lowering	174
17.6.7 Spawn Block Lowering	175
17.7 Code Generation	175
17.7.1 Cranelift Integration	175
17.7.2 Runtime Calls	176
17.8 Runtime Library	177
17.8.1 Provider Management	177
17.8.2 Agent Execution	177
17.8.3 Workflow Orchestration	177
17.8.4 Session Management	178
17.8.5 Stream Handling	178
17.8.6 JSON Schema Generation	178
17.9 Project Structure	178
17.10 Build Process	179

17.10.1 Build Modes	179
17.10.2 Server Mode	180
17.10.3 Check Mode	180
17.11 Error Messages	180
17.11.1 Missing Tool Description	180
17.11.2 Agent Referencing Undefined Provider	181
17.11.3 Workflow with Invalid Trigger	181
17.11.4 Type Mismatch in Agent.run() Output	181
17.11.5 Agent Referencing Undefined Tool	181
17.11.6 Invalid Cron Expression	182
A Reserved Keywords	183
B Operator Precedence	185
C Standard Library Reference	187

Preface

HAIRA is a statically-typed, compiled programming language for building AI agents and workflows. It combines the simplicity of Go, the safety of Rust, and the expressiveness of modern functional languages, while making agents, tools, and workflows first-class language constructs—not library imports.

Design Philosophy

1. **Agents and workflows are primitives** – `provider`, `tool`, `agent`, and `workflow` are language keywords, not framework abstractions.
2. **Type-safe orchestration** – Agent inputs, outputs, and tool schemas are verified at compile time.
3. **Go-style simplicity** – Familiar error handling, explicit imports, and straightforward syntax.
4. **Zero boilerplate** – No framework setup, no dependency injection, no configuration files.
5. **Native performance** – Compiles to standalone native binaries via Cranelift.

Target Domains

HAIRA replaces the entire agentic stack:

- **Python + LangChain/LangGraph** – for code-first agent builders
- **n8n / Make / Zapier** – for workflow automation
- **CrewAI / AutoGen** – for multi-agent systems
- **Custom chatbot backends** – REST and WebSocket with streaming
- **General-purpose programming** – CLI tools, web services, data pipelines

Implementation Roadmap

This specification is organized so that each chapter can be implemented incrementally:

Release	Chapters	Features
v0.1	1–3	Lexer, parser, basic types
v0.2	4–5	Expressions, statements
v0.3	6–7	Functions, error handling

Release	Chapters	Features
v0.4	8	Concurrency primitives (spawn, chan, select)
v0.5	9–10	Modules, standard library
v0.6	11–12	Providers, tools
v0.7	13–14	Agents, workflows
v0.8	15	Chatbot patterns, streaming
v1.0	16–17	Complete grammar, compiler completion

Part I

Core Language

Chapter 1

Overview

HAIRA is a statically-typed, compiled programming language for building AI agents and workflows. It combines the simplicity of Go, the safety of Rust, and purpose-built primitives for agentic orchestration — agents, tools, workflows, and providers are first-class language constructs, not library imports.

HAIRA compiles to native binaries via Cranelift, with no runtime dependencies beyond the standard library.

1.1 Hello, Agent

```
1 import "io"
2
3 provider local {
4     backend: "ollama"
5     host: "localhost:11434"
6     model: "llama3:8b"
7 }
8
9 agent Greeter {
10     model: local
11     system: "You are a friendly greeter. Keep it short."
12 }
13
14 fn main() {
15     reply, err = Greeter.ask("Say hello to the world!")
16     if err != nil {
17         io.println("Error: " + err.message)
18         return
19     }
20     io.println(reply)
21 }
```

To compile and run:

```
$ haira build hello.haira
$ ./hello
Hello, world! Great to meet you!
```

1.2 What Haira Replaces

HAIRA is designed to replace the fragmented toolchain currently used to build agentic systems:

Current Stack	Haira Equivalent
Python + LangChain/LangGraph	agent + tool + workflow
n8n / Make / Zapier	workflow with @webhook, @cron triggers
CrewAI / AutoGen	Multi-agent workflow composition
FastAPI / Flask	<code>haira.serve()</code> with typed endpoints
<code>.env</code> files + API configs	provider blocks

Table 1.1: Haira replaces the fragmented agentic stack

1.3 Language Characteristics

1.3.1 Four Agentic Primitives

HAIRA adds four keywords to a general-purpose language:

- **provider** — Configure LLM backends (Anthropic, OpenAI, Ollama, local models)
- **tool** — Typed functions with LLM-visible descriptions; the compiler auto-generates JSON schemas
- **agent** — LLM-powered entities with a model, system prompt, tools, and optional memory
- **workflow** — Composable pipelines with triggers, typed I/O, and native concurrency

1.3.2 Explicit Imports

All dependencies are explicitly declared at the top of each file:

```

1 import "io"
2 import "json"
3 import "tools/http"
4 import "tools/postgres"

```

1.3.3 Go-Style Error Handling

Functions that can fail return a tuple of result and error:

```

1 import "io"
2 import "fs"
3
4 fn main() {
5     content, err = fs.read_file("config.json")
6     if err != nil {
7         io.println("Error: " + err.message)
8         return
9     }
10    io.println(content)
11 }

```

1.3.4 Agents and Tools

Agents are declared with a model, system prompt, and tools. Tools are typed functions with descriptions:

```
1 import "io"
2 import "tools/http"
3
4 provider anthropic {
5     api_key: env("ANTHROPIC_API_KEY")
6     model: "claude-sonnet-4-20250514"
7 }
8
9 tool search(query: string) -> [Result] {
10     ""Search the web for information""
11
12     resp, err = http.get("https://api.search.com?q={query}")
13     if err != nil { return [], err }
14     return json.decode(resp.body, [Result])
15 }
16
17 agent Researcher {
18     model: anthropic
19     system: "You are a thorough researcher. Always cite
20             sources."
21     tools: [search]
22     temperature: 0.2
23 }
24
25 fn main() {
26     answer, err = Researcher.ask("What is quantum computing?")
27     if err != nil {
28         io.println("Error: " + err.message)
29         return
30     }
31     io.println(answer)
32 }
```

1.3.5 Workflows with Triggers

Workflows are functions with triggers. They can be exposed as webhooks, scheduled with cron, or triggered by events:

```
1 @webhook("/summarize")
2 workflow Summarizer(url: string) -> Summary {
3     content = fetch_page(url)
4     summary = Researcher.ask("Summarize this: {content}")
5     return Summary{ text: summary, word_count: summary.len() }
6 }
7
8 fn main() {
9     server = haira.serve([Summarizer])
10    io.println("Running on :8080")
11    server.listen(8080)
12 }
```

1.3.6 Streaming and Conversations

Agents support streaming responses and session-based memory for chatbot use cases:

```

1  agent Assistant {
2      model: anthropic
3      system: "You are a helpful assistant."
4      memory: conversation(max_turns: 50)
5  }
6
7  @websocket("/chat")
8  workflow Chat(message: string, session_id: string) ->
9      stream<string> {
10     return Assistant.stream(message, session: session_id)
11 }

```

1.4 Type System

HAIRA uses structural typing with type inference for local variables:

```

1  // Type inferred as int
2  count = 42
3
4  // Explicit type annotation
5  name: string = "Haira"
6
7  // Struct definition
8  struct Point {
9      x: float
10     y: float
11 }
12
13 // Option type (nullable)
14 maybe_value: int? = nil
15
16 // Arrays and maps
17 numbers: []int = [1, 2, 3]
18 scores: [string:int] = ["alice": 100, "bob": 95]

```

1.5 Control Flow

```

1  import "io"
2
3  fn main() {
4      x = 10
5      if x > 5 {
6          io.println("Large")
7      } else {
8          io.println("Small")
9      }
10
11     for i in 0..5 {
12         io.println(i)
13     }
14 }

```

```

13     }
14
15     status = 200
16     match status {
17         200 => io.println("OK")
18         404 => io.println("Not Found")
19         _  => io.println("Unknown")
20     }
21 }

```

1.6 Functions

Functions are declared with **fn**:

```

1 fn add(a: int, b: int) -> int {
2     return a + b
3 }
4
5 fn divide(a: int, b: int) -> (int, Error?) {
6     if b == 0 {
7         return 0, Error{message: "division by zero"}
8     }
9     return a / b, nil
10 }

```

1.7 Concurrency

HAIRA provides Go-style concurrency with **spawn** and channels:

```

1 import "io"
2
3 fn main() {
4     ch = chan<int>(10)
5
6     spawn {
7         for i in 0..5 {
8             ch <- i
9         }
10        close(ch)
11    }
12
13    for value in ch {
14        io.println(value)
15    }
16 }

```

Inside workflows, **spawn** blocks run steps in parallel:

```

1 @manual
2 workflow ParallelReview(article: Article) -> [Review] {
3     reviews = spawn {
4         Reviewer.run(ReviewRequest{ article: article, focus:
5             "accuracy" })
6     }
7 }

```

```

5         Reviewer.run(ReviewRequest{ article: article, focus:
           "clarity" })
6         Reviewer.run(ReviewRequest{ article: article, focus:
           "style" })
7     }
8     return reviews
9 }

```

1.8 Pipe Operator

The pipe operator `|` enables functional composition:

```

1  import "io"
2  import "string"
3  import "array"
4
5  fn main() {
6      result = "  hello, world  "
7              | string.trim
8              | string.to_upper
9              | string.split(", ")
10             | array.first
11
12      io.println(result) // "HELLO"
13 }

```

1.9 A Complete Example

A multi-agent workflow that researches a topic, writes an article, and reviews it:

```

1  import "io"
2
3  provider anthropic {
4      api_key: env("ANTHROPIC_API_KEY")
5      model: "claude-sonnet-4-20250514"
6  }
7
8  struct Article { title: string, body: string, sources: [string]
9  }
10 struct Review { aspect: string, score: float, notes: string }
11
12 tool search(query: string) -> [string] {
13     """Search the web and return relevant URLs"""
14
15     resp, err = http.get("https://api.search.com?q={query}")
16     if err != nil { return [], err }
17     return json.decode(resp.body, [string])
18 }
19
20 agent Researcher {
21     model: anthropic
22     system: "You are a thorough researcher."
23     tools: [search]
24     temperature: 0.2
25 }

```

```

24 }
25
26 agent Writer {
27     model: anthropic
28     system: "You are a clear technical writer."
29     temperature: 0.7
30 }
31
32 agent Reviewer {
33     model: anthropic
34     system: "You are a critical but constructive editor."
35     temperature: 0.1
36 }
37
38 @manual
39 workflow WriteArticle(topic: string) -> Article {
40     research = Researcher.ask("Research this topic thoroughly:
41                               {topic}")
42
43     draft: Article, err = Writer.run(
44         { research: research, topic: topic }
45     ) -> Article
46     if err != nil { return Article{}, err }
47
48     reviews = spawn {
49         Reviewer.ask("Review for accuracy: {draft.body}")
50         Reviewer.ask("Review for clarity: {draft.body}")
51     }
52
53     io.println("Reviews complete: {reviews.len()}")
54     return draft
55 }
56
57 fn main() {
58     article, err = WriteArticle("The future of agentic
59                               programming")
60     if err != nil {
61         io.println("Failed: " + err.message)
62         return
63     }
64     io.println(article.title)
65     io.println(article.body)
66 }

```

1.10 Implementation Status

This specification documents the complete HAIRA language. Implementation proceeds incrementally:

Feature	Chapter	Target Release
Lexer & Parser	2–3	v0.1
Expressions & Statements	4–5	v0.2
Functions & Error Handling	6–7	v0.3
Concurrency	8	v0.4
Modules & Stdlib	9–10	v0.5
Providers & Tools	11–12	v0.6
Agents	13	v0.7
Workflows & Chatbot Patterns	14–15	v0.8
Full Grammar & Compiler	16–17	v1.0

Table 1.2: Implementation roadmap by chapter

Chapter 2

Lexical Structure

This chapter defines the lexical grammar of HAIRA: the rules for converting source text into tokens.

2.1 Source Encoding

HAIRA source files are encoded in UTF-8. The file extension is `.haira`.

```
hello.haira
utils/string_helpers.haira
```

2.2 Line Structure

HAIRA is a line-oriented language. Statements are terminated by newlines, not semicolons.

```
1 x = 1
2 y = 2
3 z = x + y
```

2.2.1 Line Continuation

Lines can be continued with a trailing backslash or when the line ends with an operator or open bracket:

```
1 // Explicit continuation
2 long_result = very_long_function_name(arg1, arg2) \
3     + another_function(arg3)
4
5 // Implicit continuation (ends with operator)
6 total = first_value +
7     second_value +
8     third_value
9
10 // Implicit continuation (open bracket)
11 users = [
12     "alice",
13     "bob",
14     "charlie"
15 ]
```

2.3 Comments

2.3.1 Single-Line Comments

```
1 // This is a single-line comment
2 x = 42 // Inline comment
```

2.3.2 Multi-Line Comments

```
1 /*
2     This is a multi-line comment.
3     It can span multiple lines.
4 */
```

Multi-line comments can be nested:

```
1 /*
2     Outer comment
3     /* Nested comment */
4     Still in outer comment
5 */
```

2.3.3 Documentation Comments

```
1 /// This is a documentation comment for the following item.
2 /// It can span multiple lines.
3 fn calculate_area(width: float, height: float) -> float {
4     return width * height
5 }
```

2.4 Tokens

The lexer produces the following token types:

1. Keywords
2. Identifiers
3. Literals (numbers, strings, booleans)
4. Operators
5. Delimiters

2.5 Keywords

The following identifiers are reserved as keywords:

<i>General-purpose keywords</i>				
and	as	break	catch	chan
continue	defer	else	enum	false
fn	for	from	if	import
in	match	nil	not	or
return	select	spawn	struct	true
try	type	while		
<i>Agentic keywords</i>				
agent	provider	tool	workflow	

Table 2.1: Reserved keywords

Note

The four agentic keywords (**agent**, **provider**, **tool**, **workflow**) are first-class language constructs for building AI-powered systems. See Chapters 11–14.

2.6 Identifiers

Identifiers name variables, functions, types, and modules.

```

1 identifier    = letter { letter | digit | "_" } ;
2 letter       = "a".."z" | "A".."Z" | "_" ;
3 digit        = "0".."9" ;

```

Valid identifiers:

```

x
name
userName
user_name
_private
Point3D
MAX_SIZE

```

Invalid identifiers:

```

3d_point    // Cannot start with digit
my-name     // Hyphens not allowed
class       // Reserved keyword (if added)

```

2.6.1 Naming Conventions

2.7 Literals

2.7.1 Integer Literals

```

1 integer_literal = decimal_literal
2                 | hex_literal
3                 | octal_literal

```

Entity	Convention
Variables	snake_case
Functions	snake_case
Types/Structs	PascalCase
Constants	SCREAMING_SNAKE_CASE
Modules	snake_case

Table 2.2: Naming conventions

```

4         | binary_literal ;
5
6 decimal_literal = digit { digit | "_" } ;
7 hex_literal    = "0" ("x"|"X") hex_digit { hex_digit | "_" } ;
8 octal_literal  = "0" ("o"|"O") octal_digit { octal_digit | "_"
9         } ;
10 binary_literal = "0" ("b"|"B") binary_digit { binary_digit |
11         "_" } ;
12 hex_digit      = digit | "a".."f" | "A".."F" ;
13 octal_digit    = "0".."7" ;
14 binary_digit   = "0" | "1" ;

```

Examples:

```

1 decimal = 42
2 with_underscores = 1_000_000
3 hex = 0xFF
4 octal = 0o755
5 binary = 0b1010_1100

```

Note

All integer literals are of type `int` (64-bit signed).

2.7.2 Floating-Point Literals

```

1 float_literal = decimal_literal "." decimal_literal [ exponent ]
2               | decimal_literal exponent ;
3
4 exponent = ("e"|"E") ["+"|"−"] decimal_literal ;

```

Examples:

```

1 pi = 3.14159
2 scientific = 6.022e23
3 negative_exp = 1.6e-19

```

2.7.3 Boolean Literals

```

1 is_valid = true

```

```
2 is_empty = false
```

2.7.4 String Literals

```
1 string_literal = ''' { string_char } ''' ;
2 string_char    = any_char - ('"' | '\'' | newline)
3                | escape_sequence ;
```

Escape sequences:

Sequence	Meaning
\n	Newline (LF)
\r	Carriage return (CR)
\t	Horizontal tab
\\	Backslash
\"	Double quote
\0	Null character
\xNN	Hex byte (00-FF)
\u{NNNN}	Unicode code point

Table 2.3: String escape sequences

Examples:

```
1 simple = "Hello, World!"
2 with_escapes = "Line 1\nLine 2"
3 unicode = "Hello \u{1F44B}" // Wave emoji
```

2.7.5 String Interpolation

Strings can contain interpolated expressions using `${...}`:

```
1 name = "Alice"
2 age = 30
3 greeting = "Hello, ${name}! You are ${age} years old."
```

Implementation Note

String interpolation is desugared to concatenation during parsing:

```
1 greeting = "Hello, " + name + "! You are " +
  to_string(age) + " years old."
```

2.7.6 Raw Strings

Raw strings preserve backslashes literally:

```
1 regex_pattern = r"^\d{3}-\d{4}$"
2 windows_path = r"C:\Users\Alice\Documents"
```

2.7.7 Triple-Quoted Strings

Triple-quoted strings use `"""..."""` for multi-line content:

```
1 query = """
2     SELECT *
3     FROM users
4     WHERE active = true
5 """
```

The leading whitespace common to all lines is stripped.

Triple-quoted strings are also used for:

- Tool descriptions (mandatory, see Chapter 12)
- Agent system prompts (optional, for long prompts, see Chapter 13)

```
1 tool search(query: string) -> [Result] {
2     """Search the web for information"""
3     // ...
4 }
5
6 agent Assistant {
7     model: anthropic
8     system: """
9         You are a helpful assistant.
10        Always be polite and concise.
11        """
12 }
```

2.7.8 Nil Literal

```
1 empty: User? = nil
```

2.8 Operators

2.8.1 Arithmetic Operators

Operator	Description
+	Addition
-	Subtraction (or unary negation)
*	Multiplication
/	Division
%	Modulo (remainder)

Operator	Description
==	Equal
!=	Not equal
<	Less than
>	Greater than
<=	Less than or equal
>=	Greater than or equal

Operator	Description
and, &&	Logical AND
or,	Logical OR
not, !	Logical NOT

2.8.2 Comparison Operators

2.8.3 Logical Operators

2.8.4 Assignment Operators

2.8.5 Bitwise Operators

2.8.6 Bitwise Assignment Operators

2.8.7 Other Operators

2.9 Delimiters

2.10 Whitespace

Whitespace (spaces, tabs) is used to separate tokens but is otherwise insignificant, except for indentation in multi-line strings.

```

1 x=1+2           // Valid
2 x = 1 + 2      // Also valid, preferred for readability

```

2.11 Lexical Grammar Summary

```

1 token = keyword
2       | identifier
3       | integer_literal
4       | float_literal
5       | string_literal
6       | operator
7       | delimiter ;
8
9 keyword = "agent" | "and" | "as" | "break" | "catch" | "chan"
10         | "continue" | "defer" | "else" | "enum" | "false"
11         | "fn" | "for" | "from" | "if" | "import" | "in"
12         | "match" | "nil" | "not" | "or" | "provider" | "return"
13         | "select" | "spawn" | "struct" | "tool" | "true" |
          "try"

```

Operator	Description
=	Assignment
+=	Add and assign
-=	Subtract and assign
*=	Multiply and assign
/=	Divide and assign

Operator	Description
&	Bitwise AND
	Bitwise OR
^	Bitwise XOR
~	Bitwise NOT (complement)
«	Left shift
»	Right shift (arithmetic for signed, logical for unsigned)
»»	Logical right shift (always fills with zeros)

```
14         | "type" | "while" | "workflow" ;
15
16 identifier = letter { letter | digit | "_" } ;
17
18 operator = "+" | "-" | "*" | "/" | "%"
19           | "==" | "!=" | "<" | ">" | "<=" | ">="
20           | "and" | "or" | "not" | "&&" | "||" | "!"
21           | "&" | "|" | "^" | "~" | "<<" | ">>" | ">>>"
22           | "=" | "+=" | "-=" | "*=" | "/="
23           | "&=" | "|=" | "^=" | "<<=" | ">>=" | ">>>="
24           | ">" | ".." | "..=" | "<-" | "->" | "=>"
25           | "@" ;
26
27 delimiter = "(" | ")" | "[" | "]" | "{" | "}"
28           | "," | ":" | "." | "?" ;
```


Operator	Description
<code>&=</code>	Bitwise AND and assign
<code> =</code>	Bitwise OR and assign
<code>^=</code>	Bitwise XOR and assign
<code>«=</code>	Left shift and assign
<code>»=</code>	Right shift and assign
<code>»>=</code>	Logical right shift and assign

Operator	Description
<code> ></code>	Pipe (function composition)
<code>..</code>	Range (exclusive end)
<code>..=</code>	Range (inclusive end)
<code><-</code>	Channel send
<code>-></code>	Function return type
<code>=></code>	Match arm
<code>@</code>	Decorator (workflow triggers)

Delimiter	Usage
<code>()</code>	Grouping, function calls, tuples
<code>[]</code>	Array literals, indexing
<code>{ }</code>	Blocks, struct literals, maps
<code>,</code>	Separator
<code>:</code>	Type annotations, map entries
<code>.</code>	Member access
<code>?</code>	Optional type marker

Chapter 3

Types

HAIRA is a statically-typed language with structural typing and type inference for local variables. All types are known at compile time.

3.1 Type System Overview

- **Static typing** – All types resolved at compile time
- **Structural typing** – Type compatibility based on structure, not name
- **Type inference** – Local variables infer types from initializers
- **Explicit annotations** – Required for function signatures
- **No null** – Option types ($T?$) replace null pointers

3.2 Primitive Types

3.2.1 Integer Types

Type	Size	Range
int	64 bits	-2^{63} to $2^{63} - 1$
i8	8 bits	-128 to 127
i16	16 bits	-32768 to 32767
i32	32 bits	-2^{31} to $2^{31} - 1$
i64	64 bits	-2^{63} to $2^{63} - 1$
u8	8 bits	0 to 255
u16	16 bits	0 to 65535
u32	32 bits	0 to $2^{32} - 1$
u64	64 bits	0 to $2^{64} - 1$

Table 3.1: Integer types

```
1 count: int = 42
2 byte: u8 = 255
3 offset: i32 = -100
```

Note

The default `int` is an alias for `i64`. Integer literals without explicit type default to `int`.

3.2.2 Floating-Point Types

Type	Size	Precision
<code>float</code>	64 bits	IEEE 754 double
<code>f32</code>	32 bits	IEEE 754 single
<code>f64</code>	64 bits	IEEE 754 double

Table 3.2: Floating-point types

```
1 pi: float = 3.14159
2 temperature: f32 = 98.6
```

3.2.3 Boolean Type

```
1 is_valid: bool = true
2 is_empty: bool = false
```

Boolean values are used in conditionals and logical expressions. Only `true` and `false` are valid boolean values—there is no implicit conversion from other types.

3.2.4 String Type

Strings are immutable sequences of UTF-8 encoded bytes:

```
1 name: string = "Haira"
2 greeting: string = "Hello, " + name
```

String operations:

```
1 import "string"
2
3 s = "Hello, World!"
4 length = string.len(s)           // 13
5 upper = string.to_upper(s)       // "HELLO, WORLD!"
6 parts = string.split(s, ", ")    // ["Hello", "World!"]
```

3.3 Composite Types**3.3.1 Arrays**

Arrays are ordered, homogeneous, dynamically-sized collections:

```
1 // Array literal
2 numbers: []int = [1, 2, 3, 4, 5]
```

```

3
4 // Empty array
5 empty: []string = []
6
7 // Array operations
8 import "array"
9
10 first = numbers[0]           // 1
11 length = array.len(numbers)  // 5
12 numbers = array.push(numbers, 6)

```

```

1 array_type = "[" "]" type ;

```

3.3.2 Maps

Maps are unordered key-value collections:

```

1 // Map literal
2 scores: [string:int] = [
3     "alice": 100,
4     "bob": 95,
5     "charlie": 87
6 ]
7
8 // Empty map
9 empty: [string:User] = [:]
10
11 // Map operations
12 alice_score = scores["alice"] // 100
13 scores["david"] = 92          // Insert

```

```

1 map_type = "[" type ":" type "]" ;

```

Note

Map keys must be comparable types: primitives, strings, or structs containing only comparable fields.

3.3.3 Tuples

Tuples are fixed-size, heterogeneous collections:

```

1 // Tuple type
2 point: (int, int) = (10, 20)
3
4 // Accessing elements
5 x = point.0 // 10
6 y = point.1 // 20
7
8 // Multiple return values use tuples
9 fn divide(a: int, b: int) -> (int, int) {
10     return a / b, a % b

```

```

11 }
12
13 quotient, remainder = divide(17, 5)

```

```

1 tuple_type = "(" type { "," type } ")" ;

```

3.4 Option Types

Option types represent values that may or may not exist, replacing null pointers:

```

1 // Optional value
2 maybe_name: string? = nil
3 maybe_age: int? = 25
4
5 // Checking for nil
6 if maybe_name != nil {
7     io.println(maybe_name)
8 }
9
10 // Default value with 'or'
11 name = maybe_name or "Unknown"

```

```

1 option_type = type "?" ;

```

3.4.1 Unwrapping Options

```

1 value: int? = get_value()
2
3 // Safe unwrap with default
4 result = value or 0
5
6 // Conditional unwrap
7 if value != nil {
8     // value is automatically unwrapped here
9     io.println(value)
10 }
11
12 // Match on option
13 match value {
14     nil => io.println("No value")
15     v => io.println("Got: " + to_string(v))
16 }

```

3.5 Struct Types

Structs are named product types with named fields:

```

1 struct User {
2     id: int

```

```
3     name: string
4     email: string
5     active: bool
6 }
7
8 // Creating a struct
9 user = User{
10     id: 1,
11     name: "Alice",
12     email: "alice@example.com",
13     active: true
14 }
15
16 // Accessing fields
17 io.println(user.name)
18
19 // Updating fields (structs are mutable by default)
20 user.active = false
```

3.5.1 Struct Methods

Methods can be defined on structs:

```
1 struct Rectangle {
2     width: float
3     height: float
4 }
5
6 fn (r: Rectangle) area() -> float {
7     return r.width * r.height
8 }
9
10 fn (r: Rectangle) perimeter() -> float {
11     return 2 * (r.width + r.height)
12 }
13
14 // Usage
15 rect = Rectangle{width: 10.0, height: 5.0}
16 io.println(rect.area())           // 50.0
17 io.println(rect.perimeter())      // 30.0
```

3.5.2 Nested Structs

```
1 struct Address {
2     street: string
3     city: string
4     country: string
5 }
6
7 struct Person {
8     name: string
9     address: Address
10 }
11
```

```
12 person = Person{
13     name: "Bob",
14     address: Address{
15         street: "123 Main St",
16         city: "New York",
17         country: "USA"
18     }
19 }
20
21 io.println(person.address.city) // "New York"
```

3.6 Enum Types

Enums define a type with a fixed set of named values:

```
1 enum Color {
2     Red,
3     Green,
4     Blue
5 }
6
7 enum Status {
8     Pending,
9     Active,
10    Completed,
11    Failed
12 }
13
14 // Usage
15 color: Color = Color.Red
16 status: Status = Status.Active
17
18 match color {
19     Color.Red => io.println("Stop")
20     Color.Green => io.println("Go")
21     Color.Blue => io.println("Info")
22 }
```

3.6.1 Enums with Associated Data

Enums can carry associated data (tagged unions):

```
1 enum Result<T> {
2     Ok(T),
3     Err(Error)
4 }
5
6 enum Message {
7     Text(string),
8     Number(int),
9     Pair(int, int)
10 }
11
12 // Usage
13 msg: Message = Message.Text("Hello")
```



```
14
15 match msg {
16     Message.Text(s) => io.println("Text: " + s)
17     Message.Number(n) => io.println("Number: " + to_string(n))
18     Message.Pair(a, b) => io.println("Pair: " + to_string(a) +
19         ", " + to_string(b))
20 }
```

3.7 Type Aliases

Type aliases create alternative names for existing types:

```
1 type UserId = int
2 type UserMap = [string:User]
3 type Handler = fn(Request) -> Response
4
5 // Usage
6 id: UserId = 42
7 users: UserMap = [:]
```

3.8 Function Types

Functions are first-class values with types:

```
1 // Function type syntax
2 type Comparator = fn(int, int) -> bool
3 type Transformer = fn(string) -> string
4 type Callback = fn()
5
6 // Higher-order functions
7 fn apply(f: fn(int) -> int, x: int) -> int {
8     return f(x)
9 }
10
11 fn double(n: int) -> int {
12     return n * 2
13 }
14
15 result = apply(double, 21) // 42
```

3.9 Generic Types

Generic types parameterize over other types:

```
1 // Generic struct
2 struct Pair<T, U> {
3     first: T
4     second: U
5 }
6
7 // Usage
8 pair: Pair<int, string> = Pair{first: 1, second: "one"}
```

```

9
10 // Generic function
11 fn swap<T, U>(p: Pair<T, U>) -> Pair<U, T> {
12     return Pair{first: p.second, second: p.first}
13 }
14
15 swapped = swap(pair) // Pair<string, int>

```

3.9.1 Type Constraints

Generics can be constrained to types implementing certain behaviors:

```

1 // Constraint definition
2 constraint Comparable {
3     fn compare(other: Self) -> int
4 }
5
6 // Constrained generic
7 fn max<T: Comparable>(a: T, b: T) -> T {
8     if a.compare(b) > 0 {
9         return a
10    }
11    return b
12 }
13
14 // Multiple constraints
15 fn process<T: Comparable + Printable>(value: T) {
16     io.println(value.to_string())
17 }

```

3.10 Stream Types

The `stream<T>` type represents a lazy, asynchronous sequence of values produced over time. Streams are the primary mechanism for handling token-by-token output from agents and for streaming responses over WebSocket connections.

```

1 // Stream of string tokens from an agent
2 for chunk in Assistant.stream("Tell me a story") {
3     io.print(chunk)
4 }

```

3.10.1 Stream Type Syntax

```

1 stream_type = "stream" "<" type ">" ;

```

A stream is parameterized by the element type it yields:

```

1 // Common stream types
2 stream<string> // Stream of text tokens (agent output)
3 stream<byte> // Stream of raw bytes
4 stream<Event> // Stream of typed events

```

3.10.2 Consuming Streams

Streams are consumed with **for** loops. Each iteration yields the next value as it becomes available:

```
1 // Iterate over a stream
2 tokens: stream<string> = Researcher.stream("Explain quantum
3   computing")
4 for token in tokens {
5     io.print(token) // Print each token as it arrives
6 }
```

3.10.3 Streams as Return Types

Functions and workflows can return streams, enabling streaming pipelines:

```
1 // Workflow returning a stream (e.g., for WebSocket)
2 @websocket("/chat")
3 workflow StreamingChat(message: string, session_id: string) ->
4   stream<string> {
5       return Assistant.stream(message, session: session_id)
6   }
```

When a workflow returns `stream<T>`, the runtime streams each element to the client as it is produced—no buffering.

3.10.4 Stream Properties

- **Lazy** – Values are produced on demand, not computed upfront
- **Single-use** – A stream can only be iterated once
- **Asynchronous** – The **for** loop suspends until the next value is available
- **Finite** – Streams eventually terminate (the loop exits when the source is exhausted)

Note

Streams are distinct from channels (`chan<T>`). A channel is a bidirectional communication primitive between concurrent tasks. A stream is a unidirectional, read-only sequence—typically produced by an agent or an external data source.

3.11 The Error Type

The built-in `Error` type represents errors:

```
1 struct Error {
2     message: string
3     code: int?
4     source: Error?
5 }
6
7 // Creating errors
8 err = Error{message: "file not found"}
```

```

9 err_with_code = Error{message: "permission denied", code: 403}
10
11 // Error chaining
12 wrapped = Error{
13     message: "failed to read config",
14     source: err
15 }

```

3.12 Type Inference

HAIRA infers types for local variables from their initializers:

```

1 // Types inferred
2 x = 42 // int
3 pi = 3.14 // float
4 name = "Haira" // string
5 active = true // bool
6 numbers = [1, 2, 3] // []int
7
8 // Explicit annotation when needed
9 count: i32 = 100 // Specific integer type

```

Warning

Type annotations are **required** for:

- Function parameters
- Function return types
- Struct fields
- Uninitialized variables

3.13 Structural Typing

Types are compatible based on structure, not name:

```

1 struct Point2D {
2     x: float
3     y: float
4 }
5
6 struct Vector2D {
7     x: float
8     y: float
9 }
10
11 fn magnitude(p: {x: float, y: float}) -> float {
12     return math.sqrt(p.x * p.x + p.y * p.y)
13 }
14
15 // Both work because they have the same structure
16 point = Point2D{x: 3.0, y: 4.0}

```

```
17 vector = Vector2D{x: 3.0, y: 4.0}
18
19 io.println(magnitude(point))    // 5.0
20 io.println(magnitude(vector))  // 5.0
```

3.14 Type Grammar

```
1  type = primitive_type
2      | array_type
3      | map_type
4      | tuple_type
5      | option_type
6      | function_type
7      | generic_type
8      | stream_type
9      | struct_type
10     | identifier ;
11
12 primitive_type = "int" | "i8" | "i16" | "i32" | "i64"
13                | "u8" | "u16" | "u32" | "u64"
14                | "float" | "f32" | "f64"
15                | "bool" | "string" ;
16
17 array_type = "[" " " type ;
18
19 map_type = "[" type ":" type "]" ;
20
21 tuple_type = "(" type { "," type } ")" ;
22
23 option_type = type "?" ;
24
25 function_type = "fn" "(" [ type_list ] ")" [ "->" type ] ;
26
27 generic_type = identifier "<" type_list ">" ;
28
29 stream_type = "stream" "<" type ">" ;
30
31 type_list = type { "," type } ;
```


Chapter 4

Expressions

Expressions produce values. Every expression in HAIRA has a type determined at compile time.

4.1 Primary Expressions

4.1.1 Literals

```
1 42           // int literal
2 3.14         // float literal
3 "hello"      // string literal
4 true         // bool literal
5 nil          // nil literal
```

4.1.2 Identifiers

```
1 x            // Variable reference
2 user.name    // Field access
3 array[0]     // Index access
```

4.1.3 Parenthesized Expressions

```
1 (1 + 2) * 3   // Grouping for precedence
```

4.2 Arithmetic Expressions

```
1 a + b        // Addition
2 a - b        // Subtraction
3 a * b        // Multiplication
4 a / b        // Division
5 a % b        // Modulo (remainder)
6 -a          // Unary negation
```

4.2.1 Integer Division

Integer division truncates toward zero:

```
1 7 / 3          // 2
2 -7 / 3         // -2
3 7 / -3         // -2
```

4.2.2 Modulo

The modulo operator returns the remainder:

```
1 7 % 3          // 1
2 -7 % 3         // -1
3 7 % -3         // 1
```

4.3 Comparison Expressions

```
1 a == b          // Equal
2 a != b          // Not equal
3 a < b           // Less than
4 a > b           // Greater than
5 a <= b          // Less or equal
6 a >= b          // Greater or equal
```

All comparison expressions produce `bool` values.

4.3.1 Equality

Equality (`==`) compares by value for primitives and by structural equality for composite types:

```
1 1 == 1          // true
2 "hello" == "hello" // true
3 [1, 2] == [1, 2] // true
4 User{id: 1} == User{id: 1} // true (structural)
```

4.4 Logical Expressions

```
1 a and b          // Logical AND (also &&)
2 a or b           // Logical OR (also ||)
3 not a            // Logical NOT (also !)
```

4.4.1 Short-Circuit Evaluation

Logical operators use short-circuit evaluation:

```
1 // 'b' is not evaluated if 'a' is false
2 a and b
```



```

3
4 // 'b' is not evaluated if 'a' is true
5 a or b

```

4.5 Bitwise Expressions

Bitwise operators perform bit-level operations on integer types. All bitwise operators work on any integer type (int, i8–i64, u8–u64).

```

1 a & b           // Bitwise AND
2 a | b           // Bitwise OR
3 a ^ b           // Bitwise XOR
4 ~a              // Bitwise NOT (one's complement)
5 a << n          // Left shift by n bits
6 a >> n          // Right shift by n bits
7 a >>> n         // Logical right shift by n bits

```

4.5.1 Bitwise AND

Sets each bit to 1 only if both corresponding bits are 1:

```

1 0b1100 & 0b1010 // 0b1000 (8)
2 flags & MASK     // Extract bits using mask
3 value & 0xFF      // Keep only lowest 8 bits

```

4.5.2 Bitwise OR

Sets each bit to 1 if at least one corresponding bit is 1:

```

1 0b1100 | 0b1010 // 0b1110 (14)
2 flags | FLAG_ENABLED // Set a flag bit

```

4.5.3 Bitwise XOR

Sets each bit to 1 if exactly one corresponding bit is 1:

```

1 0b1100 ^ 0b1010 // 0b0110 (6)
2 value ^ mask     // Toggle bits
3 a ^ b ^ b        // Returns a (XOR is self-inverse)

```

4.5.4 Bitwise NOT

Inverts all bits (one's complement):

```

1 ~0b00001111 // 0b11110000 (for 8-bit)
2 ~0u8         // 255 (all bits set)
3 ~0           // -1 (for signed int)

```

Warning

The result of `~` depends on the bit width of the operand type. For `u8`, `~0` is 255; for `int`, `~0` is `-1`.

4.5.5 Left Shift

Shifts bits to the left, filling vacated positions with zeros:

```
1 1 << 4           // 16 (0b10000)
2 value << n       // Equivalent to value * 2^n
3 0b0001 << 3     // 0b1000 (8)
```

Warning

Shifting by a negative amount or by more than the bit width of the type is undefined behavior.

4.5.6 Right Shift

The behavior of `>>` depends on the signedness of the operand:

- **Unsigned types:** Logical shift (fills with zeros)
- **Signed types:** Arithmetic shift (fills with sign bit)

```
1 // Unsigned: logical shift (fills with 0)
2 x: u8 = 0b11110000
3 x >> 2           // 0b00111100 (60)
4
5 // Signed: arithmetic shift (fills with sign bit)
6 y: i8 = -16      // 0b11110000 in two's complement
7 y >> 2           // -4 (0b11111100, sign preserved)
```

4.5.7 Logical Right Shift

The `>>>` operator always performs a logical right shift, filling with zeros regardless of signedness:

```
1 x: i8 = -16      // 0b11110000
2 x >>> 2          // 60 (0b00111100, zeros filled)
3
4 // Useful for treating signed integers as bit patterns
5 hash: int = -1
6 hash >>> 1       // Large positive number
```

4.5.8 Common Bit Manipulation Patterns

```
1 // Check if bit n is set
2 is_set = (value >> n) & 1 == 1
3
```

```

4 // Set bit n
5 value = value | (1 << n)
6
7 // Clear bit n
8 value = value & ~(1 << n)
9
10 // Toggle bit n
11 value = value ^ (1 << n)
12
13 // Check if power of two (positive integers)
14 is_power_of_two = value > 0 and (value & (value - 1)) == 0
15
16 // Extract n bits starting at position p
17 bits = (value >> p) & ((1 << n) - 1)
18
19 // Count trailing zeros (manual approach)
20 fn count_trailing_zeros(x: u32) -> int {
21     if x == 0 { return 32 }
22     count = 0
23     while (x & 1) == 0 {
24         count += 1
25         x = x >> 1
26     }
27     return count
28 }

```

4.5.9 Compound Bitwise Assignment

```

1 flags &= MASK           // flags = flags & MASK
2 flags |= FLAG           // flags = flags | FLAG
3 flags ^= TOGGLE         // flags = flags ^ TOGGLE
4 value <<= 2              // value = value << 2
5 value >>= 1              // value = value >> 1
6 value >>>= 1             // value = value >>> 1

```

4.6 String Expressions

4.6.1 Concatenation

```

1 "Hello, " + "World!"    // "Hello, World!"

```

4.6.2 String Interpolation

```

1 name = "Alice"
2 age = 30
3 "Hello, ${name}! You are ${age} years old."

```

Expressions inside `${...}` are evaluated and converted to strings.

4.7 Array and Map Expressions

4.7.1 Array Literals

```
1 [] // Empty array (type from context)
2 [1, 2, 3] // []int
3 ["a", "b", "c"] // []string
```

4.7.2 Map Literals

```
1 [:] // Empty map (type from context)
2 ["a": 1, "b": 2] // [string:int]
3 [1: "one", 2: "two"] // [int:string]
```

4.7.3 Index Access

```
1 array[0] // First element
2 array[array.len - 1] // Last element
3 map["key"] // Map lookup (returns T?)
```

Note

Array indexing with an out-of-bounds index causes a runtime panic. Map indexing returns an option type.

4.8 Field Access

```
1 user.name // Struct field
2 point.x // Struct field
3 tuple.0 // Tuple element
```

4.9 Function Calls

```
1 func() // No arguments
2 func(a, b, c) // With arguments
3 obj.method() // Method call
4 obj.method(a, b) // Method with arguments
```

4.9.1 Chained Calls

```
1 text.trim().to_upper().split(" ")
```

4.10 Pipe Expressions

The pipe operator `|>` passes the left operand as the first argument to the right function:

```
1 // These are equivalent:
2 result = f(g(h(x)))
3 result = x |> h |> g |> f
```

4.10.1 Pipe with Arguments

When the right side has arguments, the left value becomes the first argument:

```
1 // These are equivalent:
2 result = string.split(text, ",")
3 result = text |> string.split(",")
```

4.10.2 Complex Pipelines

```
1 import "io"
2 import "string"
3 import "array"
4
5 fn main() {
6     data = "  apple, banana, cherry  "
7
8     result = data
9         |> string.trim
10        |> string.split(", ")
11        |> array.map(string.to_upper)
12        |> array.filter(fn(s) { string.len(s) > 5 })
13        |> string.join(" - ")
14
15     io.println(result) // "BANANA - CHERRY"
16 }
```

4.11 Range Expressions

```
1 0..10 // 0 to 9 (exclusive end)
2 0..=10 // 0 to 10 (inclusive end)
```

Ranges are used primarily in **for** loops:

```
1 for i in 0..5 {
2     io.println(i) // 0, 1, 2, 3, 4
3 }
4
5 for i in 0..=5 {
6     io.println(i) // 0, 1, 2, 3, 4, 5
7 }
```

4.12 Conditional Expressions

4.12.1 If Expression

`if` can be used as an expression:

```
1 max = if a > b { a } else { b }
2
3 status = if score >= 90 {
4     "A"
5 } else if score >= 80 {
6     "B"
7 } else {
8     "C"
9 }
```

Warning

When used as an expression, all branches must have the same type and `else` is required.

4.12.2 Match Expression

`match` expressions provide pattern matching:

```
1 result = match value {
2     0 => "zero"
3     1 => "one"
4     n if n < 0 => "negative"
5     _ => "other"
6 }
```

See Chapter 5 for complete pattern matching semantics.

4.13 Block Expressions

Blocks are expressions that evaluate to their last expression:

```
1 result = {
2     x = compute_x()
3     y = compute_y()
4     x + y // This is the value of the block
5 }
```

4.14 Struct Instantiation

```
1 user = User{
2     id: 1,
3     name: "Alice",
4     email: "alice@example.com"
5 }
6
```

```
7 // With shorthand (field name matches variable)
8 name = "Bob"
9 email = "bob@example.com"
10 user = User{id: 2, name, email}
```

4.15 Closure Expressions

Anonymous functions (closures):

```
1 // Full syntax
2 add = fn(a: int, b: int) -> int {
3     return a + b
4 }
5
6 // Short syntax for single expression
7 double = fn(x: int) -> int { x * 2 }
8
9 // Type inference for closures
10 numbers = [1, 2, 3]
11 doubled = array.map(numbers, fn(n) { n * 2 })
```

4.16 Option Expressions

4.16.1 Nil Coalescing

The `or` operator provides a default for optional values:

```
1 name = maybe_name or "Unknown"
2 value = get_value() or default_value
```

4.16.2 Optional Chaining

```
1 // Returns nil if any part is nil
2 street = user?.address?.street
```

4.17 Type Assertions

```
1 // Assert that value is a specific type
2 x = value as int
3
4 // Safe assertion (returns option)
5 x = value as? int
```

4.18 Operator Precedence

From highest to lowest:

1. Postfix: () [] . ?.
2. Unary: ! - ~ not
3. Multiplicative: * / %
4. Additive: + -
5. Shift: < > >>
6. Bitwise AND: &
7. Bitwise XOR: ^
8. Bitwise OR: |
9. Range:=
10. Pipe: |>
11. Comparison: < > <= >=
12. Equality: == !=
13. Logical AND: and &&
14. Logical OR: or ||
15. Assignment: = += -= *= /= &= |= ^= <= >= >>=

4.19 Expression Grammar

```

1 expression = assignment ;
2
3 assignment = or_expr [ ("=" | "+=" | "-=" | "*=" | "/"=
4               | "&=" | "|=" | "^=" | "<=" | ">=" | ">>=")
               assignment ] ;
5
6 or_expr = and_expr { ("or" | "||") and_expr } ;
7
8 and_expr = equality { ("and" | "&&") equality } ;
9
10 equality = comparison { ("==" | "!=") comparison } ;
11
12 comparison = pipe { ("<" | ">" | "<=" | ">=") pipe } ;
13
14 pipe = range { "|>" range } ;
15
16 range = bitwise_or [ (".." | "..=") bitwise_or ] ;
17
18 bitwise_or = bitwise_xor { "|" bitwise_xor } ;
19
20 bitwise_xor = bitwise_and { "^" bitwise_and } ;
21
22 bitwise_and = shift { "&" shift } ;
23
24 shift = additive { ("<<" | ">>" | ">>>") additive } ;
25

```



```
26 additive = multiplicative { ("+" | "-") multiplicative } ;
27
28 multiplicative = unary { ("*" | "/" | "%") unary } ;
29
30 unary = ("!" | "-" | "~" | "not") unary | postfix ;
31
32 postfix = primary { call | index | field } ;
33
34 call = "(" [ arguments ] ")" ;
35
36 index = "[" expression "]" ;
37
38 field = "." identifier ;
39
40 primary = identifier
41         | literal
42         | "(" expression ")"
43         | array_literal
44         | map_literal
45         | struct_literal
46         | closure
47         | if_expr
48         | match_expr
49         | block ;
50
51 arguments = expression { "," expression } ;
```


Chapter 5

Statements

Statements perform actions. Unlike expressions, statements do not produce values.

5.1 Variable Declarations

5.1.1 With Inference

```
1 x = 42                // Type inferred as int
2 name = "Haira"        // Type inferred as string
3 items = [1, 2, 3]      // Type inferred as []int
```

5.1.2 With Explicit Type

```
1 x: int = 42
2 name: string = "Haira"
3 count: int = 100
```

5.1.3 Multiple Declarations

```
1 a, b = 1, 2
2 x, y, z = compute()    // Destructuring tuple
```

5.1.4 Uninitialized Variables

```
1 x: int                // Must be assigned before use
```

Warning

Using an uninitialized variable is a compile-time error.

5.2 Assignment Statements

```
1 x = 10           // Simple assignment
2 x += 5           // Add and assign (x = x + 5)
3 x -= 3           // Subtract and assign
4 x *= 2           // Multiply and assign
5 x /= 4           // Divide and assign
```

5.2.1 Multiple Assignment

```
1 a, b = b, a      // Swap values
2 x, y = get_coordinates()
3 quotient, remainder = divide(17, 5)
```

5.2.2 Field Assignment

```
1 user.name = "Bob"
2 point.x = 10.0
```

5.2.3 Index Assignment

```
1 array[0] = 100
2 map["key"] = "value"
```

5.3 Expression Statements

Any expression can be used as a statement:

```
1 io.println("Hello") // Function call
2 process(data)        // Function call, ignoring return
3 x + y                // Valid but pointless
```

Note

The compiler may warn about expression statements that have no side effects.

5.4 If Statements

```
1 if condition {
2     // executed if condition is true
3 }
4
5 if condition {
6     // true branch
7 } else {
8     // false branch
9 }
```

```
10
11 if condition1 {
12     // first
13 } else if condition2 {
14     // second
15 } else {
16     // default
17 }
```

5.4.1 Condition Binding

```
1 // Bind and check in one statement
2 if value = get_optional_value(); value != nil {
3     io.println(value)
4 }
```

5.5 For Statements

5.5.1 Range-Based For

```
1 // Exclusive range
2 for i in 0..10 {
3     io.println(i) // 0 through 9
4 }
5
6 // Inclusive range
7 for i in 0..=10 {
8     io.println(i) // 0 through 10
9 }
```

5.5.2 Collection Iteration

```
1 // Array iteration
2 for item in items {
3     io.println(item)
4 }
5
6 // With index
7 for i, item in items {
8     io.println("${i}: ${item}")
9 }
10
11 // Map iteration
12 for key, value in scores {
13     io.println("${key}: ${value}")
14 }
```

5.5.3 String Iteration

```
1 for char in "hello" {
2     io.println(char) // Each UTF-8 character
3 }
4
5 for i, char in "hello" {
6     io.println("${i}: ${char}")
7 }
```

5.6 While Statements

```
1 while condition {
2     // body
3 }
4
5 // Infinite loop
6 while true {
7     if should_stop() {
8         break
9     }
10 }
```

5.7 Match Statements

5.7.1 Basic Matching

```
1 match value {
2     1 => io.println("one")
3     2 => io.println("two")
4     3 => io.println("three")
5     _ => io.println("other")
6 }
```

5.7.2 Multiple Patterns

```
1 match value {
2     1 | 2 | 3 => io.println("small")
3     4 | 5 | 6 => io.println("medium")
4     _ => io.println("large")
5 }
```

5.7.3 Range Patterns

```
1 match score {
2     90..100 => io.println("A")
3     80..90  => io.println("B")
4     70..80  => io.println("C")
5     60..70  => io.println("D")
6 }
```

```
6   _ => io.println("F")
7 }
```

5.7.4 Guard Clauses

```
1 match value {
2   n if n < 0 => io.println("negative")
3   n if n == 0 => io.println("zero")
4   n if n > 0 => io.println("positive")
5 }
```

5.7.5 Destructuring Patterns

```
1 // Struct destructuring
2 match user {
3   User{name: "admin", active: true} => io.println("Active
4     admin")
5   User{name: n, active: false} => io.println("Inactive: " + n)
6   _ => io.println("Regular user")
7 }
8 // Tuple destructuring
9 match point {
10  (0, 0) => io.println("origin")
11  (x, 0) => io.println("on x-axis at " + to_string(x))
12  (0, y) => io.println("on y-axis at " + to_string(y))
13  (x, y) => io.println("at (" + to_string(x) + ", " +
14    to_string(y) + ")")
15 }
16 // Enum destructuring
17 match result {
18   Result.Ok(value) => io.println("Got: " + to_string(value))
19   Result.Err(err) => io.println("Error: " + err.message)
20 }
```

5.7.6 Match Exhaustiveness

```
1 enum Direction { North, South, East, West }
2
3 match direction {
4   Direction.North => go_north()
5   Direction.South => go_south()
6   Direction.East => go_east()
7   Direction.West => go_west()
8   // No _ needed: all cases covered
9 }
```

Note

The compiler verifies that match expressions are exhaustive. If not all cases are covered and there is no wildcard (`_`), the compiler produces an error.

5.8 Block Statements

```

1 {
2     x = 10
3     y = 20
4     io.println(x + y)
5 }
```

Blocks create a new scope. Variables declared inside are not visible outside.

5.9 Return Statements

```

1 fn add(a: int, b: int) -> int {
2     return a + b
3 }
4
5 fn early_return(x: int) -> int {
6     if x < 0 {
7         return 0
8     }
9     return x * 2
10 }
11
12 // Multiple returns
13 fn divide(a: int, b: int) -> (int, Error?) {
14     if b == 0 {
15         return 0, Error{message: "division by zero"}
16     }
17     return a / b, nil
18 }
```

5.9.1 Implicit Return

The last expression in a function is implicitly returned:

```

1 fn add(a: int, b: int) -> int {
2     a + b // No return keyword needed
3 }
```

5.10 Break Statements

```

1 for i in 0..100 {
2     if i > 10 {
3         break
4     }
5 }
```



```
4     }
5     io.println(i)
6 }
7
8 while true {
9     if done() {
10         break
11     }
12 }
```

5.10.1 Labeled Break

```
1 outer: for i in 0..10 {
2     for j in 0..10 {
3         if i * j > 25 {
4             break outer
5         }
6     }
7 }
```

5.11 Continue Statements

```
1 for i in 0..10 {
2     if i % 2 == 0 {
3         continue // Skip even numbers
4     }
5     io.println(i)
6 }
```

5.11.1 Labeled Continue

```
1 outer: for i in 0..5 {
2     for j in 0..5 {
3         if j == 2 {
4             continue outer
5         }
6         io.println("${i}, ${j}")
7     }
8 }
```

5.12 Import Statements

Import statements must appear at the top of the file:

```
1 import "io"
2 import "json"
3 import db from "haira/postgres"
4 import { User, Post } from "models"
```

See Chapter 9 for complete import semantics.

5.13 Defer Statements

defer schedules a statement to run when the current scope exits:

```

1 fn read_file(path: string) -> (string, Error?) {
2     file, err = fs.open(path)
3     if err != nil {
4         return "", err
5     }
6     defer fs.close(file)
7
8     content, err = fs.read_all(file)
9     if err != nil {
10        return "", err // file is closed here
11    }
12
13    return content, nil // file is closed here too
14 }
```

5.13.1 Multiple Defers

Multiple **defer** statements execute in reverse order (LIFO):

```

1 fn example() {
2     defer io.println("first") // Runs third
3     defer io.println("second") // Runs second
4     defer io.println("third") // Runs first
5 }
6 // Output: third, second, first
```

5.14 Statement Grammar

```

1 statement = variable_decl
2             | assignment
3             | expression_stmt
4             | if_stmt
5             | for_stmt
6             | while_stmt
7             | match_stmt
8             | return_stmt
9             | break_stmt
10            | continue_stmt
11            | defer_stmt
12            | block ;
13
14 variable_decl = identifier [ ":" type ] "=" expression
15                | identifier_list "=" expression_list ;
16
17 assignment = lvalue assign_op expression ;
18
19 assign_op = "=" | "+=" | "-=" | "*=" | "/=" ;
```

```
20
21 lvalue = identifier | field_access | index_access ;
22
23 expression_stmt = expression ;
24
25 if_stmt = "if" [ simple_stmt ";" ] expression block
26           [ "else" ( if_stmt | block ) ] ;
27
28 for_stmt = "for" [ identifier "," ] identifier "in" expression
29           block ;
30
31 while_stmt = "while" expression block ;
32
33 match_stmt = "match" expression "{ " { match_arm } " } " ;
34
35 match_arm = pattern [ "if" expression ] "=>" ( expression |
36           block ) ;
37
38 return_stmt = "return" [ expression_list ] ;
39
40 break_stmt = "break" [ identifier ] ;
41
42 continue_stmt = "continue" [ identifier ] ;
43
44 defer_stmt = "defer" statement ;
45
46 block = "{ " { statement } " } " ;
```


Chapter 6

Functions

Functions are the primary unit of code organization in HAIRA. They are first-class values that can be passed as arguments, returned from other functions, and stored in variables.

6.1 Function Declarations

```
1 fn function_name(param1: Type1, param2: Type2) -> ReturnType {  
2     // body  
3 }
```

6.1.1 Basic Examples

```
1 fn greet() {  
2     io.println("Hello!")  
3 }  
4  
5 fn add(a: int, b: int) -> int {  
6     return a + b  
7 }  
8  
9 fn is_positive(n: int) -> bool {  
10    return n > 0  
11 }
```

6.1.2 No Return Type

Functions without a return type implicitly return ():

```
1 fn log_message(msg: string) {  
2     io.println("[LOG] " + msg)  
3 }
```

6.2 Parameters

6.2.1 Required Parameters

All parameters must have explicit type annotations:

```
1 fn calculate(x: int, y: int, z: float) -> float {
2     return (x + y) as float * z
3 }
```

6.2.2 Default Parameters

Parameters can have default values:

```
1 fn greet(name: string, greeting: string = "Hello") {
2     io.println(greeting + ", " + name + "!")
3 }
4
5 greet("Alice")           // "Hello, Alice!"
6 greet("Bob", "Hi")       // "Hi, Bob!"
```

Note

Parameters with defaults must come after parameters without defaults.

6.2.3 Named Arguments

Functions can be called with named arguments:

```
1 fn create_user(name: string, email: string, age: int = 0) ->
  User {
2     return User{name: name, email: email, age: age}
3 }
4
5 // Positional
6 create_user("Alice", "alice@example.com", 30)
7
8 // Named
9 create_user(name: "Bob", email: "bob@example.com")
10
11 // Mixed (positional first, then named)
12 create_user("Charlie", email: "charlie@example.com", age: 25)
```

6.2.4 Variadic Parameters

Functions can accept variable numbers of arguments:

```
1 fn sum(numbers: ...int) -> int {
2     total = 0
3     for n in numbers {
4         total += n
5     }
6     return total
7 }
```

```
7 }  
8  
9 sum(1, 2, 3)           // 6  
10 sum(1, 2, 3, 4, 5)    // 15
```

6.3 Return Values

6.3.1 Single Return

```
1 fn double(n: int) -> int {  
2     return n * 2  
3 }
```

6.3.2 Multiple Returns

Functions can return multiple values using tuples:

```
1 fn divide(a: int, b: int) -> (int, int) {  
2     return a / b, a % b  
3 }  
4  
5 quotient, remainder = divide(17, 5) // 3, 2
```

6.3.3 Implicit Return

The last expression in a function body is implicitly returned:

```
1 fn add(a: int, b: int) -> int {  
2     a + b // Implicit return  
3 }  
4  
5 fn max(a: int, b: int) -> int {  
6     if a > b { a } else { b } // Implicit return  
7 }
```

6.3.4 Early Return

Use **return** for early exit:

```
1 fn factorial(n: int) -> int {  
2     if n <= 1 {  
3         return 1  
4     }  
5     return n * factorial(n - 1)  
6 }
```

6.4 Function Types

Functions have types that can be used in type annotations:

```

1 // Function type: fn(int, int) -> int
2 type BinaryOp = fn(int, int) -> int
3
4 fn apply(op: BinaryOp, a: int, b: int) -> int {
5     return op(a, b)
6 }
7
8 fn add(a: int, b: int) -> int { a + b }
9 fn mul(a: int, b: int) -> int { a * b }
10
11 apply(add, 2, 3) // 5
12 apply(mul, 2, 3) // 6

```

6.5 Closures

Closures are anonymous functions that can capture values from their environment:

```

1 fn make_counter() -> fn() -> int {
2     count = 0
3     return fn() -> int {
4         count += 1
5         return count
6     }
7 }
8
9 counter = make_counter()
10 counter() // 1
11 counter() // 2
12 counter() // 3

```

6.5.1 Closure Syntax

```

1 // Full syntax
2 fn(a: int, b: int) -> int {
3     return a + b
4 }
5
6 // Short syntax (single expression)
7 fn(a: int, b: int) -> int { a + b }
8
9 // With type inference (in context)
10 numbers = [1, 2, 3]
11 doubled = array.map(numbers, fn(n) { n * 2 })

```

6.5.2 Capture Semantics

Closures capture variables by reference:

```

1 fn example() {
2     values = []
3 }

```



```

4     for i in 0..3 {
5         // Captures 'i' by reference
6         values = array.push(values, fn() { i })
7     }
8
9     // All closures see final value of i
10    values[0]() // 3
11    values[1]() // 3
12    values[2]() // 3
13 }

```

To capture by value, use explicit binding:

```

1 fn example() {
2     values = []
3
4     for i in 0..3 {
5         captured = i // Capture current value
6         values = array.push(values, fn() { captured })
7     }
8
9     values[0]() // 0
10    values[1]() // 1
11    values[2]() // 2
12 }

```

6.6 Methods

Methods are functions associated with a type:

```

1 struct Rectangle {
2     width: float
3     height: float
4 }
5
6 fn (r: Rectangle) area() -> float {
7     return r.width * r.height
8 }
9
10 fn (r: Rectangle) perimeter() -> float {
11     return 2.0 * (r.width + r.height)
12 }
13
14 fn (r: Rectangle) scale(factor: float) -> Rectangle {
15     return Rectangle{
16         width: r.width * factor,
17         height: r.height * factor
18     }
19 }
20
21 // Usage
22 rect = Rectangle{width: 10.0, height: 5.0}
23 rect.area() // 50.0
24 rect.perimeter() // 30.0
25 rect.scale(2.0) // Rectangle{width: 20.0, height: 10.0}

```

6.6.1 Method Receiver

The receiver can be by value or by reference:

```

1 // By value (receiver is copied)
2 fn (p: Point) distance_to(other: Point) -> float {
3     dx = p.x - other.x
4     dy = p.y - other.y
5     return math.sqrt(dx*dx + dy*dy)
6 }
7
8 // By mutable reference (can modify receiver)
9 fn (p: *Point) translate(dx: float, dy: float) {
10     p.x += dx
11     p.y += dy
12 }

```

6.7 Generic Functions

Functions can be parameterized over types:

```

1 fn identity<T>(value: T) -> T {
2     return value
3 }
4
5 fn swap<T, U>(pair: (T, U)) -> (U, T) {
6     return (pair.1, pair.0)
7 }
8
9 fn first<T>(items: []T) -> T? {
10     if array.len(items) == 0 {
11         return nil
12     }
13     return items[0]
14 }

```

6.7.1 Type Constraints

Generic types can be constrained:

```

1 constraint Comparable {
2     fn compare(other: Self) -> int
3 }
4
5 fn max<T: Comparable>(a: T, b: T) -> T {
6     if a.compare(b) > 0 {
7         return a
8     }
9     return b
10 }
11
12 fn sort<T: Comparable>(items: []T) -> []T {
13     // Sorting implementation
14 }

```

6.8 Higher-Order Functions

Functions that take or return other functions:

```
1 // Function as parameter
2 fn apply_twice(f: fn(int) -> int, x: int) -> int {
3     return f(f(x))
4 }
5
6 fn double(n: int) -> int { n * 2 }
7
8 apply_twice(double, 5) // 20
9
10 // Function as return value
11 fn make_multiplier(factor: int) -> fn(int) -> int {
12     return fn(n: int) -> int {
13         return n * factor
14     }
15 }
16
17 triple = make_multiplier(3)
18 triple(7) // 21
```

6.9 Recursion

Functions can call themselves:

```
1 fn factorial(n: int) -> int {
2     if n <= 1 {
3         return 1
4     }
5     return n * factorial(n - 1)
6 }
7
8 fn fibonacci(n: int) -> int {
9     match n {
10         0 => 0
11         1 => 1
12         _ => fibonacci(n - 1) + fibonacci(n - 2)
13     }
14 }
```

6.9.1 Tail Call Optimization

HAIRA optimizes tail-recursive functions:

```
1 fn factorial_tail(n: int, acc: int = 1) -> int {
2     if n <= 1 {
3         return acc
4     }
5     return factorial_tail(n - 1, n * acc) // Tail call
6 }
```

6.10 Function Overloading

HAIRA does not support function overloading. Each function name must be unique within its scope. Use different names or generic functions instead:

```

1 // Instead of overloading, use descriptive names
2 fn add_ints(a: int, b: int) -> int { a + b }
3 fn add_floats(a: float, b: float) -> float { a + b }
4
5 // Or use generics
6 fn add<T: Addable>(a: T, b: T) -> T { a + b }

```

6.11 The main Function

Every executable HAIRA program must have a `main` function:

```

1 fn main() {
2     io.println("Hello, World!")
3 }
4
5 // With exit code
6 fn main() -> int {
7     io.println("Hello, World!")
8     return 0
9 }

```

6.12 Function Grammar

```

1 function_decl = "fn" [ receiver ] identifier [ type_params ]
2               "(" [ params ] ")" [ "->" type ] block ;
3
4 receiver = "(" identifier ":" [ "*" ] type ")" ;
5
6 type_params = "<" type_param { "," type_param } ">" ;
7
8 type_param = identifier [ ":" constraint_list ] ;
9
10 constraint_list = identifier { "+" identifier } ;
11
12 params = param { "," param } ;
13
14 param = identifier ":" [ "..." ] type [ "=" expression ] ;
15
16 closure = "fn" "(" [ closure_params ] ")" [ "->" type ]
17          ( block | expression ) ;
18
19 closure_params = closure_param { "," closure_param } ;
20
21 closure_param = identifier [ ":" type ] ;

```

Chapter 7

Error Handling

HAIRA uses explicit error handling inspired by Go. Errors are values, not exceptions. Functions that can fail return both a result and an optional error.

7.1 Philosophy

- **Errors are values** – Not special control flow
- **Explicit handling** – Callers must handle or propagate errors
- **No hidden control flow** – No exceptions or stack unwinding
- **Composable** – Errors can be wrapped and chained

7.2 The Error Type

The built-in Error type represents errors:

```
1 struct Error {
2     message: string
3     code: int?
4     source: Error?
5 }
```

7.2.1 Creating Errors

```
1 // Simple error
2 err = Error{message: "something went wrong"}
3
4 // With error code
5 err = Error{message: "permission denied", code: 403}
6
7 // Wrapped error (error chaining)
8 err = Error{
9     message: "failed to read config",
10    source: original_error
11 }
```

7.3 Error Return Pattern

Functions that can fail return a tuple of (result, Error?):

```
1 fn divide(a: int, b: int) -> (int, Error?) {
2     if b == 0 {
3         return 0, Error{message: "division by zero"}
4     }
5     return a / b, nil
6 }
7
8 fn read_file(path: string) -> (string, Error?) {
9     // Implementation
10 }
11
12 fn parse_int(s: string) -> (int, Error?) {
13     // Implementation
14 }
```

7.4 Handling Errors

7.4.1 Basic Error Checking

```
1 result, err = divide(10, 0)
2 if err != nil {
3     io.println("Error: " + err.message)
4     return
5 }
6 io.println("Result: " + to_string(result))
```

7.4.2 Propagating Errors

```
1 fn process_file(path: string) -> (Data, Error?) {
2     content, err = read_file(path)
3     if err != nil {
4         return Data{}, err // Propagate error
5     }
6
7     data, err = parse_data(content)
8     if err != nil {
9         return Data{}, err // Propagate error
10    }
11
12    return data, nil
13 }
```

7.4.3 Wrapping Errors

Add context when propagating:

```
1 fn load_config(path: string) -> (Config, Error?) {
```

```
2     content, err = read_file(path)
3     if err != nil {
4         return Config{}, Error{
5             message: "failed to load config from " + path,
6             source: err
7         }
8     }
9
10    config, err = parse_config(content)
11    if err != nil {
12        return Config{}, Error{
13            message: "invalid config format",
14            source: err
15        }
16    }
17
18    return config, nil
19 }
```

7.5 Error Checking Patterns

7.5.1 Guard Pattern

Check errors early and return:

```
1 fn process(input: string) -> (Output, Error?) {
2     // Guard: validate input
3     if string.len(input) == 0 {
4         return Output{}, Error{message: "input cannot be empty"}
5     }
6
7     // Guard: check prerequisites
8     ready, err = check_ready()
9     if err != nil {
10        return Output{}, err
11    }
12    if not ready {
13        return Output{}, Error{message: "system not ready"}
14    }
15
16    // Main logic
17    result = do_processing(input)
18    return result, nil
19 }
```

7.5.2 Defer for Cleanup

Use **defer** to ensure cleanup happens even on error:

```
1 fn with_file(path: string) -> (string, Error?) {
2     file, err = fs.open(path)
3     if err != nil {
4         return "", err
5     }
6     defer fs.close(file) // Always closes, even on error
```

```
7
8     content, err = fs.read_all(file)
9     if err != nil {
10         return "", err // file is closed by defer
11     }
12
13     return content, nil // file is closed by defer
14 }
```

7.6 Error Utilities

7.6.1 Checking Error Types

```
1 fn is_not_found(err: Error?) -> bool {
2     if err == nil {
3         return false
4     }
5     return err.code == 404 or string.contains(err.message, "not
6         found")
7 }
8
9 fn is_permission_denied(err: Error?) -> bool {
10     if err == nil {
11         return false
12     }
13     return err.code == 403
14 }
```

7.6.2 Error Chain Traversal

```
1 fn root_cause(err: Error?) -> Error? {
2     if err == nil {
3         return nil
4     }
5     current = err
6     while current.source != nil {
7         current = current.source
8     }
9     return current
10 }
11
12 fn error_chain(err: Error?) -> []Error {
13     errors = []
14     current = err
15     while current != nil {
16         errors = array.push(errors, current)
17         current = current.source
18     }
19     return errors
20 }
```


7.7 Option Types and Errors

For operations that may not find a value (not an error condition), use option types:

```
1 // Use option when "not found" is normal
2 fn find_user(id: int) -> User? {
3     // Returns nil if not found
4 }
5
6 // Use error when "not found" is exceptional
7 fn get_user(id: int) -> (User, Error?) {
8     // Returns error if not found
9 }
```

7.7.1 Converting Between Options and Errors

```
1 // Option to error
2 fn require_user(id: int) -> (User, Error?) {
3     user = find_user(id)
4     if user == nil {
5         return User{}, Error{message: "user not found", code:
6             404}
7     }
8     return user, nil
9 }
10
11 // Error to option (ignoring error)
12 fn try_parse_int(s: string) -> int? {
13     result, err = parse_int(s)
14     if err != nil {
15         return nil
16     }
17     return result
18 }
```

7.8 Panic and Recovery

For truly unrecoverable situations, use `panic`:

```
1 fn assert(condition: bool, message: string) {
2     if not condition {
3         panic(message)
4     }
5 }
6
7 fn unwrap<T>(opt: T?, message: string) -> T {
8     if opt == nil {
9         panic(message)
10    }
11    return opt
12 }
```

Warning

panic terminates the program. Use it only for programming errors, not for expected failure conditions. Prefer returning errors for all recoverable situations.

7.8.1 When to Use Panic

- Programming bugs (assertions)
- Invariant violations
- Unrecoverable initialization failures
- Unreachable code paths

7.8.2 When to Use Errors

- File not found
- Network failures
- Invalid user input
- Resource exhaustion
- Any expected failure condition

7.9 Best Practices

7.9.1 Return Errors, Don't Log and Continue

```
1 // Bad: logs and continues with invalid state
2 fn process(data: Data) -> Result {
3     result, err = validate(data)
4     if err != nil {
5         log.error("validation failed: " + err.message)
6         // Continues with unvalidated data!
7     }
8     // ...
9 }
10
11 // Good: returns error to caller
12 fn process(data: Data) -> (Result, Error?) {
13     result, err = validate(data)
14     if err != nil {
15         return Result{}, Error{
16             message: "validation failed",
17             source: err
18         }
19     }
20     // ...
21 }
```

7.9.2 Add Context When Wrapping

```

1 // Bad: just propagates
2 if err != nil {
3     return Result{}, err
4 }
5
6 // Good: adds context
7 if err != nil {
8     return Result{}, Error{
9         message: "failed to process user " + to_string(user_id),
10        source: err
11    }
12 }

```

7.9.3 Handle Errors at the Right Level

```

1 // Low-level: specific error
2 fn read_config_file(path: string) -> ([]byte, Error?) {
3     // Returns "file not found" or "permission denied"
4 }
5
6 // Mid-level: wrapped with context
7 fn load_config(path: string) -> (Config, Error?) {
8     data, err = read_config_file(path)
9     if err != nil {
10        return Config{}, Error{
11            message: "could not load config",
12            source: err
13        }
14    }
15    // Parse...
16 }
17
18 // High-level: user-friendly handling
19 fn main() {
20     config, err = load_config("config.json")
21     if err != nil {
22         io.println("Error: " + err.message)
23         // Could show root_cause for debugging
24         os.exit(1)
25     }
26     // Continue...
27 }

```

7.10 Error Handling Grammar

Error handling uses standard language constructs:

```

1 error_return = "(" type "," "Error?" ")" ;
2
3 error_check = "if" identifier "!=" "nil" block ;
4

```

```
5 error_create = "Error" "{"  
6     "message" ":" expression  
7     [ "," "code" ":" expression ]  
8     [ "," "source" ":" expression ]  
9     "}" ;
```

Part II

Concurrency and Modules

Chapter 8

Concurrency

HAIRA provides Go-style concurrency primitives: lightweight spawned tasks and channels for communication. The philosophy is “share memory by communicating” rather than “communicate by sharing memory.”

8.1 Concurrency Model

- **Spawned tasks** – Lightweight concurrent execution units
- **Channels** – Typed conduits for communication
- **Select** – Multiplexing channel operations
- **No shared mutable state** – Communication via channels

8.2 Spawn

The **spawn** keyword starts a new concurrent task:

```
1 import "io"
2 import "time"
3
4 fn main() {
5     spawn {
6         time.sleep(1000)
7         io.println("Hello from spawned task!")
8     }
9
10    io.println("Main continues immediately")
11    time.sleep(2000) // Wait for spawned task
12 }
```

8.2.1 Spawning Functions

```
1 fn worker(id: int) {
2     io.println("Worker " + to_string(id) + " started")
3     time.sleep(1000)
4     io.println("Worker " + to_string(id) + " done")
5 }
6
```

```

7 fn main() {
8     for i in 0..5 {
9         spawn worker(i)
10    }
11    time.sleep(3000) // Wait for all workers
12 }

```

8.2.2 Spawn Returns Handle

```

1 fn compute() -> int {
2     time.sleep(1000)
3     return 42
4 }
5
6 fn main() {
7     handle = spawn compute()
8
9     // Do other work...
10
11    result = await handle // Wait for result
12    io.println("Result: " + to_string(result))
13 }

```

8.3 Channels

Channels are typed conduits for passing values between tasks:

```

1 // Create a channel
2 ch = chan<int>() // Unbuffered channel
3 ch = chan<int>(10) // Buffered channel (capacity 10)
4 ch = chan<string>() // String channel

```

8.3.1 Sending and Receiving

```

1 ch = chan<int>()
2
3 // Send (blocks until received for unbuffered)
4 ch <- 42
5
6 // Receive (blocks until value available)
7 value = <-ch

```

8.3.2 Basic Channel Example

```

1 import "io"
2
3 fn main() {
4     ch = chan<string>()
5

```



```
6     spawn {
7         ch <- "Hello from spawned task!"
8     }
9
10    message = <-ch
11    io.println(message)
12 }
```

8.3.3 Buffered Channels

```
1 ch = chan<int>(3) // Buffer size 3
2
3 // These don't block (buffer has space)
4 ch <- 1
5 ch <- 2
6 ch <- 3
7
8 // This would block (buffer full)
9 // ch <- 4
10
11 // Receive
12 io.println(<-ch) // 1
13 io.println(<-ch) // 2
14 io.println(<-ch) // 3
```

8.3.4 Closing Channels

```
1 ch = chan<int>(10)
2
3 spawn {
4     for i in 0..10 {
5         ch <- i
6     }
7     close(ch) // Signal no more values
8 }
9
10 // Iterate until channel closed
11 for value in ch {
12     io.println(value)
13 }
14
15 io.println("Channel closed, loop exited")
```

8.3.5 Checking Channel State

```
1 // Receive with closed check
2 value, ok = <-ch
3 if not ok {
4     io.println("Channel closed")
5 }
6
```

```

7 // Non-blocking receive
8 value, ok = ch.try_receive()
9 if ok {
10     io.println("Got: " + to_string(value))
11 } else {
12     io.println("No value available")
13 }
14
15 // Non-blocking send
16 ok = ch.try_send(42)
17 if not ok {
18     io.println("Channel full or closed")
19 }

```

8.4 Select

The **select** statement waits on multiple channel operations:

```

1 import "io"
2 import "time"
3
4 fn main() {
5     ch1 = chan<string>()
6     ch2 = chan<string>()
7
8     spawn {
9         time.sleep(1000)
10        ch1 <- "from channel 1"
11    }
12
13    spawn {
14        time.sleep(500)
15        ch2 <- "from channel 2"
16    }
17
18    // Wait for first available
19    select {
20        msg = <-ch1 => io.println("Received: " + msg)
21        msg = <-ch2 => io.println("Received: " + msg)
22    }
23 }

```

8.4.1 Select with Default

Non-blocking select:

```

1 select {
2     value = <-ch => io.println("Got: " + to_string(value))
3     default => io.println("No value ready")
4 }

```

8.4.2 Select with Timeout

```

1 timeout = time.after(5000) // 5 seconds
2
3 select {
4     value = <-ch => io.println("Got: " + to_string(value))
5     <-timeout => io.println("Timed out!")
6 }

```

8.4.3 Select in Loop

```

1 fn worker(jobs: chan<Job>, results: chan<Result>, done:
   chan<bool>) {
2     while true {
3         select {
4             job = <-jobs => {
5                 result = process(job)
6                 results <- result
7             }
8             <-done => {
9                 io.println("Worker shutting down")
10                return
11            }
12        }
13    }
14 }

```

8.5 Common Patterns

8.5.1 Worker Pool

```

1 fn worker(id: int, jobs: chan<int>, results: chan<int>) {
2     for job in jobs {
3         io.println("Worker " + to_string(id) + " processing " +
4             to_string(job))
5         time.sleep(100)
6         results <- job * 2
7     }
8 }
9
10 fn main() {
11     num_jobs = 10
12     num_workers = 3
13
14     jobs = chan<int>(num_jobs)
15     results = chan<int>(num_jobs)
16
17     // Start workers
18     for i in 0..num_workers {
19         spawn worker(i, jobs, results)
20     }
21
22     // Send jobs
23     for j in 0..num_jobs {

```

```
23     jobs <- j
24   }
25   close(jobs)
26
27   // Collect results
28   for _ in 0..num_jobs {
29     result = <-results
30     io.println("Result: " + to_string(result))
31   }
32 }
```

8.5.2 Fan-Out, Fan-In

```
1  fn producer(out: chan<int>) {
2    for i in 0..100 {
3      out <- i
4    }
5    close(out)
6  }
7
8  fn worker(in: chan<int>, out: chan<int>) {
9    for value in in {
10     out <- value * value
11   }
12 }
13
14 fn main() {
15   input = chan<int>(100)
16   output = chan<int>(100)
17
18   // Producer
19   spawn producer(input)
20
21   // Fan-out to multiple workers
22   for _ in 0..4 {
23     spawn worker(input, output)
24   }
25
26   // Fan-in (collect results)
27   spawn {
28     time.sleep(1000)
29     close(output)
30   }
31
32   for result in output {
33     io.println(result)
34   }
35 }
```

8.5.3 Pipeline

```
1  fn generate(out: chan<int>) {
2    for i in 2..100 {
```

```

3     out <- i
4   }
5   close(out)
6 }
7
8 fn filter(in: chan<int>, out: chan<int>, prime: int) {
9   for n in in {
10     if n % prime != 0 {
11       out <- n
12     }
13   }
14   close(out)
15 }
16
17 fn sieve() -> chan<int> {
18   primes = chan<int>()
19
20   spawn {
21     ch = chan<int>(100)
22     spawn generate(ch)
23
24     while true {
25       prime, ok = <-ch
26       if not ok {
27         break
28       }
29       primes <- prime
30
31       next = chan<int>(100)
32       spawn filter(ch, next, prime)
33       ch = next
34     }
35     close(primes)
36   }
37
38   return primes
39 }
40
41 fn main() {
42   for prime in sieve() {
43     io.println(prime)
44     if prime > 50 {
45       break
46     }
47   }
48 }

```

8.5.4 Semaphore

```

1 struct Semaphore {
2   ch: chan<bool>
3 }
4
5 fn new_semaphore(n: int) -> Semaphore {
6   sem = Semaphore{ch: chan<bool>(n)}
7   for _ in 0..n {

```

```

8         sem.ch <- true
9     }
10    return sem
11 }
12
13 fn (s: Semaphore) acquire() {
14     <-s.ch
15 }
16
17 fn (s: Semaphore) release() {
18     s.ch <- true
19 }
20
21 // Usage: limit concurrent operations
22 fn main() {
23     sem = new_semaphore(3) // Max 3 concurrent
24
25     for i in 0..10 {
26         spawn {
27             sem.acquire()
28             defer sem.release()
29
30             io.println("Task " + to_string(i) + " running")
31             time.sleep(1000)
32         }
33     }
34
35     time.sleep(5000)
36 }

```

8.6 Channel Types

8.6.1 Directional Channels

Functions can restrict channel direction:

```

1 // Send-only channel
2 fn producer(out: chan<-int) {
3     out <- 42
4     // <-out // Error: cannot receive from send-only channel
5 }
6
7 // Receive-only channel
8 fn consumer(in: <-chan<int>) {
9     value = <-in
10    // in <- 42 // Error: cannot send to receive-only channel
11 }
12
13 fn main() {
14     ch = chan<int>()
15     spawn producer(ch) // Converts to chan<-int
16     spawn consumer(ch) // Converts to <-chan<int>
17 }

```

8.7 Synchronization

8.7.1 WaitGroup

Wait for multiple tasks to complete:

```
1 import "sync"
2
3 fn main() {
4     wg = sync.WaitGroup{}
5
6     for i in 0..5 {
7         wg.add(1)
8         spawn {
9             defer wg.done()
10            io.println("Task " + to_string(i))
11            time.sleep(1000)
12        }
13    }
14
15    wg.wait() // Block until all done
16    io.println("All tasks completed")
17 }
```

8.7.2 Mutex

For rare cases requiring shared state:

```
1 import "sync"
2
3 struct Counter {
4     mu: sync.Mutex
5     value: int
6 }
7
8 fn (c: *Counter) increment() {
9     c.mu.lock()
10    defer c.mu.unlock()
11    c.value += 1
12 }
13
14 fn (c: *Counter) get() -> int {
15     c.mu.lock()
16     defer c.mu.unlock()
17     return c.value
18 }
```

Warning

Prefer channels over mutexes. Mutexes are provided for interoperability and performance-critical sections, but channel-based designs are generally safer and more idiomatic.

8.8 Concurrency Grammar

```
1 spawn_expr = "spawn" ( block | function_call ) ;
2
3 channel_type = "chan" "<" type ">"
4               | "chan<-" type          (* send-only *)
5               | "<-chan<" type ">"      (* receive-only *) ;
6
7 channel_create = "chan" "<" type ">" "(" [ expression ] ")" ;
8
9 send_stmt = expression "<-" expression ;
10
11 receive_expr = "<-" expression ;
12
13 select_stmt = "select" "{" { select_case } [ default_case ] "}"
14             ;
15 select_case = pattern "=" receive_expr "=>" ( expression |
16         block )
17         | send_stmt "=>" ( expression | block ) ;
18 default_case = "default" "=>" ( expression | block ) ;
```


Chapter 9

Modules and Imports

HAIRA uses explicit imports for all dependencies. Every external symbol must be imported before use.

9.1 Module System Overview

- **Explicit imports** – All dependencies declared at file top
- **No implicit globals** – Everything must be imported
- **Simple resolution** – Standard library, project-local, external
- **Namespaced access** – `module.function()` syntax

9.2 Import Syntax

9.2.1 Basic Import

```
1 import "io"
2 import "json"
3 import "http"
4
5 fn main() {
6     io.println("Hello")
7 }
```

9.2.2 Aliased Import

```
1 import fmt from "io"
2 import db from "haira/postgres"
3
4 fn main() {
5     fmt.println("Hello")
6     db.connect("localhost:5432")
7 }
```

9.2.3 Selective Import

Import specific symbols:

```
1 import { println, readln } from "io"
2 import { User, Post } from "models"
3
4 fn main() {
5     println("Enter name:")
6     name = readln()
7     println("Hello, " + name)
8 }
```

9.2.4 Glob Import

Import all exports (use sparingly):

```
1 import * from "math"
2
3 fn main() {
4     result = sqrt(pow(3.0, 2.0) + pow(4.0, 2.0))
5     io.println(result) // 5.0
6 }
```

Warning

Glob imports can cause naming conflicts and make code harder to read. Prefer explicit imports.

9.3 Module Resolution

Imports are resolved in this order:

1. **Standard library** – Built-in modules ("io", "json", etc.)
2. **Project-local** – Relative to project root ("models/user")
3. **External packages** – From package registry ("github.com/...")

9.3.1 Standard Library

```
1 import "io"           // Standard I/O
2 import "string"        // String manipulation
3 import "math"          // Mathematical functions
4 import "json"          // JSON encoding/decoding
5 import "http"          // HTTP client/server
6 import "fs"            // File system
7 import "os"            // Operating system
8 import "time"          // Time and duration
9 import "crypto"        // Cryptography
10 import "regex"         // Regular expressions
```

9.3.2 Project-Local Imports

```

1 // Project structure:
2 // myapp/
3 //   main.haira
4 //   models/
5 //     user.haira
6 //     post.haira
7 //   utils/
8 //     helpers.haira
9
10 // In main.haira:
11 import "models/user"
12 import "models/post"
13 import "utils/helpers"
14
15 // In models/post.haira:
16 import "models/user" // Absolute from project root

```

9.3.3 External Packages

```

1 import "github.com/haira-libs/aws"
2 import "github.com/haira-libs/postgres"
3 import pg from "github.com/haira-libs/postgres"

```

External packages are declared in `haira.toml`:

```

1 [dependencies]
2 aws = "github.com/haira-libs/aws@1.0.0"
3 postgres = "github.com/haira-libs/postgres@2.1.0"

```

9.4 Module Definitions

9.4.1 File as Module

Each `.haira` file is a module. The file name determines the module name:

```

1 // File: utils/helpers.haira
2 // Module: utils/helpers
3
4 fn format_name(first: string, last: string) -> string {
5     return first + " " + last
6 }
7
8 fn validate_email(email: string) -> bool {
9     return string.contains(email, "@")
10 }

```

9.4.2 Exporting

By default, all top-level declarations are exported. Use underscore prefix for private:

```
1 // Public (exported)
2 fn public_function() { }
3 struct PublicType { }
4
5 // Private (not exported)
6 fn _private_function() { }
7 struct _PrivateType { }
8
9 // Private helper
10 _cache = [:]
```

9.4.3 Package Initialization

Optional `init()` function runs when module is first imported:

```
1 // database.haira
2
3 _pool: ConnectionPool?
4
5 fn init() {
6     _pool = create_pool()
7     io.println("Database module initialized")
8 }
9
10 fn get_connection() -> (Connection, Error?) {
11     if _pool == nil {
12         return Connection{}, Error{message: "not initialized"}
13     }
14     return _pool.get(), nil
15 }
```

Note

`init()` functions are called in dependency order. Circular dependencies are a compile-time error.

9.5 Package Structure

9.5.1 Single-File Package

```
1 myapp/
2   main.haira
3   haira.toml
```

9.5.2 Multi-File Package

```
1 myapp/
2   main.haira
3   models/
4     user.haira
```

```
5     post.haira
6     mod.haira      # Package entry point (optional)
7     services/
8     auth.haira
9     api.haira
10    utils/
11    helpers.haira
12    haira.toml
```

9.5.3 The mod.haira File

Re-exports from a directory:

```
1 // models/mod.haira
2 import { User } from "models/user"
3 import { Post } from "models/post"
4
5 // Re-export
6 export { User, Post }
```

Now consumers can import from the directory:

```
1 import { User, Post } from "models"
```

9.6 Import Order

Imports should be organized in groups:

```
1 // 1. Standard library
2 import "io"
3 import "json"
4 import "http"
5
6 // 2. External packages
7 import "github.com/haira-libs/aws"
8
9 // 3. Project-local
10 import "models/user"
11 import "services/auth"
```

9.7 Circular Dependencies

Circular imports are not allowed:

```
1 // a.haira
2 import "b" // Error if b imports a
3
4 // b.haira
5 import "a" // Creates circular dependency
```

Solution: Extract shared code to a third module:

```

1 // shared.haira
2 struct Common { }
3
4 // a.haira
5 import "shared"
6
7 // b.haira
8 import "shared"

```

9.8 Conditional Imports

Import based on build configuration:

```

1 #[cfg(os = "windows")]
2 import "windows/api"
3
4 #[cfg(os = "linux")]
5 import "linux/api"
6
7 #[cfg(os = "macos")]
8 import "macos/api"

```

9.9 The haira.toml File

Project configuration:

```

1 [package]
2 name = "myapp"
3 version = "1.0.0"
4 authors = ["Your Name <you@example.com>"]
5
6 [dependencies]
7 aws = "github.com/haira-libs/aws@1.0.0"
8 postgres = "github.com/haira-libs/postgres@2.1.0"
9
10 [dev-dependencies]
11 testing = "github.com/haira-libs/testing@1.0.0"
12
13 [build]
14 target = "native"
15 optimization = "release"
16
17 [ai]
18 backend = "llama.cpp"
19 model_path = "~/.haira/models/codellama-7b.gguf"

```

9.10 Module Grammar

```

1 import_statement = "import" import_spec ;
2

```

```
3 import_spec = string_literal (* basic
   *)
4         | identifier "from" string_literal (*
   aliased *)
5         | "{" identifier_list "}" "from" string_literal (*
   selective *)
6         | "*" "from" string_literal ; (* glob *)
7
8 export_statement = "export" "{" identifier_list "}" ;
9
10 identifier_list = identifier { "," identifier } ;
11
12 module = { import_statement } { export_statement } { top_level
   } ;
```


Chapter 10

Standard Library

The HAIRA standard library provides essential functionality for common programming tasks. All modules must be explicitly imported.

10.1 Core Modules

Module	Description
<code>io</code>	Input/output operations
<code>string</code>	String manipulation
<code>array</code>	Array operations
<code>map</code>	Map/dictionary operations
<code>math</code>	Mathematical functions
<code>conv</code>	Type conversions

Table 10.1: Core modules

10.2 io Module

Basic input/output operations.

```
1 import "io"
2
3 // Output
4 io.print("Hello")           // No newline
5 io.println("Hello")        // With newline
6 io.printf("Value: %d\n", 42) // Formatted
7
8 // Input
9 line = io.readLine()        // Read line from stdin
10
11 // Errors
12 io.eprintln("Error!")      // Print to stderr
```

10.2.1 io Functions

```
1 fn print(s: string)
```

```

2 fn println(s: string)
3 fn printf(format: string, args: ...any)
4 fn readln() -> string
5 fn eprintln(s: string)
6 fn eprintf(format: string, args: ...any)

```

10.3 string Module

String manipulation functions.

```

1 import "string"
2
3 s = "  Hello, World!  "
4
5 // Basic operations
6 string.len(s) // 18
7 string.is_empty("") // true
8
9 // Trimming
10 string.trim(s) // "Hello, World!"
11 string.trim_left(s) // "Hello, World!  "
12 string.trim_right(s) // "  Hello, World!"
13
14 // Case
15 string.to_upper(s) // "  HELLO, WORLD!  "
16 string.to_lower(s) // "  hello, world!  "
17
18 // Search
19 string.contains(s, "World") // true
20 string.starts_with(s, " H") // true
21 string.ends_with(s, "! ") // true
22 string.index_of(s, "o") // 4
23 string.last_index_of(s, "o") // 8
24
25 // Split/Join
26 string.split("a,b,c", ",") // ["a", "b", "c"]
27 string.join(["a", "b"], "-") // "a-b"
28
29 // Substring
30 string.substring(s, 2, 7) // "Hello"
31 string.char_at(s, 2) // "H"
32
33 // Replace
34 string.replace(s, "World", "Haira") // "  Hello, Haira!  "
35 string.replace_all(s, " ", "_") // "__Hello,_World!__"
36
37 // Repeat
38 string.repeat("ab", 3) // "ababab"

```

10.3.1 string Functions

```

1 fn len(s: string) -> int
2 fn is_empty(s: string) -> bool
3 fn trim(s: string) -> string

```

```

4 fn trim_left(s: string) -> string
5 fn trim_right(s: string) -> string
6 fn to_upper(s: string) -> string
7 fn to_lower(s: string) -> string
8 fn contains(s: string, sub: string) -> bool
9 fn starts_with(s: string, prefix: string) -> bool
10 fn ends_with(s: string, suffix: string) -> bool
11 fn index_of(s: string, sub: string) -> int
12 fn last_index_of(s: string, sub: string) -> int
13 fn split(s: string, sep: string) -> []string
14 fn join(parts: []string, sep: string) -> string
15 fn substring(s: string, start: int, end: int) -> string
16 fn char_at(s: string, index: int) -> string
17 fn replace(s: string, old: string, new: string) -> string
18 fn replace_all(s: string, old: string, new: string) -> string
19 fn repeat(s: string, n: int) -> string

```

10.4 array Module

Array operations.

```

1 import "array"
2
3 arr = [1, 2, 3, 4, 5]
4
5 // Basic
6 array.len(arr) // 5
7 array.is_empty([]) // true
8
9 // Access
10 array.first(arr) // 1 (or nil if empty)
11 array.last(arr) // 5 (or nil if empty)
12 array.get(arr, 2) // 3 (or nil if out of bounds)
13
14 // Modification (returns new array)
15 array.push(arr, 6) // [1, 2, 3, 4, 5, 6]
16 array.pop(arr) // ([1, 2, 3, 4], 5)
17 array.insert(arr, 0, 0) // [0, 1, 2, 3, 4, 5]
18 array.remove(arr, 2) // [1, 2, 4, 5]
19
20 // Slicing
21 array.slice(arr, 1, 4) // [2, 3, 4]
22 array.take(arr, 3) // [1, 2, 3]
23 array.drop(arr, 2) // [3, 4, 5]
24
25 // Search
26 array.contains(arr, 3) // true
27 array.index_of(arr, 3) // 2
28 array.find(arr, fn(x) { x > 3 }) // 4
29
30 // Transform
31 array.map(arr, fn(x) { x * 2 }) // [2, 4, 6, 8, 10]
32 array.filter(arr, fn(x) { x > 2 }) // [3, 4, 5]
33 array.reduce(arr, 0, fn(acc, x) { acc + x }) // 15
34
35 // Sort

```

```

36 array.sort(arr) // [1, 2, 3, 4, 5]
37 array.sort_by(arr, fn(a, b) { b - a }) // [5, 4, 3, 2, 1]
38
39 // Other
40 array.reverse(arr) // [5, 4, 3, 2, 1]
41 array.concat(arr, [6, 7]) // [1, 2, 3, 4, 5, 6, 7]
42 array.flatten([[1, 2], [3, 4]]) // [1, 2, 3, 4]
43 array.unique([1, 2, 2, 3]) // [1, 2, 3]

```

10.5 map Module

Map/dictionary operations.

```

1  import "map"
2
3  m = {"a": 1, "b": 2, "c": 3}
4
5  // Basic
6  map.len(m) // 3
7  map.is_empty(m) // false
8
9  // Access
10 map.get(m, "a") // 1 (or nil if not found)
11 map.has(m, "a") // true
12
13 // Modification (returns new map)
14 map.set(m, "d", 4) // {"a": 1, "b": 2, "c": 3, "d": 4}
15 map.remove(m, "b") // {"a": 1, "c": 3}
16
17 // Iteration helpers
18 map.keys(m) // ["a", "b", "c"]
19 map.values(m) // [1, 2, 3]
20 map.entries(m) // [{"a", 1}, {"b", 2}, {"c", 3}]
21
22 // Transform
23 map.map_values(m, fn(v) { v * 2 }) // {"a": 2, "b": 4, "c": 6}
24 map.filter(m, fn(k, v) { v > 1 }) // {"b": 2, "c": 3}
25
26 // Merge
27 map.merge(m, [{"d": 4}]) // {"a": 1, "b": 2, "c": 3, "d": 4}

```

10.6 math Module

Mathematical functions.

```

1  import "math"
2
3  // Constants
4  math.PI // 3.14159265358979
5  math.E // 2.71828182845905
6

```

```

7  // Basic
8  math.abs(-5)                // 5
9  math.min(3, 7)              // 3
10 math.max(3, 7)              // 7
11 math.clamp(5, 0, 10)        // 5
12 math.clamp(-5, 0, 10)       // 0
13
14 // Rounding
15 math.floor(3.7)              // 3.0
16 math.ceil(3.2)              // 4.0
17 math.round(3.5)             // 4.0
18 math.trunc(3.7)             // 3.0
19
20 // Power/Root
21 math.pow(2.0, 10.0)          // 1024.0
22 math.sqrt(16.0)             // 4.0
23 math.cbrt(27.0)             // 3.0
24
25 // Exponential/Logarithm
26 math.exp(1.0)                // 2.718...
27 math.log(math.E)             // 1.0
28 math.log10(100.0)           // 2.0
29 math.log2(8.0)              // 3.0
30
31 // Trigonometry
32 math.sin(0.0)                // 0.0
33 math.cos(0.0)                // 1.0
34 math.tan(0.0)                // 0.0
35 math.asin(0.0)              // 0.0
36 math.acos(1.0)              // 0.0
37 math.atan(0.0)              // 0.0
38 math.atan2(1.0, 1.0)        // 0.785... (PI/4)
39
40 // Random
41 math.random()                // Random float 0.0..1.0
42 math.random_int(1, 100)      // Random int 1..100

```

10.7 conv Module

Type conversion utilities.

```

1  import "conv"
2
3  // To string
4  conv.int_to_string(42)       // "42"
5  conv.float_to_string(3.14)   // "3.14"
6  conv.bool_to_string(true)    // "true"
7
8  // From string
9  conv.string_to_int("42")     // (42, nil)
10 conv.string_to_int("abc")     // (0, Error)
11 conv.string_to_float("3.14") // (3.14, nil)
12 conv.string_to_bool("true")   // (true, nil)
13
14 // Number conversions
15 conv.int_to_float(42)         // 42.0

```

```
16 conv.float_to_int(3.7)           // 3 (truncates)
17
18 // Base conversions
19 conv.int_to_hex(255)              // "ff"
20 conv.int_to_binary(10)           // "1010"
21 conv.int_to_octal(8)             // "10"
22 conv.hex_to_int("ff")            // (255, nil)
```

10.8 Extended Modules

10.8.1 fs Module

File system operations.

```
1 import "fs"
2
3 // Read/Write
4 content, err = fs.read_file("file.txt")
5 err = fs.write_file("file.txt", "content")
6 err = fs.append_file("file.txt", "more")
7
8 // File operations
9 exists = fs.exists("file.txt")
10 err = fs.remove("file.txt")
11 err = fs.rename("old.txt", "new.txt")
12 err = fs.copy("src.txt", "dst.txt")
13
14 // Directory operations
15 err = fs.mkdir("dir")
16 err = fs.mkdir_all("path/to/dir")
17 entries, err = fs.read_dir("dir")
18 err = fs.remove_all("dir")
19
20 // File info
21 info, err = fs.stat("file.txt")
22 info.size           // Size in bytes
23 info.is_dir         // Is directory?
24 info.modified       // Modification time
```

10.8.2 os Module

Operating system interface.

```
1 import "os"
2
3 // Environment
4 os.getenv("HOME")           // Get env variable
5 os.setenv("KEY", "value")   // Set env variable
6 os.environ()                // All env variables
7
8 // Process
9 os.args                     // Command line arguments
10 os.exit(0)                  // Exit with code
11 os.getpid()                 // Process ID
12
```

```

13 // Execution
14 output, err = os.exec("ls", ["-la"])

```

10.8.3 time Module

Time and duration.

```

1  import "time"
2
3  // Current time
4  now = time.now()
5  now.year           // 2024
6  now.month          // 1-12
7  now.day            // 1-31
8  now.hour           // 0-23
9  now.minute         // 0-59
10 now.second          // 0-59
11
12 // Formatting
13 time.format(now, "2006-01-02") // "2024-03-15"
14
15 // Parsing
16 t, err = time.parse("2024-03-15", "2006-01-02")
17
18 // Duration
19 time.sleep(1000)           // Sleep 1 second (ms)
20 duration = time.Since(start) // Time since start
21
22 // Timers
23 timer = time.After(5000)   // Channel that fires after 5s
24 ticker = time.Tick(1000)   // Channel that fires every 1s

```

10.8.4 json Module

JSON encoding/decoding.

```

1  import "json"
2
3  struct User {
4      name: string
5      age: int
6  }
7
8  // Encoding
9  user = User{name: "Alice", age: 30}
10 data, err = json.encode(user) // '{"name": "Alice", "age": 30}'
11
12 // Decoding
13 user, err = json.decode<User>(data)
14
15 // Pretty printing
16 data, err = json.encode_pretty(user)
17
18 // Raw JSON
19 value, err = json.parse('{"key": "value"}')

```

```
20 value["key"] // "value"
```

10.8.5 http Module

HTTP client and server.

```
1 import "http"
2
3 // GET request
4 response, err = http.get("https://api.example.com/users")
5 response.status // 200
6 response.body // Response body
7
8 // POST request
9 response, err = http.post(
10     "https://api.example.com/users",
11     [{"Content-Type": "application/json"}],
12     '{"name": "Alice"}'
13 )
14
15 // Server
16 server = http.Server{port: 8080}
17
18 server.handle("/", fn(req: http.Request) -> http.Response {
19     return http.Response{
20         status: 200,
21         body: "Hello, World!"
22     }
23 })
24
25 server.listen()
```

10.8.6 regex Module

Regular expressions.

```
1 import "regex"
2
3 // Compile pattern
4 pattern, err = regex.compile(r"\d+")
5
6 // Match
7 regex.is_match(pattern, "abc123") // true
8
9 // Find
10 match = regex.find(pattern, "abc123def") // "123"
11 matches = regex.find_all(pattern, "a1b2c3") // ["1", "2", "3"]
12
13 // Replace
14 regex.replace(pattern, "a1b2", "X") // "aXb2"
15 regex.replace_all(pattern, "a1b2", "X") // "aXbX"
16
17 // Capture groups
18 pattern, _ = regex.compile(r"(\w+)@(\w+)")
19 captures = regex.captures(pattern, "alice@example")
```



```
20 // captures[0] = "alice@example"
21 // captures[1] = "alice"
22 // captures[2] = "example"
```

10.8.7 crypto Module

Cryptographic functions.

```
1 import "crypto"
2
3 // Hashing
4 crypto.md5("hello")           // Hash as hex string
5 crypto.sha256("hello")
6 crypto.sha512("hello")
7
8 // HMAC
9 crypto.hmac_sha256("message", "key")
10
11 // Random
12 crypto.random_bytes(32)       // 32 random bytes
13
14 // Encoding
15 crypto.base64_encode(bytes)
16 crypto.base64_decode(string)
17 crypto.hex_encode(bytes)
18 crypto.hex_decode(string)
```

10.8.8 net Module

Low-level networking.

```
1 import "net"
2
3 // TCP client
4 conn, err = net.dial("tcp", "example.com:80")
5 conn.write("GET / HTTP/1.1\r\n\r\n")
6 data, err = conn.read(1024)
7 conn.close()
8
9 // TCP server
10 listener, err = net.listen("tcp", ":8080")
11 while true {
12     conn, err = listener.accept()
13     spawn handle_connection(conn)
14 }
```

10.9 Testing Module

```
1 import "testing"
2
3 fn test_addition(t: testing.T) {
4     result = add(2, 3)
```

```
5     t.assert_eq(result, 5)
6 }
7
8 fn test_division(t: testing.T) {
9     result, err = divide(10, 2)
10    t.assert_nil(err)
11    t.assert_eq(result, 5)
12
13    _, err = divide(10, 0)
14    t.assert_not_nil(err)
15 }
```

Run tests with:

```
$ haira test
```

Part III

Agentic Orchestration

Chapter 11

Providers

Providers configure LLM backends. They are declared at the top level and referenced by agents.

11.1 Provider Declaration

```
1 provider provider_name {  
2     field: value  
3     field: value  
4 }
```

11.1.1 Cloud Providers

```
1 provider anthropic {  
2     api_key: env("ANTHROPIC_API_KEY")  
3     model: "claude-sonnet-4-20250514"  
4 }  
5  
6 provider openai {  
7     api_key: env("OPENAI_API_KEY")  
8     model: "gpt-4o"  
9 }
```

11.1.2 Local Providers

```
1 provider local {  
2     backend: "ollama"  
3     host: "localhost:11434"  
4     model: "llama3:8b"  
5 }
```

Field	Type	Required	Description
model	string	Yes	Default model identifier
api_key	string	No	API authentication key
backend	string	No	Backend type ("ollama", "llama.cpp")
host	string	No	Backend host address
temperature	float	No	Default temperature (0.0–2.0)
max_tokens	int	No	Default max tokens per response

Table 11.1: Provider fields

11.2 Provider Fields

11.3 Environment Variables

The `env()` function reads environment variables at runtime:

```

1 provider anthropic {
2     api_key: env("ANTHROPIC_API_KEY")
3     model: env("ANTHROPIC_MODEL") or "claude-sonnet-4-20250514"
4 }
```

Warning

Never hardcode API keys in source files. Always use `env()` to read them from the environment.

11.4 Referencing Providers

Providers are referenced by name in agent declarations:

```

1 agent Writer {
2     model: anthropic           // uses anthropic provider's
3     default model
4     system: "You are a writer."
5 }
6 agent FastHelper {
7     model: local               // uses local Ollama provider
8     system: "You are a fast helper."
9 }
```

11.5 Multiple Providers

A program can declare multiple providers. Agents choose which to use:

```

1 provider anthropic {
2     api_key: env("ANTHROPIC_API_KEY")
3     model: "claude-sonnet-4-20250514"
4 }
5
6 provider local {
```

```
7     backend: "ollama"
8     host: "localhost:11434"
9     model: "llama3:8b"
10 }
11
12 // Use expensive cloud model for important tasks
13 agent Analyst {
14     model: anthropic
15     system: "You analyze data carefully."
16 }
17
18 // Use cheap local model for simple tasks
19 agent Classifier {
20     model: local
21     system: "Classify the input into categories."
22 }
```

11.6 Provider Grammar

```
1 provider_decl = "provider" identifier "{" { provider_field }
   "}" ;
2 provider_field = identifier ":" expression ;
```


Chapter 12

Tools

Tools are typed functions with LLM-visible descriptions. The compiler auto-generates JSON schemas from the type signature, enabling agents to call tools with structured arguments.

12.1 Tool Declaration

```
1 tool tool_name(param: Type, param: Type = default) ->
  ReturnType {
2   """Description visible to the LLM"""
3
4   // implementation
5 }
```

A tool is like **fn** with two additions:

1. The **tool** keyword instead of **fn**
2. A mandatory triple-quoted description as the first element of the body

12.2 Basic Examples

12.2.1 HTTP Tool

```
1 import "tools/http"
2 import "json"
3
4 struct SearchResult {
5   url: string
6   title: string
7   snippet: string
8 }
9
10 tool search(query: string, max_results: int = 5) ->
  [SearchResult] {
11   """
12   Search the web for information.
13   Returns up to max_results results.
14   Supports boolean operators in query.
15   """
```

```

16
17     resp, err =
18         http.get("https://api.search.com?q={query}&n={max_results}")
19     if err != nil { return [], err }
20     return json.decode(resp.body, [SearchResult])

```

12.2.2 Database Tool

```

1 import "tools/postgres"
2
3 tool get_user(user_id: string) -> User {
4     """Look up a user by their ID"""
5
6     user, err = pg.query_one("SELECT * FROM users WHERE id =
7         $1", [user_id])
8     if err != nil { return User{}, err }
9     return user
10 }

```

12.2.3 External Service Tool

```

1 import "tools/http"
2
3 tool send_slack(channel: string, message: string) -> bool {
4     """Send a message to a Slack channel"""
5
6     resp, err =
7         http.post("https://slack.com/api/chat.postMessage", {
8             channel: channel,
9             text: message
10         })
11     return err == nil

```

12.2.4 Agent-Delegating Tool

Tools can call agents internally:

```

1 tool summarize(text: string) -> string {
2     """Summarize text into key points"""
3
4     result, _ = Writer.ask("Summarize concisely: {text}")
5     return result
6 }

```

12.3 Triple-Quote Description

The description is mandatory and must be the first element of the tool body. It is a triple-quoted string ("""...""") that can span multiple lines:

```

1  tool analyze(data: [DataPoint]) -> Analysis {
2      """
3      Analyze a dataset and produce a statistical summary.
4
5      Includes:
6      - Mean, median, and standard deviation
7      - Outlier detection
8      - Trend analysis over time
9
10     Use this tool when the user asks about patterns or
11       statistics.
12     """
13     // implementation
14 }

```

Note

The compiler will emit an error if a tool is missing its description. This is enforced because the description is sent to the LLM as part of the tool-calling protocol.

12.4 Auto-Generated JSON Schema

The compiler generates a JSON schema from the tool's type signature. For example:

```

1  tool search(query: string, max_results: int = 5) ->
2    [SearchResult] {
3      """Search the web for information"""
4      // ...
5  }

```

Generates (conceptually):

```

1  {
2    "name": "search",
3    "description": "Search the web for information",
4    "parameters": {
5      "type": "object",
6      "properties": {
7        "query": { "type": "string" },
8        "max_results": { "type": "integer", "default": 5 }
9      },
10     "required": ["query"]
11   }
12 }

```

This happens at compile time. No hand-written JSON schemas are needed.

12.5 Tool Packs (Standard Library)

HAIRA ships with built-in tool packs for common integrations:

```

1 import "tools/http"           // http.get, http.post, http.put,
   http.delete
2 import "tools/slack"          // slack.send, slack.read_channel
3 import "tools/github"         // github.create_issue,
   github.list_prs, github.get_diff
4 import "tools/postgres"       // pg.query, pg.query_one, pg.insert,
   pg.update
5 import "tools/email"          // email.send
6 import "tools/fs"             // fs.read, fs.write, fs.list

```

Tool packs provide pre-built **tool** declarations. They can be used directly in agent tools lists or called from within other tools and workflows.

12.6 Error Handling in Tools

Tools follow HAIRA's standard error handling pattern:

```

1 tool fetch_page(url: string) -> string {
2     """Fetch the content of a web page"""
3
4     resp, err = http.get(url)
5     if err != nil {
6         return "", err
7     }
8     if resp.status != 200 {
9         return "", Error{message: "HTTP {resp.status}"}
10    }
11    return resp.body
12 }

```

When a tool returns an error, the agent receives the error message and can decide how to proceed (retry, try a different tool, or report the error to the user).

12.7 Tool Grammar

```

1 tool_decl      = "tool" identifier "(" [ params ] ")" [ "->"
   type ]
2
3               "{" triple_string { statement } "}" ;
4
5 triple_string = '"""' { any_char } '"""' ;

```

Chapter 13

Agents

Agents are LLM-powered entities. They have a model, a system prompt, optional tools, and optional memory. Agents are the primary way to interact with language models in HAIRA.

13.1 Agent Declaration

```
1 agent AgentName {
2     model: provider_name
3     system: "System prompt describing the agent's behavior."
4     tools: [tool1, tool2]
5     temperature: 0.5
6     memory: conversation(max_turns: 50)
7 }
```

13.2 Agent Fields

Field	Type	Required	Description
model	identifier	Yes	Provider to use
system	string	Yes	System prompt
tools	[tool]	No	List of tools the agent can call
temperature	float	No	Sampling temperature (0.0–2.0)
max_tokens	int	No	Max tokens per response
max_steps	int	No	Max tool-calling iterations
memory	memory type	No	Memory configuration (default: none)

Table 13.1: Agent fields

13.2.1 Multiline System Prompts

For longer system prompts, use triple-quoted strings:

```
1 agent SupportBot {
2     model: anthropic
3     system: """
4         You are a customer support agent for Acme Corp.
```

```

5         Use the knowledge base to answer questions.
6         Look up orders when customers ask about purchases.
7         Create tickets for issues you cannot resolve.
8         Always be polite and helpful.
9     """
10    tools: [search_kb, lookup_order, create_ticket]
11    temperature: 0.3
12 }

```

13.3 Agent Methods

Agents have three methods for interacting with the LLM:

13.3.1 .ask() — Text In, Text Out

The simplest interaction. Send a text prompt, get a text response:

```

1 answer, err = Researcher.ask("What is quantum computing?")
2 if err != nil {
3     io.println("Error: " + err.message)
4     return
5 }
6 io.println(answer)

```

Signature: `.ask(prompt: string, session: string?) -> (string, Error?)`

13.3.2 .run() — Structured In, Structured Out

For typed interactions. Send a struct, get a typed response. The output type is inferred from the left-side type annotation:

```

1 struct ResearchRequest {
2     topic: string
3     depth: string = "detailed"
4 }
5
6 struct ResearchResult {
7     summary: string
8     sources: [string]
9     confidence: float
10 }
11
12 research: ResearchResult, err = Researcher.run(ResearchRequest{
13     topic: "quantum computing"
14 })

```

The compiler generates JSON schemas for both the input and output types. The agent is instructed to return structured data matching the output schema.

Signature: `.run(input: T, session: string?) -> (U, Error?)`

Where U is inferred from the left-side type annotation.

13.3.3 .stream() — Text In, Streaming Out

For real-time token-by-token responses. Returns an iterable stream:

```

1 for chunk in Assistant.stream("Tell me a story") {
2     io.print(chunk)
3 }
4 io.println() // newline after stream completes

```

Signature: `.stream(prompt: string, session: string?) -> stream<string>`

The `stream<string>` type is iterable with `for`. Each iteration yields the next token (or chunk) as the LLM generates it.

13.4 Memory

By default, agents are stateless — each call is independent. Adding memory enables agents to remember conversation history across calls.

13.4.1 No Memory (Default)

```

1 agent Classifier {
2     model: local
3     system: "Classify inputs into categories."
4 }
5
6 // Each call is independent
7 cat1 = Classifier.ask("This is a bug report") // "technical"
8 cat2 = Classifier.ask("What did I just send you?" // has no
    context

```

13.4.2 Conversation Memory

Keeps the last N turns of chat history:

```

1 agent Assistant {
2     model: anthropic
3     system: "You are a helpful assistant."
4     memory: conversation(max_turns: 50)
5 }

```

13.4.3 Summary Memory

Summarizes older messages to stay within a token budget. Useful for long-running conversations:

```

1 agent ProjectHelper {
2     model: anthropic
3     system: "You help with long-running software projects."
4     memory: summary(max_tokens: 2000)
5 }

```

Type	Parameter	Behavior
<code>none</code>	—	Stateless (default)
<code>conversation(max_turns: N)</code>	<code>N: int</code>	Keeps last N message pairs
<code>summary(max_tokens: N)</code>	<code>N: int</code>	Summarizes to fit within N tokens

Table 13.2: Built-in memory types

13.4.4 Memory Types

13.5 Sessions

When an agent has memory, calls are scoped to a **session**. A session groups related interactions — for example, one user’s conversation.

```

1 // With session --- memory persists within same session
2 answer1, _ = Assistant.ask("My name is Alice", session:
3   "user-123")
4 answer2, _ = Assistant.ask("What is my name?", session:
5   "user-123")
6 // answer2 = "Your name is Alice"
7
8 // Different session --- separate memory
9 answer3, _ = Assistant.ask("What is my name?", session:
10  "user-456")
11 // answer3 = "I don't know your name yet"

```

The `session:` parameter is a named argument available on all agent methods:

```

1 // .ask() with session
2 reply, err = Agent.ask("Hello", session: session_id)
3
4 // .run() with session
5 result: T, err = Agent.run(input, session: session_id)
6
7 // .stream() with session
8 for chunk in Agent.stream("Hello", session: session_id) {
9   io.print(chunk)
10 }

```

Without a `session:` parameter, each call starts a fresh conversation — even if the agent has memory configured.

13.6 Tool-Calling Loop

When an agent has tools, the runtime executes a loop:

1. Send the prompt (and conversation history) to the LLM
2. If the LLM requests a tool call, execute the tool
3. Send the tool result back to the LLM
4. Repeat until the LLM produces a final response (or `max_steps` is reached)

This is transparent to the caller:


```

1 // The agent may call search() multiple times internally
2 // before producing a final answer
3 answer, err = Researcher.ask("Find recent papers on quantum
   error correction")

```

The `max_steps` field limits how many tool-calling iterations are allowed:

```

1 agent Researcher {
2   model: anthropic
3   system: "You are a thorough researcher."
4   tools: [search, read_url]
5   max_steps: 10 // at most 10 tool calls per request
6   temperature: 0.2
7 }

```

13.7 Error Handling

Agent calls can fail. Always check for errors:

```

1 answer, err = Researcher.ask("Complex query here")
2 if err != nil {
3   io.println("Agent error: " + err.message)
4   // Handle: retry, fallback, or report
5 }

```

Common error cases:

- LLM provider unreachable or returns an error
- Max steps exceeded (tool-calling loop did not converge)
- Structured output did not match expected schema
- Timeout exceeded

13.8 Examples

13.8.1 Stateless Classifier

```

1 agent Classifier {
2   model: local
3   system: "Classify the input as: positive, negative, or
   neutral. Reply with just the label."
4   temperature: 0.0
5 }
6
7 fn main() {
8   label, _ = Classifier.ask("I love this product!")
9   io.println(label) // "positive"
10 }

```

13.8.2 Research Agent with Tools

```

1  agent Researcher {
2      model: anthropic
3      system: "You are a thorough researcher. Always cite your
4              sources."
5      tools: [search, read_url]
6      temperature: 0.2
7      max_steps: 15
8  }
9
10 fn main() {
11     answer, err = Researcher.ask("What are the latest advances
12                                 in fusion energy?")
13     if err != nil {
14         io.println("Error: " + err.message)
15         return
16     }
17     io.println(answer)
18 }

```

13.8.3 Conversational Assistant

```

1  agent Tutor {
2      model: anthropic
3      system: "You are a patient math tutor. Explain concepts
4              step by step."
5      memory: conversation(max_turns: 30)
6      temperature: 0.5
7  }
8
9  fn main() {
10     session = "student-001"
11
12     reply1, _ = Tutor.ask("What is a derivative?", session:
13                           session)
14     io.println(reply1)
15
16     reply2, _ = Tutor.ask("Can you give me an example?",
17                           session: session)
18     io.println(reply2)
19
20     reply3, _ = Tutor.ask("Now explain the chain rule",
21                           session: session)
22     io.println(reply3)
23     // Tutor remembers the full conversation context
24 }

```

13.9 Agent Grammar

```

1  agent_decl    = "agent" identifier "{" { agent_field } "}" ;
2

```

```
3 agent_field    = identifier ":" agent_value ;
4
5 agent_value    = expression
6                 | string_lit
7                 | triple_string
8                 | "[" ident_list "]"
9                 | memory_value ;
10
11 memory_value   = "conversation" "(" [ arguments ] ")"
12                 | "summary" "(" [ arguments ] ")"
13                 | "none" ;
```


Chapter 14

Workflows

Workflows are the central abstraction in HAIRA. A workflow is a composable function with a trigger, typed parameters, and a typed return value. Workflows replace both n8n-style visual automations and LangGraph-style agent graphs.

14.1 Workflow Declaration

A workflow is a function preceded by a trigger decorator:

```
1 @trigger_type("config")
2 workflow WorkflowName(param: Type, param: Type) -> ReturnType {
3     // body
4 }
```

Workflows without a trigger decorator can be called as sub-workflows but cannot be triggered externally.

14.2 Triggers

Triggers define how a workflow is invoked. They use the @ decorator syntax.

14.2.1 @webhook — HTTP Endpoint

```
1 @webhook("/summarize")
2 workflow Summarizer(url: string) -> Summary {
3     content = fetch_page(url)
4     summary = Researcher.ask("Summarize: {content}")
5     return Summary{ text: summary, word_count: summary.len() }
6 }
```

The workflow parameters become the JSON request body. The return type becomes the JSON response body.

```
$ curl -X POST http://localhost:8080/summarize \
    -d '{"url": "https://example.com/article"}'

{"text": "...", "word_count": 150}
```

Options:

```

1 @webhook("/path") // POST (default)
2 @webhook("/path", method: "GET") // GET
3 @webhook("/path", method: "PUT") // PUT

```

14.2.2 @websocket — Bidirectional Streaming

```

1 @websocket("/chat")
2 workflow Chat(message: string, session_id: string) ->
3   stream<string> {
4     return Assistant.stream(message, session: session_id)
5   }

```

WebSocket workflows receive messages and can return `stream<string>` for real-time streaming responses.

14.2.3 @cron — Scheduled Execution

```

1 @cron("0 9 * * *") // every day at 9 AM
2 workflow DailyReport() -> Report {
3   data = fetch_metrics()
4   report: Report, _ = Analyst.run(AnalyzeRequest{ data: data
5     })
6   send_email("team@company.com", "Daily Report",
7     report.summary)
8   return report
9 }

```

Cron expressions follow standard cron syntax (minute, hour, day-of-month, month, day-of-week).

14.2.4 @event — Event-Driven

```

1 @event("order.created")
2 workflow ProcessOrder(order: Order) -> OrderResult {
3   validated = validate_order(order)
4   if validated.has_errors {
5     return OrderResult{ status: "rejected" }
6   }
7   charge_payment(order)
8   return OrderResult{ status: "confirmed" }
9 }

```

Events are published using `haira.emit()`:

```

1 haira.emit("order.created", Order{ id: "123", total: 99.99 })

```

14.2.5 @manual — CLI Only

```

1 @manual

```

```

2 workflow Migrate(version: string) -> { status: string } {
3     // only invoked from CLI or main()
4     run_migrations(version)
5     return { status: "done" }
6 }

```

```
$ haira run Migrate --input '{"version": "v2"}'
```

14.2.6 No Trigger (Sub-Workflow)

Workflows without a trigger can only be called from other workflows or from `main()`:

```

1 workflow EnrichLead(email: string) -> EnrichedLead {
2     company = lookup_company(email)
3     social = lookup_social(email)
4     return EnrichedLead{ email: email, company: company,
5         social: social }
5 }

```

14.3 Sequential Steps

The workflow body is sequential by default. Each line can use the results of previous lines:

```

1 @manual
2 workflow ArticlePipeline(topic: string) -> Article {
3     // Step 1: Research
4     research: ResearchResult, err =
5         Researcher.run(ResearchRequest{ topic: topic })
6     if err != nil { return Article{}, err }
7
8     // Step 2: Write draft
9     draft: Article, err = Writer.run(WriteRequest{
10         content: research.summary,
11         sources: research.sources
12     })
13     if err != nil { return Article{}, err }
14     return draft
15 }

```

14.4 Parallel Execution

Use `spawn` blocks to run steps in parallel:

```

1 @manual
2 workflow ReviewArticle(article: Article) -> [Review] {
3     reviews = spawn {
4         Reviewer.run(ReviewRequest{ article: article, focus:
5             "accuracy" })
6         Reviewer.run(ReviewRequest{ article: article, focus:
7             "clarity" })
8     }
9 }

```

```

6         Reviewer.run(ReviewRequest{ article: article, focus:
           "engagement" })
7     }
8     return reviews
9 }

```

A **spawn** block runs all statements concurrently and returns a list of results when all complete. This is equivalent to `Promise.all()` in JavaScript.

14.5 Branching

Standard **if/else** and **match** work inside workflows:

```

1 @event("order.created")
2 workflow ProcessOrder(order: Order) -> OrderResult {
3     validated = validate_order(order)
4
5     if validated.has_errors {
6         send_slack("#alerts", "Order {order.id} failed")
7         return OrderResult{ status: "rejected" }
8     }
9
10    stock = check_stock(validated.items)
11
12    if stock.available {
13        charge_payment(order)
14        send_email(order.customer_email, "Order confirmed!")
15        return OrderResult{ status: "confirmed" }
16    } else {
17        send_email(order.customer_email, "Items out of stock")
18        return OrderResult{ status: "backordered" }
19    }
20 }

```

14.6 Loops

```

1 @manual
2 workflow ProcessBatch(urls: [string]) -> [string] {
3     results = for url in urls {
4         content = fetch_page(url)
5         Researcher.ask("Summarize: {content}")
6     }
7     return results
8 }

```

Loops inside workflows are sequential by default. For parallel iteration, use **spawn**:

```

1 @manual
2 workflow ProcessBatchParallel(urls: [string]) -> [string] {
3     results = spawn {
4         for url in urls {
5             fetch_page(url) | Researcher.ask("Summarize: {_}")
6         }
7     }
8 }

```



```

7     }
8     return results
9 }

```

14.7 Sub-Workflows

Workflows can call other workflows like functions:

```

1 workflow ScoreLead(company: Company, social: SocialProfile) ->
  ScoreResult {
2     score: ScoreResult, err = Analyst.run(ScoreRequest{
3         company: company,
4         social: social
5     })
6     return score
7 }
8
9 workflow EnrichLead(email: string) -> EnrichedLead {
10     company = lookup_company(email)
11     social = lookup_social(email)
12     score = ScoreLead(company, social)
13     return EnrichedLead{ email: email, company: company, score:
14         score.value }
15 }
16 @webhook("/new-lead")
17 workflow SalesAutomation(email: string) -> { result: string } {
18     lead = EnrichLead(email)
19
20     if lead.score > 0.8 {
21         send_slack("#sales", "Hot lead: {email}")
22     }
23
24     return { result: "processed" }
25 }

```

Sub-workflows compose naturally. The compiler validates type compatibility at each call site.

14.8 Error Handling

Workflows use standard HAIRA error handling:

```

1 @manual
2 workflow SafeProcess(data: string) -> { result: string } {
3     result, err = risky_operation(data)
4     if err != nil {
5         send_slack("#errors", "Failed: {err.message}")
6         result = fallback_operation(data)
7     }
8     return { result: result }
9 }

```

14.9 Running Workflows

14.9.1 From main()

```

1 fn main() {
2     result, err = ArticlePipeline("AI agents")
3     if err != nil {
4         io.println("Failed: " + err.message)
5         return
6     }
7     io.println(result.title)
8 }

```

14.9.2 Server Mode

Start a server that listens for all triggered workflows:

```

1 fn main() {
2     server = haira.serve([
3         Summarizer,           // POST /summarize
4         Chat,                 // WebSocket /chat
5         ProcessOrder,        // on order.created events
6         DailyReport,         // cron 0 9 * * *
7     ])
8
9     io.println("Haira server running on :8080")
10    server.listen(8080)
11 }

```

14.9.3 CLI

```

# Run a workflow directly
$ haira run ArticlePipeline --input '{"topic": "AI agents"}'

# Start server mode
$ haira serve main.haira --port 8080

# Run with environment
$ ANTHROPIC_API_KEY=sk-... haira serve main.haira

```

14.10 Workflow Grammar

```

1 workflow_decl = { decorator } "workflow" identifier
2               "(" [ params ] ")" [ "->" type ] block ;
3
4 decorator     = "@" identifier [ "(" [ arguments ] ")" ] ;
5
6 spawn_block   = "spawn" "{" { statement } "}" ;

```

Chapter 15

Chatbot Patterns

One of the most common use cases for HAIRA is building chatbot backends. This chapter shows patterns from simple to advanced.

15.1 Simple REST Chatbot

A chatbot accessible via HTTP POST:

```
1 import "io"
2
3 provider anthropic {
4     api_key: env("ANTHROPIC_API_KEY")
5     model: "claude-sonnet-4-20250514"
6 }
7
8 agent Assistant {
9     model: anthropic
10    system: "You are a helpful assistant."
11    memory: conversation(max_turns: 50)
12    temperature: 0.7
13 }
14
15 @webhook("/chat")
16 workflow Chat(message: string, session_id: string) -> { reply:
17     string } {
18     reply, err = Assistant.ask(message, session: session_id)
19     if err != nil {
20         return { reply: "Sorry, something went wrong." }
21     }
22     return { reply: reply }
23 }
24
25 fn main() {
26     server = haira.serve([Chat])
27     io.println("Chat server on :8080")
28     server.listen(8080)
29 }
```

Usage:

```
$ curl -X POST http://localhost:8080/chat \
  -d '{"message": "Hello!", "session_id": "user-123"}'
```

```

{"reply": "Hi there! How can I help you today?"}

$ curl -X POST http://localhost:8080/chat \
  -d '{"message": "What did I just say?", "session_id":
    "user-123"}'

{"reply": "You said \"Hello!\""}

```

The agent remembers the conversation because both calls share the same `session_id`.

15.2 Streaming WebSocket Chatbot

For real-time token-by-token responses over WebSocket:

```

1 @websocket("/chat")
2 workflow StreamingChat(message: string, session_id: string) ->
3   stream<string> {
4     return Assistant.stream(message, session: session_id)
5   }
6
7 fn main() {
8   server = haira.serve([StreamingChat])
9   server.listen(8080)
10 }

```

The `stream<string>` return type tells HAIRA to stream each token over the WebSocket as it arrives. No buffering — the client sees tokens in real time.

15.3 Dual-Mode Chatbot

Expose the same agent over both REST (for simple integrations) and WebSocket (for real-time UIs):

```

1 agent Assistant {
2   model: anthropic
3   system: "You are a helpful assistant."
4   memory: conversation(max_turns: 50)
5 }
6
7 @webhook("/chat")
8 workflow ChatREST(message: string, session_id: string) -> {
9   reply: string } {
10    reply, err = Assistant.ask(message, session: session_id)
11    return { reply: reply or "Error occurred." }
12 }
13
14 @websocket("/chat/stream")
15 workflow ChatStream(message: string, session_id: string) ->
16   stream<string> {
17     return Assistant.stream(message, session: session_id)
18   }
19
20 fn main() {
21   server = haira.serve([ChatREST, ChatStream])
22   server.listen(8080)
23 }

```

```
21 }
```

Both endpoints share the same agent and session memory.

15.4 RAG Chatbot

A chatbot that searches a knowledge base before answering:

```
1 import "tools/postgres"
2 import "tools/http"
3
4 tool search_kb(query: string) -> [KBArticle] {
5     """Search the knowledge base for relevant articles"""
6
7     results, err = pg.query(
8         "SELECT * FROM articles WHERE content @@ to_tsquery($1)
9         LIMIT 5",
10        [query]
11    )
12    if err != nil { return [], err }
13    return results
14 }
15
16 tool lookup_order(order_id: string) -> Order {
17     """Look up a customer order by ID"""
18
19     order, err = pg.query_one(
20         "SELECT * FROM orders WHERE id = $1",
21        [order_id]
22    )
23    if err != nil { return Order{}, err }
24    return order
25 }
26
27 tool create_ticket(subject: string, body: string, priority:
28     string = "normal") -> Ticket {
29     """Create a support ticket"""
30
31     resp, err = http.post("https://helpdesk.api/tickets", {
32         subject: subject, body: body, priority: priority
33     })
34     if err != nil { return Ticket{}, err }
35     return json.decode(resp.body, Ticket)
36 }
37
38 agent SupportBot {
39     model: anthropic
40     system: """
41         You are a customer support agent for Acme Corp.
42         Use the knowledge base to answer questions.
43         Look up orders when customers ask about purchases.
44         Create tickets for issues you cannot resolve directly.
45         Always be polite and helpful.
46     """
47     tools: [search_kb, lookup_order, create_ticket]
48     memory: conversation(max_turns: 100)
49     temperature: 0.3
```

```

48 }
49
50 @websocket("/support")
51 workflow CustomerSupport(message: string, session_id: string)
52   -> stream<string> {
53     return SupportBot.stream(message, session: session_id)
54   }

```

The agent automatically decides when to search the knowledge base, look up an order, or create a ticket based on the user's message.

15.5 Multi-Agent Routing

Route user messages to specialized agents based on intent:

```

1  agent Router {
2    model: anthropic
3    system: ""
4      Classify the user message into one category: billing,
5      technical, general.
6      Reply with just the category name, nothing else.
7    ""
8    temperature: 0.0
9  }
10
11 agent BillingAgent {
12   model: anthropic
13   system: "You handle billing questions. You can look up
14     invoices and process refunds."
15   tools: [lookup_invoice, process_refund]
16   memory: conversation(max_turns: 30)
17 }
18
19 agent TechAgent {
20   model: anthropic
21   system: "You handle technical support. You can search docs
22     and check system status."
23   tools: [search_docs, check_status]
24   memory: conversation(max_turns: 30)
25 }
26
27 agent GeneralAgent {
28   model: anthropic
29   system: "You handle general inquiries."
30   memory: conversation(max_turns: 30)
31 }
32
33 @websocket("/chat")
34 workflow SmartChat(message: string, session_id: string) ->
35   stream<string> {
36     category, _ = Router.ask(message)
37
38     match category {
39       "billing" => return BillingAgent.stream(message,
40         session: session_id)
41       "technical" => return TechAgent.stream(message,
42         session: session_id)

```

```

37         - => return GeneralAgent.stream(message,
38             session: session_id)
39     }

```

15.6 Chatbot with Structured Actions

Sometimes you want the chatbot to return structured data alongside text:

```

1  struct ChatResponse {
2      reply: string
3      actions: [Action]
4  }
5
6  struct Action {
7      type: string          // "navigate", "show_modal", "refresh"
8      target: string
9  }
10
11 agent UIAssistant {
12     model: anthropic
13     system: ""
14     You are a UI assistant. When answering, also suggest UI
15     actions.
16     For example, if the user asks about their order,
17     suggest navigating
18     to the orders page.
19     ""
20     tools: [search_kb, lookup_order]
21     memory: conversation(max_turns: 50)
22 }
23
24 @webhook("/assistant")
25 workflow UIChat(message: string, session_id: string) ->
26     ChatResponse {
27         response: ChatResponse, err = UIAssistant.run(
28             { message: message },
29             session: session_id
30         )
31         if err != nil {
32             return ChatResponse{ reply: "Sorry, an error
33                 occurred.", actions: [] }
34         }
35         return response
36     }

```

15.7 Best Practices

1. **Always use sessions for conversation** — without a session, memory agents start fresh every call
2. **Set appropriate max_turns** — too high wastes tokens, too low loses context
3. **Use streaming for interactive UIs** — REST is fine for backend-to-backend, but

users expect real-time responses

4. **Give tools clear descriptions** — the agent decides when to use tools based on descriptions
5. **Use a Router agent for complex bots** — instead of one agent doing everything, route to specialists
6. **Handle errors gracefully** — always provide a fallback response when agents fail

Part IV

Implementation

Chapter 16

Complete Grammar

This chapter provides the complete formal grammar for HAIRA in Extended Backus-Naur Form (EBNF), including all core language constructs and agentic extensions (providers, tools, agents, and workflows).

16.1 Notation

Notation	Meaning
=	Definition
	Alternative
[...]	Optional (0 or 1)
{ ... }	Repetition (0 or more)
(...)	Grouping
"..."	Terminal string
;	End of production

Table 16.1: EBNF notation

16.2 Lexical Grammar

16.2.1 Characters

```
1 letter      = "a".."z" | "A".."Z" | "_" ;
2 digit       = "0".."9" ;
3 hex_digit   = digit | "a".."f" | "A".."F" ;
4 octal_digit = "0".."7" ;
5 binary_digit = "0" | "1" ;
6 any_char    = (* any Unicode character *) ;
```

16.2.2 Whitespace and Comments

```
1 whitespace = " " | "\t" | "\r" ;
2 newline    = "\n" ;
3 line_comment = "//" { any_char } newline ;
4 block_comment = "/*" { any_char | block_comment } "*/" ;
```

```
5 doc_comment    = "///" { any_char } newline ;
```

Note

Block comments nest: `/* outer /* inner */ still outer */` is valid. Doc comments (`///`) attach to the immediately following declaration and are preserved in compiled metadata.

16.2.3 Identifiers

```
1 identifier      = letter { letter | digit } ;
```

16.2.4 Keywords

```
1 keyword         = "agent" | "and" | "as" | "break" | "catch"
2                 | "chan" | "const" | "continue" | "defer"
3                 | "else" | "enum" | "false" | "fn" | "for"
4                 | "from" | "if" | "impl" | "import" | "in"
5                 | "match" | "nil" | "not" | "or" | "provider"
6                 | "return" | "select" | "spawn" | "struct" |
7                 | "true" | "try" | "type" | "unsafe" | "where"
8                 | "while" | "workflow" ;
```

Note

The keywords **agent**, **provider**, **tool**, and **workflow** are reserved for agentic declarations. The keyword **unsafe** is reserved for future use. All keywords are case-sensitive.

16.2.5 Literals

```
1 integer_lit     = decimal_lit | hex_lit | octal_lit | binary_lit ;
2 decimal_lit     = digit { digit | "_" } ;
3 hex_lit         = "0" ("x"|"X") hex_digit { hex_digit | "_" } ;
4 octal_lit       = "0" ("o"|"O") octal_digit { octal_digit | "_" }
5                 ;
6 binary_lit      = "0" ("b"|"B") binary_digit { binary_digit | "_"
7                 } ;
8 float_lit       = decimal_lit "." decimal_lit [ exponent ]
9                 | decimal_lit exponent ;
10 exponent        = ("e"|"E") ["+"|"-"] decimal_lit ;
11 string_lit      = ''' { string_char } '''
12                 | "r" ''' { raw_char } '''
13                 | triple_string ;
14 triple_string    = '"""' { any_char } '"""' ;
15 string_char      = any_char - ('"' | "\" | newline)
16                 | escape_seq ;
17 escape_seq       = "\" ( "n" | "r" | "t" | "\" | "'" | "0"
```

```

18         | "x" hex_digit hex_digit
19         | "u" "{" hex_digit { hex_digit } "}" ) ;
20 raw_char    = any_char - ',' ;
21
22 bool_lit    = "true" | "false" ;
23 nil_lit     = "nil" ;

```

Note

Triple-quoted strings ("\"...\"") preserve newlines and leading indentation. They are used for tool descriptions, agent system prompts, and multi-line string literals. The common leading whitespace is stripped at compile time.

16.2.6 Operators

```

1 operator    = arith_op | comp_op | logic_op | bitwise_op
2             | shift_op | assign_op | other_op ;
3
4 arith_op     = "+" | "-" | "*" | "/" | "%" ;
5 comp_op      = "==" | "!=" | "<" | ">" | "<=" | ">=" ;
6 logic_op     = "&&" | "||" | "!" ;
7 bitwise_op   = "&" | "|" | "^" | "~" ;
8 shift_op     = "<<" | ">>" | ">>>" ;
9 assign_op    = "=" | "+=" | "-=" | "*=" | "/="
10             | "&=" | "|=" | "^=" | "<<=" | ">>=" | ">>>=" ;
11 other_op     = "|>" | ".." | "..=" | "<-" | "->" | "=>" | "?"
               | "@" ;

```

Note

The `|>` operator is the pipe operator for function composition. The `@` symbol is used for decorator syntax on workflow declarations. The `<-` operator is used for channel send and receive operations.

16.2.7 Delimiters

```

1 delimiter    = "(" | ")" | "[" | "]" | "{" | "}"
2             | "," | ":" | "." | ";" | "#" ;

```

16.3 Syntactic Grammar

16.3.1 Program Structure

```

1 program      = { import_stmt } { top_level } ;
2
3 top_level    = fn_decl
4             | struct_decl
5             | enum_decl
6             | type_alias
7             | const_decl

```

```

8         | impl_block
9         | constraint_decl
10        | provider_decl
11        | tool_decl
12        | agent_decl
13        | workflow_decl ;

```

16.3.2 Imports

```

1 import_stmt  = "import" import_spec ;
2
3 import_spec  = string_lit
4               | identifier "from" string_lit
5               | "{" ident_list "}" "from" string_lit
6               | "*" "from" string_lit ;
7
8 ident_list   = identifier { "," identifier } ;

```

16.3.3 Core Declarations

```

1 fn_decl      = [ attributes ] "fn" [ receiver ] identifier
2               [ generic_params ] "(" [ params ] ")"
3               [ "->" type ] [ where_clause ] block ;
4
5 receiver     = "(" identifier ":" [ "*" ] type ")" ;
6
7 generic_params = "<" generic_param { "," generic_param } ">" ;
8
9 generic_param = identifier [ ":" constraint_list ] ;
10
11 constraint_list = identifier { "+" identifier } ;
12
13 where_clause  = "where" where_bound { "," where_bound } ;
14
15 where_bound   = identifier ":" constraint_list ;
16
17 params        = param { "," param } ;
18
19 param         = identifier ":" [ "..." ] type [ "=" expression
20               ] ;
21
22 struct_decl   = [ attributes ] "struct" identifier
23               [ generic_params ] "{ { field_decl } }" ;
24
25 field_decl    = [ attributes ] identifier ":" type ;
26
27 enum_decl     = [ attributes ] "enum" identifier
28               [ generic_params ] "{ enum_variants }" ;
29
30 enum_variants = enum_variant { "," enum_variant } [ "," ] ;
31
32 enum_variant  = identifier [ "(" type_list ")" ] ;

```

```

33 type_alias      = "type" identifier [ generic_params ] "=" type ;
34
35 const_decl      = "const" identifier [ ":" type ] "=" expression ;
36
37 impl_block      = "impl" [ generic_params ] identifier
38                  "for" type [ where_clause ]
39                  "{" { fn_decl } "}" ;
40
41 constraint_decl = "constraint" identifier [ generic_params ]
42                  "{" { fn_signature } "}" ;
43
44 fn_signature     = "fn" identifier "(" [ type_params ] ")" [ "->"
45                  type ] ;
46
47 type_params      = type { "," type } ;

```

16.3.4 Attributes

Attributes use the `#[...]` syntax and can be applied to functions, structs, enums, and fields. They are distinct from the `@` decorator syntax used for workflow triggers.

```

1 attributes      = { attribute } ;
2
3 attribute       = "#[" identifier [ "(" attr_args ")" ] "]" ;
4
5 attr_args       = attr_arg { "," attr_arg } ;
6
7 attr_arg        = identifier [ "=" literal ] ;

```

16.3.5 Decorators

Decorators use the `@` syntax and are used exclusively for workflow trigger declarations.

```

1 decorator       = "@" identifier [ "(" [ arguments ] ")" ] ;

```

16.3.6 Provider Declarations

Providers configure LLM backends. They are declared at the top level and referenced by name in agent declarations.

```

1 provider_decl   = "provider" identifier "{" { provider_field }
2                  "}" ;
3
4 provider_field  = identifier ":" expression ;

```

16.3.7 Tool Declarations

Tools are typed functions with a mandatory triple-quoted description. The compiler auto-generates JSON schemas from the type signature.

```

1 tool_decl       = "tool" identifier "(" [ params ] ")" [ "->"
2                  type ]

```

```

2         "{" triple_string { statement } "}" ;
3
4 triple_string = '"""' { any_char } '"""' ;

```

Note

The `triple_string` must be the first element of the tool body. The compiler emits an error if a tool is missing its description.

16.3.8 Agent Declarations

Agents are LLM-powered entities with a model, system prompt, optional tools, and optional memory.

```

1 agent_decl    = "agent" identifier "{" { agent_field } "}" ;
2
3 agent_field   = identifier ":" agent_value ;
4
5 agent_value   = expression
6               | string_lit
7               | triple_string
8               | "[" ident_list "]"
9               | memory_value ;
10
11 memory_value  = "conversation" "(" [ arguments ] ")"
12               | "summary" "(" [ arguments ] ")"
13               | "none" ;

```

Note

Agent fields include `model` (required), `system` (required), and optional fields such as `tools`, `temperature`, `max_tokens`, `max_steps`, and `memory`. The `model` field references a provider by name.

16.3.9 Workflow Declarations

Workflows are composable functions with optional trigger decorators. A workflow without a decorator can only be called as a sub-workflow.

```

1 workflow_decl = { decorator } "workflow" identifier
2               "(" [ params ] ")" [ "->" type ] block ;

```

16.3.10 Types

```

1 type          = simple_type
2               | array_type
3               | map_type
4               | tuple_type
5               | option_type
6               | fn_type
7               | generic_type
8               | stream_type

```



```

 9      | pointer_type
10      | union_type
11      | channel_type ;
12
13 simple_type = "int" | "i8" | "i16" | "i32" | "i64"
14             | "u8" | "u16" | "u32" | "u64"
15             | "float" | "f32" | "f64"
16             | "bool" | "string"
17             | identifier ;
18
19 array_type  = "[" "]" type ;
20
21 map_type    = "[" type ":" type "]" ;
22
23 tuple_type  = "(" type { "," type } ")" ;
24
25 option_type = type "?" ;
26
27 fn_type     = "fn" "(" [ type_list ] ")" [ "->" type ] ;
28
29 type_list   = type { "," type } ;
30
31 generic_type = identifier "<" type_list ">" ;
32
33 stream_type  = "stream" "<" type ">" ;
34
35 pointer_type = "*" type ;
36
37 union_type   = type { "|" type } ;
38
39 channel_type = "chan" "<" type ">"
40             | "chan<-" type
41             | "<-chan" "<" type ">" ;

```

Note

The `stream<T>` type represents a lazy, asynchronous, single-use sequence of values. It is distinct from `chan<T>`, which is a bidirectional communication channel between concurrent tasks. Streams are typically produced by agent `.stream()` calls and consumed with `for` loops.

16.3.11 Statements

```

1 statement = var_decl
2           | assignment
3           | expr_stmt
4           | if_stmt
5           | for_stmt
6           | while_stmt
7           | match_stmt
8           | return_stmt
9           | break_stmt
10          | continue_stmt
11          | defer_stmt
12          | spawn_stmt

```

```

13         | send_stmt
14         | select_stmt
15         | block ;
16
17 var_decl    = ident_list [ ":" type ] "=" expr_list ;
18
19 ident_list  = identifier { "," identifier } ;
20
21 expr_list   = expression { "," expression } ;
22
23 assignment  = lvalue_list assign_op expr_list ;
24
25 lvalue_list = lvalue { "," lvalue } ;
26
27 lvalue      = identifier
28             | lvalue "." identifier
29             | lvalue "[" expression "]" ;
30
31 expr_stmt   = expression ;
32
33 if_stmt     = "if" [ simple_stmt ";" ] expression block
34             [ "else" ( if_stmt | block ) ] ;
35
36 simple_stmt = var_decl | assignment | expr_stmt ;
37
38 for_stmt    = "for" [ identifier "," ] identifier "in"
39             expression block ;
40
41 while_stmt  = "while" expression block ;
42
43 match_stmt  = "match" expression "{" { match_arm } "}" ;
44
45 match_arm   = pattern [ "if" expression ] "=>" ( expression |
46             block ) ;
47
48 return_stmt = "return" [ expr_list ] ;
49
50 break_stmt  = "break" [ identifier ] ;
51
52 continue_stmt = "continue" [ identifier ] ;
53
54 defer_stmt  = "defer" statement ;
55
56 spawn_stmt  = "spawn" ( block | fn_call ) ;
57
58 send_stmt   = expression "<-" expression ;
59
60 select_stmt = "select" "{" { select_case } [ default_case ]
61             "}" ;
62
63 select_case = pattern "=" "<-" expression "=>" ( expression |
64             block )
65             | expression "<-" expression "=>" ( expression |
66             block ) ;
67
68 default_case = "default" "=>" ( expression | block ) ;
69
70 block       = "{" { statement } "}" ;

```

```

66
67 fn_call      = identifier "(" [ arguments ] ")"
68              | identifier "." identifier "(" [ arguments ] ")"
              ;

```

Note

Inside a workflow, a **spawn** block runs all its statements concurrently and returns a list of results when all complete. This is equivalent to `Promise.all()` in JavaScript. Outside a workflow, **spawn** launches a concurrent goroutine-style task.

16.3.12 Expressions

```

1 expression    = assign_expr ;
2
3 assign_expr    = or_expr [ ("=" | "+=" | "-=" | "*=" | "/="
4                        | "&=" | "|=" | "^=" | "<=" | ">=" | ">>=")
5                  assign_expr ] ;
6
7 or_expr        = and_expr { ("or" | "||") and_expr } ;
8
9 and_expr       = equality { ("and" | "&&") equality } ;
10
11 equality       = comparison { ("==" | "!=") comparison } ;
12
13 comparison     = pipe { ("<" | ">" | "<=" | ">=") pipe } ;
14
15 pipe          = range { "|>" range } ;
16
17 range          = bitwise_or [ (".." | "..=") bitwise_or ] ;
18
19 bitwise_or     = bitwise_xor { "|" bitwise_xor } ;
20
21 bitwise_xor    = bitwise_and { "^" bitwise_and } ;
22
23 bitwise_and    = shift { "&" shift } ;
24
25 shift          = additive { ("<<" | ">>" | ">>>") additive } ;
26
27 additive       = multiplicative { ("+" | "-") multiplicative } ;
28
29 multiplicative = unary { ("*" | "/" | "%") unary } ;
30
31 unary          = ("!" | "-" | "~" | "not") unary
32                | postfix ;
33
34 postfix        = primary { postfix_op } ;
35
36 postfix_op     = call
37                | index
38                | field
39                | optional_chain ;
40
41 call           = "(" [ arguments ] ")" ;
42

```

```

43 arguments      = argument { "," argument } ;
44
45 argument        = [ identifier ":" ] expression ;
46
47 index           = "[" expression "]" ;
48
49 field           = "." identifier ;
50
51 optional_chain  = "?" "." identifier ;
52
53 primary         = identifier
54                 | literal
55                 | "(" expression ")"
56                 | array_lit
57                 | map_lit
58                 | struct_lit
59                 | closure
60                 | if_expr
61                 | match_expr
62                 | receive_expr
63                 | block ;
64
65 literal         = integer_lit | float_lit | string_lit
66                 | bool_lit | nil_lit ;
67
68 array_lit       = "[" [ expr_list ] "]" ;
69
70 map_lit         = "[" ":" "]"
71                 | "[" map_entry { "," map_entry } [ "," ] "]" ;
72
73 map_entry       = expression ":" expression ;
74
75 struct_lit      = identifier "{" [ field_inits ] "}" ;
76
77 field_inits     = field_init { "," field_init } [ "," ] ;
78
79 field_init      = identifier [ ":" expression ] ;
80
81 closure         = "fn" "(" [ closure_params ] ")" [ "->" type ]
82                 ( block | "{" expression "}" ) ;
83
84 closure_params  = closure_param { "," closure_param } ;
85
86 closure_param   = identifier [ ":" type ] ;
87
88 if_expr         = "if" expression block "else" ( if_expr | block
89                 ) ;
90
91 match_expr      = "match" expression "{" { match_arm } "}" ;
92
93 receive_expr    = "<-" expression ;

```

16.3.13 Patterns

```

1 pattern        = literal_pat
2                 | ident_pat

```

```

3      | wildcard_pat
4      | tuple_pat
5      | struct_pat
6      | enum_pat
7      | array_pat
8      | range_pat
9      | or_pat ;
10
11 literal_pat  = integer_lit | float_lit | string_lit | bool_lit
12             ;
13 ident_pat    = identifier ;
14
15 wildcard_pat = "_" ;
16
17 tuple_pat    = "(" pattern { "," pattern } ")" ;
18
19 struct_pat    = identifier "{" [ field_pats ] "}" ;
20
21 field_pats    = field_pat { "," field_pat } ;
22
23 field_pat     = identifier [ ":" pattern ] ;
24
25 enum_pat      = identifier "." identifier [ "(" [ pattern_list
26               ] ")" ] ;
27
28 pattern_list  = pattern { "," pattern } ;
29
30 array_pat     = "[" [ pattern_list ] [ "," "..." [ identifier ]
31               ] "]" ;
32
33 range_pat     = literal ".." [ "=" ] literal ;
34
35 or_pat        = pattern { "|" pattern } ;

```

16.4 Grammar Summary

16.4.1 Entry Points

- **program** — Complete source file
- **expression** — Single expression (REPL)
- **statement** — Single statement
- **type** — Type annotation

16.4.2 Precedence (High to Low)

1. Postfix: `() [] . ?`.
2. Unary: `! - ~ not`
3. Multiplicative: `* / %`
4. Additive: `+ -`

5. Shift: `<< >> >>>`
6. Bitwise AND: `&`
7. Bitwise XOR: `^`
8. Bitwise OR: `|`
9. Range: `... ..=`
10. Pipe: `|>`
11. Comparison: `< > <= >=`
12. Equality: `== !=`
13. Logical AND: `and &&`
14. Logical OR: `or ||`
15. Assignment: `= += -= *= /= &= |= ^= <= >= >>=`

16.4.3 Associativity

- Left-associative: All binary operators except assignment
- Right-associative: Assignment operators, unary operators
- Non-associative: Comparison operators, range operators

Chapter 17

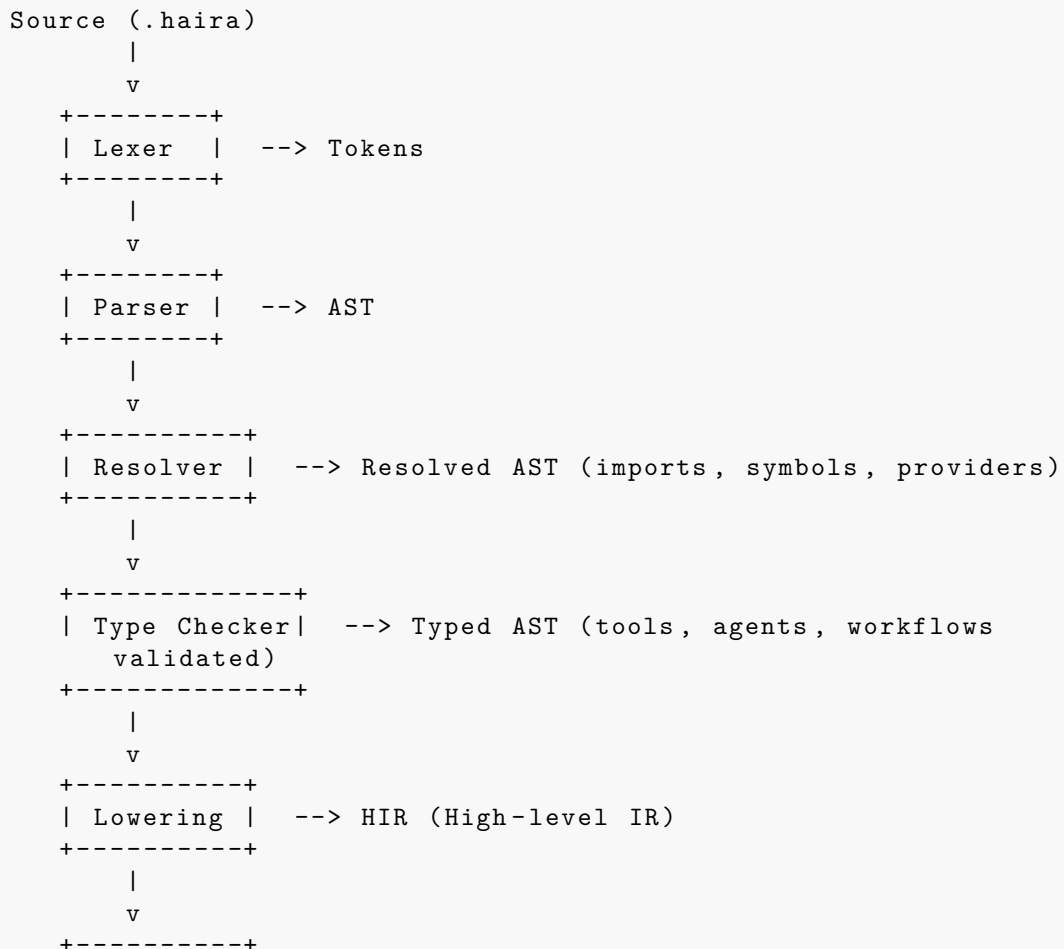
Compiler Architecture

This chapter describes the architecture and implementation of the HAIRA compiler, updated to reflect HAIRA’s identity as an agentic orchestration language.

17.1 Overview

The HAIRA compiler (**hairac**) transforms source code into native executables. It is written in Rust and uses Cranelift for code generation. Agentic constructs — **provider**, **tool**, **agent**, and **workflow** with **@** decorator triggers — are first-class elements of the compilation pipeline.

The compiler pipeline consists of the following stages:



```
| Codegen | --> Cranelift IR --> Native binary
+-----+
```

Note

There is no “AI Phase” in the pipeline. Previous versions of HAIRA included a compile-time AI code generation stage using the `ai` keyword and a Canonical Intermediate Representation (CIR). These have been removed. HAIRA is now a language for *orchestrating* AI agents at runtime, not for generating code with AI at compile time.

17.2 Lexer

The lexer (scanner) converts source text into a stream of tokens.

17.2.1 Token Types

```
1 enum TokenKind {
2     // Literals
3     IntLit(i64),
4     FloatLit(f64),
5     StringLit(String),
6     TripleQuoteStringLit(String),    // """..."""
7     BoolLit(bool),
8
9     // Identifiers and keywords
10    Ident(String),
11    Keyword(Keyword),
12
13    // Operators
14    Plus, Minus, Star, Slash, Percent,
15    Eq, EqEq, NotEq, Lt, Gt, LtEq, GtEq,
16    And, Or, Not, AndAnd, OrOr, Bang,
17    PlusEq, MinusEq, StarEq, SlashEq,
18    Pipe, DotDot, DotDotEq, Arrow, FatArrow,
19    LeftArrow, Question,
20    At,                                // @ (decorator prefix)
21
22    // Delimiters
23    LParen, RParen, LBracket, RBracket, LBrace, RBrace,
24    Comma, Colon, Dot, Semicolon,
25
26    // Special
27    Newline, Eof, Error(String),
28 }
```

17.2.2 Keywords

The keyword enum includes the four agentic keywords:

```
1 enum Keyword {
2     // General-purpose keywords
3     And, As, Break, Catch, Chan, Continue, Defer,
```



```

4      Else, Enum, False, Fn, For, From, If, Import,
5      In, Match, Nil, Not, Or, Return, Select, Spawn,
6      Struct, True, Try, Type, While,
7
8      // Agentic keywords
9      Agent,
10     Provider,
11     Tool,
12     Workflow,
13 }

```

Note

The `ai` keyword from previous versions is no longer part of the language. It is not reserved and may be used as an identifier.

17.2.3 Lexer Implementation

The lexer is a hand-written scanner that operates on UTF-8 source text:

```

1 struct Lexer<'a> {
2     source: &'a str,
3     chars: Peekable<CharIndices<'a>>,
4     line: usize,
5     column: usize,
6 }
7
8 impl<'a> Lexer<'a> {
9     fn next_token(&mut self) -> Token {
10         self.skip_whitespace();
11
12         match self.peek_char() {
13             None => Token::new(TokenKind::Eof, self.span()),
14             Some(c) => match c {
15                 '0'..'9' => self.scan_number(),
16                 '"' => self.scan_string_or_triple_quote(),
17                 'a'..'z' | 'A'..'Z' | '_' =>
18                     self.scan_identifier(),
19                 '@' => self.single_token(TokenKind::At),
20                 '+' => self.scan_operator(TokenKind::Plus,
21                                         TokenKind::PlusEq),
22                 // ... other cases
23             }
24         }
25
26         fn scan_string_or_triple_quote(&mut self) -> Token {
27             if self.match_prefix("\"\"\"") {
28                 self.scan_triple_quote_string()
29             } else {
30                 self.scan_string()
31             }
32         }
33     }
34 }

```

The `scan_string_or_triple_quote` method distinguishes between regular strings

("...") and triple-quoted strings ("..."") by checking for three consecutive quote characters.

17.3 Parser

The parser constructs an Abstract Syntax Tree (AST) from the token stream using recursive descent with Pratt parsing for expressions.

17.3.1 AST Nodes

The core expression and statement nodes remain unchanged from the base language:

```

1  enum Expr {
2      Literal(Literal),
3      Ident(Ident),
4      Binary(Box<Expr>, BinOp, Box<Expr>),
5      Unary(UnaryOp, Box<Expr>),
6      Call(Box<Expr>, Vec<Expr>),
7      FieldAccess(Box<Expr>, Ident),
8      Index(Box<Expr>, Box<Expr>),
9      If(Box<Expr>, Block, Option<Block>),
10     Match(Box<Expr>, Vec<MatchArm>),
11     Closure(Vec<Param>, Option<Type>, Block),
12     Array(Vec<Expr>),
13     Map(Vec<(Expr, Expr)>),
14     Struct(Ident, Vec<(Ident, Expr)>),
15     Range(Box<Expr>, Box<Expr>, bool),
16     Pipe(Box<Expr>, Box<Expr>),
17 }
18
19 enum Stmt {
20     VarDecl(Vec<Ident>, Option<Type>, Vec<Expr>),
21     Assignment(Vec<LValue>, AssignOp, Vec<Expr>),
22     Expr(Expr),
23     If(Expr, Block, Option<Block>),
24     For(Option<Ident>, Ident, Expr, Block),
25     While(Expr, Block),
26     Match(Expr, Vec<MatchArm>),
27     Return(Option<Vec<Expr>>),
28     Break(Option<Ident>),
29     Continue(Option<Ident>),
30     Defer(Box<Stmt>),
31     Spawn(SpawnTarget),
32     Block(Block),
33 }

```

17.3.2 Agentic AST Nodes

Four new top-level declaration nodes support the agentic constructs:

```

1  struct ProviderDecl {
2      name: Ident,
3      fields: Vec<(Ident, Expr)>,
4  }
5

```

```

6 struct ToolDecl {
7     name: Ident,
8     params: Vec<Param>,
9     return_type: Option<Type>,
10    description: String,          // from triple-quoted string
11    body: Block,
12 }
13
14 struct AgentDecl {
15     name: Ident,
16     fields: Vec<(Ident, AgentFieldValue)>,
17 }
18
19 struct WorkflowDecl {
20     decorators: Vec<Decorator>,
21     name: Ident,
22     params: Vec<Param>,
23     return_type: Option<Type>,
24     body: Block,
25 }
26
27 struct Decorator {
28     name: Ident,                  // e.g., "webhook", "cron",
29     "event"
30     args: Vec<Expr>,             // e.g., "/summarize", "0 9 * *"
31 }

```

17.3.3 Parsing Agentic Constructs

The recursive descent parser is extended to handle the new keywords at the top level:

```

1 impl Parser<'> {
2     fn parse_top_level(&mut self) -> Result<Decl, ParseError> {
3         match self.peek_keyword() {
4             Some(Keyword::Fn)      => self.parse_fn_decl(),
5             Some(Keyword::Struct)  => self.parse_struct_decl(),
6             Some(Keyword::Enum)    => self.parse_enum_decl(),
7             Some(Keyword::Type)    => self.parse_type_decl(),
8             Some(Keyword::Import)  => self.parse_import(),
9             Some(Keyword::Provider) =>
10                 self.parse_provider_decl(),
11             Some(Keyword::Tool)    => self.parse_tool_decl(),
12             Some(Keyword::Agent)   => self.parse_agent_decl(),
13             Some(Keyword::Workflow) =>
14                 self.parse_workflow_decl(vec![]),
15             _ if self.check(TokenKind::At) => {
16                 let decorators = self.parse_decorators()?;
17                 self.expect_keyword(Keyword::Workflow)?;
18                 self.parse_workflow_decl(decorators)
19             }
20             _ =>
21                 Err(ParseError::UnexpectedToken(self.peek().clone())),
22         }
23     }
24
25     fn parse_decorators(&mut self) -> Result<Vec<Decorator>,

```

```

23     ParseError> {
24         let mut decorators = Vec::new();
25         while self.check(TokenKind::At) {
26             self.advance(); // consume @
27             let name = self.parse_ident()?;
28             let args = if self.check(TokenKind::LParen) {
29                 self.parse_call_args()?
30             } else {
31                 vec![]
32             };
33             decorators.push(Decorator { name, args });
34         }
35         Ok(decorators)
36     }
37
38     fn parse_tool_decl(&mut self) -> Result<Decl, ParseError> {
39         self.expect_keyword(Keyword::Tool)?;
40         let name = self.parse_ident()?;
41         let params = self.parse_params()?;
42         let return_type = self.parse_optional_return_type()?;
43         self.expect(TokenKind::LBrace)?;
44         let description = self.expect_triple_quote_string()?;
45         let body = self.parse_block_body()?;
46         Ok(Decl::Tool(ToolDecl { name, params, return_type,
47                               description, body }))
48     }
49 }

```

Implementation Note

The decorator parser greedily consumes all consecutive @-prefixed lines before the **workflow** keyword. This means decorators must appear immediately before the workflow declaration with no intervening blank lines or other declarations.

17.4 Resolver

The resolver performs name resolution, import handling, and symbol binding. It is extended to handle agentic constructs.

17.4.1 Symbol Table

The symbol table is extended with new symbol kinds for agentic constructs:

```

1 struct SymbolTable {
2     scopes: Vec<Scope>,
3     current: usize,
4 }
5
6 struct Scope {
7     symbols: HashMap<String, Symbol>,
8     parent: Option<usize>,
9 }
10
11 enum Symbol {
12     Variable { ty: TypeId, mutable: bool },

```

```

13     Function { sig: FunctionSig },
14     Type { def: TypeDef },
15     Module { exports: HashMap<String, Symbol> },
16     Provider { fields: ProviderFields },
17     Tool { sig: ToolSig, description: String },
18     Agent { fields: AgentFields },
19     Workflow { sig: WorkflowSig, triggers: Vec<Trigger> },
20 }

```

17.4.2 Provider Resolution

The resolver validates provider declarations:

- The `model` field is required.
- The `api_key` field, if present, must be a string expression (typically an `env()` call).
- The `backend` field, if present, must be a recognized backend identifier ("ollama", "llama.cpp").
- Provider names are registered in the module scope for reference by agents.

```

1 fn resolve_provider(&mut self, decl: &ProviderDecl) ->
  Result<(), ResolveError> {
2     // Ensure required fields are present
3     if !decl.has_field("model") {
4         return Err(ResolveError::MissingField {
5             construct: "provider",
6             name: decl.name.clone(),
7             field: "model",
8         });
9     }
10
11     // Register in symbol table
12     self.bind_symbol(
13         &decl.name,
14         Symbol::Provider { fields:
15             self.extract_provider_fields(decl)? },
16     )
17 }

```

17.4.3 Tool Registration

The resolver processes tool declarations and prepares metadata for schema generation:

- The tool name is registered in the module scope.
- Parameter types are recorded for JSON schema generation.
- The triple-quoted description is stored as metadata.
- Default parameter values are validated.

```

1 fn resolve_tool(&mut self, decl: &ToolDecl) -> Result<(),
  ResolveError> {
2     // Description is mandatory (enforced by parser,
      double-checked here)
3     if decl.description.is_empty() {
4         return
          Err(ResolveError::MissingToolDescription(decl.name.clone()));
5     }
6
7     // Resolve parameter types and body
8     self.push_scope();
9     for param in &decl.params {
10         self.resolve_type(&param.ty)?;
11         self.bind_local(&param.name, &param.ty)?;
12     }
13     self.resolve_block(&decl.body)?;
14     self.pop_scope();
15
16     // Register tool symbol
17     self.bind_symbol(
18         &decl.name,
19         Symbol::Tool {
20             sig: self.build_tool_sig(decl)?,
21             description: decl.description.clone(),
22         },
23     )
24 }

```

17.4.4 Agent Validation

The resolver verifies that agent declarations reference valid providers and tools:

- The `model` field must reference a declared provider.
- Each entry in the `tools` list must reference a declared tool.
- The `system` field must be a string expression.
- Memory configuration, if present, must use a recognized memory type.

```

1 fn resolve_agent(&mut self, decl: &AgentDecl) -> Result<(),
  ResolveError> {
2     // Validate model references a declared provider
3     let model_field = decl.get_field("model")
4         .ok_or(ResolveError::MissingField {
5             construct: "agent", name: decl.name.clone(), field:
              "model",
6         })?;
7     self.expect_provider_ref(model_field)?;
8
9     // Validate tool references
10    if let Some(tools) = decl.get_field("tools") {
11        for tool_name in tools.as_ident_list()? {
12            self.expect_tool_ref(&tool_name)?;
13        }
14    }

```

```

15
16     // Register agent symbol
17     self.bind_symbol(
18         &decl.name,
19         Symbol::Agent { fields:
20             self.extract_agent_fields(decl)? },
21     )
22 }

```

17.4.5 Workflow Trigger Validation

The resolver validates workflow decorators:

- `@webhook` requires a string literal path argument.
- `@cron` requires a valid cron expression string.
- `@event` requires a string literal event name.
- `@websocket` requires a string literal path argument.
- `@manual` takes no arguments.
- Unrecognized decorator names produce an error.

```

1 fn resolve_workflow(&mut self, decl: &WorkflowDecl) ->
2   Result<(), ResolveError> {
3     // Validate each decorator
4     let mut triggers = Vec::new();
5     for decorator in &decl.decorators {
6         let trigger = match decorator.name.as_str() {
7             "webhook" =>
8                 self.validate_webhook_trigger(decorator)?,
9             "websocket" =>
10                 self.validate_websocket_trigger(decorator)?,
11             "cron" =>
12                 self.validate_cron_trigger(decorator)?,
13             "event" =>
14                 self.validate_event_trigger(decorator)?,
15             "manual" =>
16                 self.validate_manual_trigger(decorator)?,
17             unknown => return
18                 Err(ResolveError::UnknownTrigger {
19                     name: unknown.to_string(),
20                     span: decorator.span,
21                 }),
22         };
23         triggers.push(trigger);
24     }
25
26     // Resolve params and body
27     self.push_scope();
28     for param in &decl.params {
29         self.resolve_type(&param.ty)?;
30         self.bind_local(&param.name, &param.ty)?;
31     }
32     self.resolve_block(&decl.body)?;

```

```

26     self.pop_scope();
27
28     // Register workflow symbol
29     self.bind_symbol(
30         &decl.name,
31         Symbol::Workflow {
32             sig: self.build_workflow_sig(decl)?,
33             triggers,
34         },
35     )
36 }

```

17.5 Type Checker

The type checker verifies type correctness and performs type inference. It is extended to handle the types and patterns introduced by agentic constructs.

17.5.1 Type Representation

The type system includes a new `stream<T>` type for streaming agent responses:

```

1  enum Type {
2      // Primitives
3      Int, I8, I16, I32, I64,
4      UInt, U8, U16, U32, U64,
5      Float, F32, F64,
6      Bool,
7      String,
8
9      // Composites
10     Array(Box<Type>),
11     Map(Box<Type>, Box<Type>),
12     Tuple(Vec<Type>),
13     Option(Box<Type>),
14
15     // Named
16     Struct(StructId),
17     Enum(EnumId),
18
19     // Functions
20     Function(Vec<Type>, Box<Type>),
21
22     // Generics
23     TypeVar(TypeVarId),
24     Generic(GenericId, Vec<Type>),
25
26     // Streaming
27     Stream(Box<Type>),          // stream<T>
28
29     // Special
30     Never,
31     Error,
32 }

```


17.5.2 Agent Method Type Checking

Agent methods have specific type signatures that the type checker enforces:

- `Agent.ask(prompt: string, session: string?) -> (string, Error?)` — Text in, text out.
- `Agent.run(input: T, session: string?) -> (U, Error?)` — Structured in, structured out. The output type `U` is inferred from the left-side type annotation at the call site.
- `Agent.stream(prompt: string, session: string?) -> stream<string>` — Text in, streaming out.

```

1 fn check_agent_method_call(
2     &mut self,
3     agent: &Symbol,
4     method: &str,
5     args: &[Expr],
6     expected_return: Option<&Type>,
7 ) -> Result<Type, TypeError> {
8     match method {
9         "ask" => {
10             self.check_arg_type(&args[0], &Type::String)?;
11             Ok(Type::Tuple(vec![Type::String,
12                                 Type::Option(Box::new(Type::Error))]))
13         }
14         "run" => {
15             let input_ty = self.infer_expr(&args[0])?;
16             self.check_has_json_schema(&input_ty)?;
17
18             // Output type inferred from left-side annotation
19             let output_ty = expected_return
20                 .ok_or(TypeError::MissingTypeAnnotation {
21                     hint: "agent.run() requires a type
22                         annotation on the left side",
23                 })?;
24             self.check_has_json_schema(output_ty)?;
25
26             Ok(Type::Tuple(vec![
27                 output_ty.clone(),
28                 Type::Option(Box::new(Type::Error)),
29             ]))
30         }
31         "stream" => {
32             self.check_arg_type(&args[0], &Type::String)?;
33             Ok(Type::Stream(Box::new(Type::String)))
34         }
35         _ => Err(TypeError::NoSuchMethod {
36             ty: "agent",
37             method: method.to_string(),
38         }),
39     }
40 }
41 }
```

17.5.3 Left-Side Type Annotation Inference

For `Agent.run()`, the output type is determined by the type annotation on the left side of the assignment:

```

1 // The compiler infers that Researcher.run() should produce
   ResearchResult
2 research: ResearchResult, err = Researcher.run(ResearchRequest{
3   topic: "quantum computing"
4 })

```

The type checker extracts `ResearchResult` from the left-side annotation and uses it as the expected output type. It then validates that both `ResearchRequest` and `ResearchResult` are struct types with JSON-serializable fields, so that the runtime can generate and validate JSON schemas.

17.5.4 Workflow Input/Output Types

Workflow parameters define the input type. For triggered workflows, the compiler validates that parameter and return types are JSON-serializable:

```

1 fn check_workflow(&mut self, decl: &WorkflowDecl) -> Result<(),
   TypeError> {
2   // For triggered workflows, params must be JSON-serializable
3   if !decl.decorators.is_empty() {
4     for param in &decl.params {
5       self.check_json_serializable(&param.ty)?;
6     }
7     if let Some(ret_ty) = &decl.return_type {
8       self.check_json_serializable(ret_ty)?;
9     }
10  }
11
12  // Check body
13  self.push_scope();
14  for param in &decl.params {
15    self.bind_local(&param.name, &param.ty);
16  }
17  let body_ty = self.check_block(&decl.body)?;
18
19  // Verify return type matches
20  if let Some(ret_ty) = &decl.return_type {
21    self.unify(&body_ty, ret_ty)?;
22  }
23  self.pop_scope();
24
25  Ok(())
26 }

```

17.5.5 Tool Auto-Schema Generation

The type checker produces JSON schema metadata for each tool declaration. This metadata is embedded in the compiled binary and used by the runtime when registering tools with LLM providers:

```

1 fn generate_tool_schema(&self, decl: &ToolDecl) -> ToolSchema {
2     ToolSchema {
3         name: decl.name.clone(),
4         description: decl.description.clone(),
5         parameters: JsonSchema::Object {
6             properties: decl.params.iter().map(|p| {
7                 (p.name.clone(),
8                  self.type_to_json_schema(&p.ty))
9             }).collect(),
10        required: decl.params.iter()
11            .filter(|p| p.default.is_none())
12            .map(|p| p.name.clone())
13            .collect(),
14    },
15 }

```

The type-to-schema mapping is:

Haira Type	JSON Schema Type
int	"integer"
float	"number"
bool	"boolean"
string	"string"
[T]	"array" with items
Struct	"object" with properties
T?	adds "null" to type union

Table 17.1: Type-to-JSON-schema mapping

17.6 Lowering

Lowering transforms the typed AST into HIR (High-level IR). Agentic constructs are lowered to calls into the HAIRA runtime library.

17.6.1 HIR Structure

```

1 struct HirModule {
2     functions: Vec<HirFunction>,
3     types: Vec<HirType>,
4     constants: Vec<HirConst>,
5     providers: Vec<HirProvider>,
6     tools: Vec<HirTool>,
7     agents: Vec<HirAgent>,
8     workflows: Vec<HirWorkflow>,
9 }
10
11 struct HirFunction {
12     name: Symbol,
13     params: Vec<HirParam>,
14     return_type: HirType,
15     body: Vec<HirStmt>,
16     locals: Vec<HirLocal>,

```

```

17 }
18
19 enum HirStmt {
20     Assign(LocalId, HirExpr),
21     Store(HirPlace, HirExpr),
22     Call(Option<LocalId>, FnId, Vec<HirExpr>),
23     RuntimeCall(Option<LocalId>, RuntimeFn, Vec<HirExpr>),
24     If(HirExpr, BlockId, Option<BlockId>),
25     Loop(BlockId),
26     Break(Option<LoopId>),
27     Continue(Option<LoopId>),
28     Return(Option<HirExpr>),
29 }

```

17.6.2 Provider Lowering

A **provider** declaration is lowered to a runtime configuration call:

```

1 // Source:
2 //   provider anthropic {
3 //       api_key: env("ANTHROPIC_API_KEY")
4 //       model: "claude-sonnet-4-20250514"
5 //   }
6
7 // Lowered to:
8 HirStmt::RuntimeCall(
9     Some(provider_local),
10    RuntimeFn::RegisterProvider,
11    vec![
12        HirExpr::StringLit("anthropic"),
13        HirExpr::Map(vec![
14            ("api_key", HirExpr::Call(env_fn,
15                                     vec![HirExpr::StringLit("ANTHROPIC_API_KEY")])),
16            ("model",
17             HirExpr::StringLit("claude-sonnet-4-20250514")),
18        ]),
19    ],
20 )

```

17.6.3 Tool Lowering

A **tool** declaration is lowered to a runtime tool registration that includes the compiled function body and the auto-generated JSON schema:

```

1 // Source:
2 //   tool search(query: string, max_results: int = 5) ->
3 //       [SearchResult] {
4 //           ""Search the web for information""
5 //           // ... body ...
6 //       }
7
8 // Lowered to:
9 // 1. The tool body becomes a regular HirFunction
10 HirFunction {
11     name: Symbol("__tool_search"),

```

```

11     params: vec![param_query, param_max_results],
12     return_type: array_of_search_result,
13     body: /* lowered body */,
14 }
15
16 // 2. A runtime registration call with the JSON schema
17 HirStmt::RuntimeCall(
18     None,
19     RuntimeFn::RegisterTool,
20     vec![
21         HirExpr::StringLit("search"),
22         HirExpr::FnRef(Symbol("__tool_search")),
23         HirExpr::StaticData(tool_json_schema), // compiled
           schema bytes
24     ],
25 )

```

17.6.4 Agent Lowering

An **agent** declaration is lowered to a runtime agent instantiation:

```

1 // Source:
2 //   agent Researcher {
3 //       model: anthropic
4 //       system: "You are a thorough researcher."
5 //       tools: [search, read_url]
6 //       temperature: 0.2
7 //       max_steps: 15
8 //   }
9
10 // Lowered to:
11 HirStmt::RuntimeCall(
12     Some(agent_local),
13     RuntimeFn::CreateAgent,
14     vec![
15         HirExpr::StringLit("Researcher"),
16         HirExpr::ProviderRef("anthropic"),
17         HirExpr::StringLit("You are a thorough researcher."),
18         HirExpr::Array(vec![
19             HirExpr::ToolRef("search"),
20             HirExpr::ToolRef("read_url"),
21         ]),
22         HirExpr::Map(vec![
23             ("temperature", HirExpr::FloatLit(0.2)),
24             ("max_steps", HirExpr::IntLit(15)),
25         ]),
26     ],
27 )

```

17.6.5 Agent Method Call Lowering

Agent method calls are lowered to runtime API calls:

```

1 // Agent.ask() --> runtime ask call
2 // answer, err = Researcher.ask("query")

```

```

3  HirStmt::RuntimeCall(
4      Some(result_local),
5      RuntimeFn::AgentAsk,
6      vec![
7          HirExpr::AgentRef("Researcher"),
8          HirExpr::StringLit("query"),
9          HirExpr::Nil, // no session
10     ],
11 )
12
13 // Agent.run() --> runtime run call with output schema
14 // result: T, err = Agent.run(input)
15 HirStmt::RuntimeCall(
16     Some(result_local),
17     RuntimeFn::AgentRun,
18     vec![
19         HirExpr::AgentRef("Agent"),
20         input_expr,
21         HirExpr::StaticData(input_schema),
22         HirExpr::StaticData(output_schema),
23         HirExpr::Nil, // no session
24     ],
25 )
26
27 // Agent.stream() --> runtime stream call
28 // for chunk in Agent.stream("prompt") { ... }
29 HirStmt::RuntimeCall(
30     Some(stream_local),
31     RuntimeFn::AgentStream,
32     vec![
33         HirExpr::AgentRef("Agent"),
34         HirExpr::StringLit("prompt"),
35         HirExpr::Nil, // no session
36     ],
37 )

```

17.6.6 Workflow Lowering

Workflows are lowered differently based on their triggers:

```

1  // @webhook("/summarize")
2  // workflow Summarizer(url: string) -> Summary { ... }
3
4  // Lowered to:
5  // 1. The workflow body becomes a regular function
6  HirFunction {
7      name: Symbol("__workflow_Summarizer"),
8      params: vec![param_url],
9      return_type: summary_type,
10     body: /* lowered body */,
11 }
12
13 // 2. A runtime trigger registration
14 HirStmt::RuntimeCall(
15     None,
16     RuntimeFn::RegisterWebhook,
17     vec![

```

```

18     HirExpr::StringLit("/summarize"),
19     HirExpr::StringLit("POST"),
20     HirExpr::FnRef(Symbol("__workflow_Summarizer")),
21     HirExpr::StaticData(input_schema),
22     HirExpr::StaticData(output_schema),
23   ],
24 )

```

Trigger	Runtime Registration
@webhook	RuntimeFn::RegisterWebhook — HTTP handler
@websocket	RuntimeFn::RegisterWebsocket — WebSocket handler
@cron	RuntimeFn::RegisterCron — Cron job
@event	RuntimeFn::RegisterEvent — Event listener
@manual	RuntimeFn::RegisterManual — CLI-only
(none)	No registration — callable as sub-workflow

Table 17.2: Workflow trigger lowering

17.6.7 Spawn Block Lowering

A **spawn** block inside a workflow is lowered to parallel execution runtime calls:

```

1  // Source:
2  //   reviews = spawn {
3  //       Reviewer.run(req1)
4  //       Reviewer.run(req2)
5  //       Reviewer.run(req3)
6  //   }
7
8  // Lowered to:
9  HirStmt::RuntimeCall(
10     Some(reviews_local),
11     RuntimeFn::SpawnAll,
12     vec![
13         HirExpr::Array(vec![
14             HirExpr::Closure(/* Reviewer.run(req1) */),
15             HirExpr::Closure(/* Reviewer.run(req2) */),
16             HirExpr::Closure(/* Reviewer.run(req3) */),
17         ]),
18     ],
19 )

```

17.7 Code Generation

Code generation uses Cranelift to produce native machine code.

17.7.1 Cranelift Integration

The code generation stage translates HIR into Cranelift IR, which is then compiled to native code. The process is the same as for non-agentic code — function bodies, control flow, and expressions are translated directly:

```

1 fn codegen_function(&mut self, func: &HirFunction) ->
  Result<(), CodegenError> {
2   let mut builder = FunctionBuilder::new(&mut self.ctx.func,
     &mut self.builder_ctx);
3
4   // Create entry block
5   let entry = builder.create_block();
6   builder.append_block_params_for_function_params(entry);
7   builder.switch_to_block(entry);
8
9   // Generate parameters
10  for (i, param) in func.params.iter().enumerate() {
11    let val = builder.block_params(entry)[i];
12    self.locals.insert(param.id, val);
13  }
14
15  // Generate body
16  for stmt in &func.body {
17    self.codegen_stmt(&mut builder, stmt)?;
18  }
19
20  builder.seal_all_blocks();
21  builder.finalize();
22
23  Ok(())
24 }

```

17.7.2 Runtime Calls

Agentic constructs compile to calls into the HAIRA runtime library (`libhaira-runtime`). The code generator emits standard function calls to runtime symbols:

```

1 fn codegen_runtime_call(
2   &mut self,
3   builder: &mut FunctionBuilder,
4   func: RuntimeFn,
5   args: &[HirExpr],
6 ) -> Value {
7   let runtime_fn = self.import_runtime_function(func);
8   let arg_values: Vec<Value> = args.iter()
9     .map(|a| self.codegen_expr(builder, a))
10    .collect();
11   let call = builder.ins().call(runtime_fn, &arg_values);
12   builder.inst_results(call)[0]
13 }

```

The compiled binary links against `libhaira-runtime`, which provides the implementations for all agentic operations (LLM API calls, tool execution, workflow orchestration, etc.). See Section 17.8 for details.

Note

All agentic behavior happens at *runtime*, not compile time. The compiler's role is to validate, type-check, and generate schemas for agentic constructs, then emit calls

into the runtime library. The compiled binary is standalone but requires network access to reach LLM providers.

17.8 Runtime Library

The HAIRA runtime library (`libhaira-runtime`) provides the execution environment for agentic constructs. It is linked into every compiled HAIRA binary that uses providers, tools, agents, or workflows.

17.8.1 Provider Management

The runtime manages provider connections:

- **API key resolution** — Reads API keys from environment variables at startup.
- **Model selection** — Routes requests to the correct LLM backend based on provider configuration.
- **Connection pooling** — Maintains HTTP connection pools to provider APIs.
- **Rate limiting** — Handles provider rate limits with automatic retry and backoff.
- **Local backends** — Manages connections to local inference servers (Ollama, llama.cpp).

17.8.2 Agent Execution

The runtime implements the agent execution loop:

1. Assemble the message payload (system prompt, conversation history, user message).
2. Send the request to the LLM provider.
3. If the LLM response contains tool calls, execute each tool and collect results.
4. Send tool results back to the LLM.
5. Repeat steps 2–4 until the LLM produces a final response or `max_steps` is reached.
6. For `.run()` calls, parse the LLM response as structured JSON and deserialize into the expected output type.
7. For `.stream()` calls, yield tokens as they arrive from the LLM.

17.8.3 Workflow Orchestration

The runtime provides infrastructure for workflow triggers:

- **HTTP server** — An embedded HTTP server (built on a lightweight async runtime) handles `@webhook` and `@websocket` triggers. Request bodies are deserialized into workflow parameter types; return values are serialized as JSON responses.
- **Cron scheduler** — A built-in cron scheduler executes `@cron`-triggered workflows on schedule.
- **Event bus** — An in-process event bus dispatches `@event`-triggered workflows when events are emitted via `haira.emit()`.
- **Manual runner** — The CLI can invoke `@manual` workflows directly with JSON input.

17.8.4 Session Management

The runtime manages conversation memory scoped by session ID:

- Each session maintains an isolated conversation history.
- `conversation(max_turns: N)` memory trims history to the last N message pairs.
- `summary(max_tokens: N)` memory periodically summarizes older messages to stay within the token budget.
- Sessions are stored in-memory by default, with optional persistence backends.

17.8.5 Stream Handling

The runtime provides async token streaming for `Agent.stream()`:

- Server-sent events (SSE) from LLM providers are parsed incrementally.
- Tokens are yielded to the caller via an async iterator interface.
- For `@websocket` workflows returning `stream<string>`, tokens are forwarded directly to the WebSocket connection.

17.8.6 JSON Schema Generation

The runtime uses type metadata embedded by the compiler to:

- Register tool schemas with LLM providers during tool-calling.
- Instruct agents to produce structured output matching a given schema (for `.run()` calls).
- Validate and deserialize incoming webhook request bodies.
- Serialize workflow return values as JSON responses.

17.9 Project Structure

The compiler and runtime are organized as a Rust workspace with the following crates:

```

haira/
  Cargo.toml                # Workspace root

  crates/
    haira-lexer/             # Lexical analysis
    haira-parser/            # Parsing (+ provider, tool,
      agent, workflow)
    haira-ast/               # AST definitions (+ agentic
      nodes)
    haira-resolver/          # Name resolution (+
      provider/agent/tool resolution)
    haira-types/             # Type system (+ stream<T>)
    haira-typeck/            # Type checking
    haira-hir/               # High-level IR
    haira-codegen/           # Code generation (Cranelift)
    haira-runtime/           # Runtime library (agents,
      workflows, providers)

```

haira-driver/	# Compilation driver
haira-cli/	# CLI (build, run, serve, check)
haira-lsp/	# Language server

Note

The `haira-ai`, `haira-cir`, and `haira-local-ai` crates from previous versions have been removed. Their responsibilities are replaced by `haira-runtime`, which handles all agentic execution at runtime rather than compile time.

Each crate has a focused responsibility:

Crate	Responsibility
haira-lexer	Tokenizes source text
haira-parser	Builds AST from tokens
haira-ast	Defines all AST node types
haira-resolver	Resolves imports, symbols, providers, tools, agents
haira-types	Type definitions and type utilities
haira-typeck	Type checking and inference
haira-hir	High-level IR definitions
haira-codegen	Cranelift code generation
haira-runtime	Runtime library for agentic execution
haira-driver	Orchestrates the compilation pipeline
haira-cli	Command-line interface
haira-lsp	Language server protocol implementation

Table 17.3: Crate responsibilities

17.10 Build Process

The `haira` CLI provides the following commands:

```
# Compile to native binary
haira build main.haira

# Compile and run immediately
haira run main.haira

# Compile and start server (for trigger-based workflows)
haira serve main.haira

# Type-check without code generation
haira check main.haira

# Interactive REPL
haira repl
```

17.10.1 Build Modes

```
# Debug build (default) --- fast compilation, includes debug
info
haira build main.haira

# Release build --- optimized, slower compilation
haira build main.haira --release
```

17.10.2 Server Mode

The `haira serve` command compiles the program and starts the runtime server, which listens for webhook requests, WebSocket connections, cron schedules, and events:

```
# Start server on default port (8080)
haira serve main.haira

# Start server on a specific port
haira serve main.haira --port 3000

# Start with environment variables
ANTHROPIC_API_KEY=sk-... haira serve main.haira
```

17.10.3 Check Mode

The `haira check` command runs the full pipeline up to and including type checking, but skips code generation. This is useful for fast feedback during development:

```
haira check main.haira
```

It validates all agentic constructs — provider fields, tool descriptions, agent references, workflow triggers, and type compatibility — without producing a binary.

17.11 Error Messages

The compiler produces detailed, contextual error messages. Errors related to agentic constructs follow the same format as core language errors.

17.11.1 Missing Tool Description

```
error[E0101]: tool is missing a description
--> src/tools.haira:5:1
|
5 | tool search(query: string) -> [Result] {
| ~~~~~ tool declaration
6 |     resp = http.get("https://api.search.com?q={query}")
|     ---- expected a """"..."""" description as the first
    element
|
= help: add a triple-quoted description: """"Search the web
      for information""""
= note: tool descriptions are sent to the LLM and cannot be
      omitted
```

17.11.2 Agent Referencing Undefined Provider

```

error[E0102]: undefined provider 'openai'
--> src/agents.haira:8:12
  |
8 |         model: openai
  |               ~~~~~~ not found in this scope
  |
= help: did you mean 'anthropic'?
= note: declare a provider with: provider openai { model:
      "gpt-4o" ... }

```

17.11.3 Workflow with Invalid Trigger

```

error[E0103]: unknown trigger '@daily'
--> src/workflows.haira:12:1
  |
12 | @daily
  | ~~~~~~ not a recognized trigger
  |
= help: valid triggers are: @webhook, @websocket, @cron,
      @event, @manual
= help: for daily execution, use: @cron("0 0 * * *")

```

17.11.4 Type Mismatch in Agent.run() Output

```

error[E0104]: type mismatch in agent.run() output
--> src/main.haira:20:5
  |
20 |         result: string, err = Analyst.run(request)
  |               ~~~~~~ expected a struct type, found 'string'
  |
= note: agent.run() requires a struct type annotation for
      structured output
= help: define a struct for the output:
      struct AnalysisResult { summary: string, score:
        float }
      result: AnalysisResult, err = Analyst.run(request)

```

17.11.5 Agent Referencing Undefined Tool

```

error[E0105]: undefined tool 'search_web'
--> src/agents.haira:10:18
  |
10 |         tools: [search_web, read_url]
  |                ~~~~~~ not found in this scope
  |
= help: did you mean 'search'?
= note: tools must be declared with the 'tool' keyword
      before being

```

```
referenced in an agent
```

17.11.6 Invalid Cron Expression

```
error[E0106]: invalid cron expression
--> src/workflows.haira:3:7
   |
3  | @cron("every day")
   |      ^^^^^^^^^^^ not a valid cron expression
   |
= help: cron expressions use the format: minute hour
        day-of-month month day-of-week
= help: example: @cron("0 9 * * *") for every day at 9 AM
```

Appendix A

Reserved Keywords

The following identifiers are reserved as keywords and cannot be used as identifiers:

agent	and	as	break	catch
chan	const	continue	defer	else
enum	false	fn	for	from
if	impl	import	in	match
nil	not	or	provider	return
select	spawn	struct	tool	true
try	type	unsafe	where	while
workflow				

Appendix B

Operator Precedence

Operators are listed from highest to lowest precedence:

Precedence	Operators	Associativity
1 (highest)	() [] .	Left
2	! - ~ (unary) not	Right
3	* / %	Left
4	+ -	Left
5	<< >> >>>	Left
6	& (bitwise AND)	Left
7	^ (bitwise XOR)	Left
8	(bitwise OR)	Left
9	=	None
10	> (pipe)	Left
11	< > <= >=	Left
12	== !=	Left
13	and &&	Left
14	or	Left
15 (lowest)	= += -= *= /= &= = ^= <= >= >>=	Right

Appendix C

Standard Library Reference

Quick reference for standard library modules. See Chapter 10 for full documentation.

Module	Description
<code>io</code>	Input/output operations
<code>string</code>	String manipulation
<code>math</code>	Mathematical functions
<code>array</code>	Array operations
<code>map</code>	Map/dictionary operations
<code>json</code>	JSON encoding/decoding
<code>http</code>	HTTP client/server
<code>fs</code>	File system operations
<code>os</code>	Operating system interface
<code>time</code>	Time and duration
<code>crypto</code>	Cryptographic functions
<code>regex</code>	Regular expressions
<code>net</code>	Network primitives
Tool Packs (importable tools for agents)	
<code>tools/http</code>	HTTP tools (get, post, put, delete)
<code>tools/slack</code>	Slack integration (send, read)
<code>tools/github</code>	GitHub API (create_issue, list_prs)
<code>tools/postgres</code>	PostgreSQL (query, insert)
<code>tools/email</code>	Email sending
<code>tools/fs</code>	File system tools for agents